MICROSOFT SQL SERVER & T-SQL

FROM BEGINNER TO ADVANCED

A Comprehensive, Professional Reference Handbook

Built on PostgreSQL/MySQL Foundations — SQL Server Architecture, T-SQL, Performance, Administration, Analytics, Data Engineering, and Enterprise Development

Kajola Gbenga Data Scientist & AI Engineer

Table of Contents

1. SQL Server Fundamentals
2. SQL Server Tools
3. Database and Schema Management
4. SQL Server Data Types
5. Table Design
6. CRUD Operations
7. SELECT and T-SQL Querying
8. Operators and Expressions
9. T-SQL Functions
10. Conditional Logic
11. Joins
12. Subqueries
13. CTEs
14. Window Functions
15. Temporary Objects
16. APPLY Operators
17. PIVOT and UNPIVOT
18. Set Operations
19. Views
20. Stored Procedures
21. User-Defined Functions
22. Triggers
23. Transactions and ACID
24. Transaction Isolation
25. Error Handling
26. Indexes
27. Columnstore
28. Query Performance
29. Explaining Execution Plans
30. Query Optimization
31. SQL Server JSON
32. XML
33. Data Cleaning With T-SQL
34. Database Design and Normalization
35. SQL Server Security
36. Dynamic SQL
37. Backup and Restore
38. SQL Server Agent
39. Data Import and Export
40. tempdb

41. SQL Server System Catalog
42. Temporal Tables
43. Change Data Capture and Change Tracking
44. Partitioning
45. High Availability
46. Replication
47. SQL Server for Data Analytics
48. SQL Server for Data Science
49. Python + SQL Server
50. SQL Server for Data Engineering
51. SQL Server Data Warehousing
52. SQL Server + Power BI
53. SQL Server for Backend Development
54. SQL Server for AI and Machine Learning
55. PostgreSQL vs MySQL vs SQL Server
56. SQL Portability
57. Real-World SQL Server Projects
58. Complete SQL Server Capstone
59. Interview Preparation
60. SQL Server Coding Challenges
61. Common SQL Server Mistakes
62. Professional Best Practices
63. SQL Server Cheat Sheet

1. SQL Server Fundamentals

1.1 What SQL Server Is

Microsoft SQL Server is a relational database management system (RDBMS) developed by Microsoft, running primarily on Windows and (since SQL Server 2017) Linux and Docker containers. Like PostgreSQL and MySQL, it stores data in tables, enforces relationships and constraints, and is queried with SQL — but it is built around Microsoft's own database engine, tooling ecosystem, and a SQL dialect called **T-SQL (Transact-SQL)**.

1.2 SQL Server vs SQL vs T-SQL

Term	What It Is
SQL	The ANSI/ISO-standardized query language itself — `SELECT`, `INSERT`, `JOIN`, and so on, common in some form across virtually every RDBMS.
T-SQL (Transact-SQL)	Microsoft's proprietary extension of SQL, adding procedural constructs (`BEGIN/END`, `WHILE`, variables, `TRY/CATCH`), SQL Server-specific functions, and syntax found nowhere else (`TOP`, `OUTPUT`, `APPLY`, `MERGE`).
SQL Server	The actual database engine/product that executes T-SQL.

Throughout this handbook, every syntax block is labeled **[Standard SQL]**, **[T-SQL]**, or **[SQL Server Feature]** so you always know whether something is portable or Microsoft-specific — directly relevant since you already know PostgreSQL and MySQL.

1.3 SQL Server Architecture

At a high level, SQL Server consists of the **Database Engine** (the core service that stores, processes, and secures data — handling query execution, transactions, and storage), the **SQL Server Agent** (a job scheduler, Part 38), and various services (Analysis Services, Integration Services, Reporting Services) not covered in depth here since this handbook focuses on the relational engine and T-SQL.

1.4 SQL Server Instances

An **instance** is a single running installation of the SQL Server Database Engine — one physical or virtual machine can host multiple independent instances (a **default instance** and any number of **named instances**), each with its own set of databases, logins, and configuration, fully isolated from one another. This has no direct equivalent in a typical PostgreSQL/MySQL setup, where "one server process, one port" is more common; SQL Server instances are more like several independent PostgreSQL clusters coexisting on one machine, each addressable as `ServerName\InstanceName`.

1.5 Core Object Hierarchy

```
SQL Server Instance
    └── Database
            └── Schema
                    └── Objects (Tables, Views, Procedures, Functions,
Triggers)
                            └── Rows / Columns
```

This is the single most important structural concept to internalize coming from PostgreSQL (which has an identical `database → schema → object` hierarchy) or MySQL (where, notably, a MySQL "database" and "schema" are actually the same thing — there is no separate schema layer). In SQL Server, exactly as in PostgreSQL, a schema is a genuine namespace *within* a database — `Sales.Orders` and `HR.Orders` can coexist as two entirely different tables in the same database.

1.6 Server vs Instance vs Database vs Schema vs Table vs User vs Login

Term	Meaning
Server	The physical or virtual machine (or cloud resource) hosting SQL Server.
Instance	One installation of the Database Engine on that server (Section 1.4).
Database	A container for schemas, tables, and other objects — the unit typically backed up, restored, and secured independently (Part 3).
Schema	A namespace within a database grouping related objects (e.g. `Sales`, `HR`).
Table	A structured collection of rows and columns within a schema.
Login	A server-level security principal — an identity that can connect to the *instance* (Part 35).
User	A database-level security principal, typically mapped to a login, granting access *within a specific database*.

This login/user distinction is a genuinely SQL Server-specific (and SQL Server-important) concept — PostgreSQL uses a single unified "role" for both server- and database-level access, and MySQL ties privileges directly to a `user@host` combination without this separate two-tier model. Covered fully in Part 35.

1.7 Databases, Tables, Views, Stored Procedures, Functions, Triggers, Indexes, Constraints, Transactions

These core concepts — tables, views, stored procedures, functions, triggers, indexes, constraints, and transactions — are already familiar from PostgreSQL/MySQL and are not re-taught from scratch here; each gets its own dedicated, SQL Server-focused part later in this

handbook (Parts 5, 11-25) where the specific T-SQL syntax and SQL Server behavior are covered in depth.

1.8 System Databases

Every SQL Server instance automatically includes four system databases, distinct from any user-created database:

Database	Purpose
master	Records instance-level metadata: logins, configuration, and the existence of every other database. Never store application data here.
model	A template database — every new database is created as a copy of `model`'s initial structure and settings.
msdb	Used by SQL Server Agent (Part 38) to store jobs, schedules, alerts, and backup/restore history.
tempdb	A shared scratch database for temporary tables, sorting, hashing, and other intermediate work — recreated fresh on every server restart (covered fully in Part 40).

Neither PostgreSQL nor MySQL has a direct equivalent to this specific four-database system layer — PostgreSQL has a `postgres` maintenance database and internal catalogs, MySQL has `information_schema`/`mysql` system databases, but SQL Server's `model`/`msdb`/`tempdb` split, and especially `tempdb`'s central role in query execution (Part 40), is distinctly its own architecture worth learning deliberately.

1.9 User Databases

A **user database** is any database you create to hold actual application data — `CREATE DATABASE Sales;` (Part 3) — logically and physically separate from the four system databases above.

---

2. SQL Server Tools

2.1 SQL Server Management Studio (SSMS)

SSMS is Microsoft's full-featured, Windows-only graphical client for SQL Server — the closest equivalent to pgAdmin (PostgreSQL) or MySQL Workbench (MySQL), providing Object Explorer (a tree view of every database, table, and object), a Query Editor, execution-plan visualization (Part 29), and administrative tooling (backups, security, jobs) in one application.

2.2 Azure Data Studio

Azure Data Studio is a modern, cross-platform (Windows/macOS/Linux), lightweight alternative to SSMS, built on the same editor foundation as VS Code — notebooks, Git integration, and extensions are first-class, at the cost of some of SSMS's deeper administrative GUI tooling. A reasonable default choice if you're not on Windows or prefer a VS Code-like experience.

2.3 VS Code SQL Tooling

VS Code, with the "SQL Server (mssql)" extension, provides connection management, query execution, and IntelliSense directly inside the editor many developers already use daily for application code — a practical choice when SQL Server work is one part of a broader development workflow rather than the primary focus.

2.4 sqlcmd

```
sqlcmd -S localhost -d Sales -U sa -P YourPassword -Q "SELECT COUNT(*) FROM dbo.Orders;"
```

`sqlcmd` is SQL Server's command-line query tool — the direct equivalent of `psql` (PostgreSQL) or the `mysql` CLI client — essential for scripting, automation, and any environment without a GUI (containers, CI pipelines, remote servers).

Flag	Meaning
`-S`	Server/instance to connect to.
`-d`	Target database.
`-U / -P`	SQL Server Authentication username/password (Section 2.6).
`-E`	Use Windows Authentication instead of `-U/-P`.
`-Q`	Run a single query and exit (vs `-i` for a script file).

2.5 SQL Server Configuration Manager

A Windows-specific administrative tool for managing SQL Server *services* (starting/stopping the engine and Agent), configuring network protocols (TCP/IP, named pipes), and setting the port an instance listens on — administrative, instance-level configuration rather than day-to-day query work.

2.6 Authentication: Windows vs SQL Server Authentication

Mode	How It Works
Windows Authentication	Uses the currently logged-in Windows/Active Directory identity; no separate SQL Server password to manage. The default and generally recommended mode on Windows-based enterprise networks.

Mode	How It Works
SQL Server Authentication	A traditional username/password stored and verified by SQL Server itself, independent of the OS — required for non-Windows clients, cross-platform apps, and cloud scenarios (Azure SQL).

This dual-authentication-mode design is distinctly SQL Server's own; PostgreSQL and MySQL each have their own separate authentication method configuration (`pg_hba.conf`, MySQL's plugin-based auth) but neither maps cleanly onto "Windows identity vs database-native credential" as a first-class, built-in choice.

2.7 Connecting, Creating Databases and Users, Running and Saving Queries

```
-- [T-SQL] Connect via SSMS/Azure Data Studio: enter Server name,
choose auth mode, connect.
-- Then, in a new Query Editor window targeting the desired database:

CREATE DATABASE Sales;
GO

USE Sales;
GO

CREATE LOGIN app_user WITH PASSWORD = 'Str0ng!Passw0rd';
CREATE USER app_user FOR LOGIN app_user;
```

`GO` is a **batch separator** recognized by SSMS/`sqlcmd` (not a real T-SQL statement, and not sent to the server itself) — it tells the client tool "send everything above this point as one batch, then start a new batch." This matters because certain statements (like `CREATE DATABASE` followed immediately by `USE`) must be in separate batches. Query scripts are saved as `.sql` files, versioned in source control exactly like application code — a professional habit worth establishing from the start (Part 62).

3. Database and Schema Management

3.1 CREATE DATABASE / ALTER DATABASE / DROP DATABASE

```
-- [Standard SQL, SQL Server extends it heavily]
CREATE DATABASE Sales;
GO

ALTER DATABASE Sales SET RECOVERY SIMPLE;
GO

DROP DATABASE Sales;
```

The base `CREATE DATABASE` syntax is portable across PostgreSQL/MySQL/SQL Server; SQL Server's `ALTER DATABASE` gains many database-wide options (recovery model, compatibility level, file management) that PostgreSQL and MySQL either don't expose the same way or handle via separate configuration.

3.2 CREATE SCHEMA / ALTER SCHEMA / DROP SCHEMA

```
CREATE SCHEMA Sales;
GO
CREATE TABLE Sales.Orders (OrderID INT PRIMARY KEY, Amount
DECIMAL(10,2));

ALTER SCHEMA HR TRANSFER Sales.Orders;   -- moves the table into a
different schema
DROP SCHEMA Sales;
```

[T-SQL note for MySQL users]: MySQL has no independent schema layer — `CREATE SCHEMA` in MySQL is just a synonym for `CREATE DATABASE`. In SQL Server (like PostgreSQL), a schema is a true sub-container inside a database; a single database routinely holds many schemas (`Sales`, `HR`, `dbo` the default schema) each organizing a related set of objects.

3.3 Database Files — MDF, NDF, LDF

File Type	Extension	Purpose
Primary data file	`.mdf`	Holds the database's core data and system tables; every database has exactly one.
Secondary data file(s)	`.ndf`	Optional additional data files, used to spread data across multiple disks/filegroups for very large databases.
Log file	`.ldf`	The transaction log — a sequential record of every change, used for crash recovery, transaction rollback, and point-in-time restore (Part 37).

This is a genuinely SQL Server-specific storage architecture. PostgreSQL organizes storage as a directory of files per table/index inside a "cluster" data directory; MySQL (InnoDB) uses tablespace files (`.ibd`) per table by default. SQL Server's explicit MDF/NDF/LDF file model, with the transaction log as a distinctly separate physical file, directly shapes both backup strategy (Part 37) and performance tuning (`tempdb`'s own files, Part 40).

3.4 Filegroups

```
ALTER DATABASE Sales ADD FILEGROUP Archive;

ALTER DATABASE Sales ADD FILE (
    NAME = Sales_Archive,
    FILENAME = 'C:\SQLData\Sales_Archive.ndf'
) TO FILEGROUP Archive;
```

A **filegroup** is a named collection of one or more data files that tables/indexes can be explicitly placed onto — the **PRIMARY** filegroup (holding the `.mdf` and always present) is the default; additional, secondary filegroups let large tables, indexes, or partitions (Part 44) be physically placed on specific disks for performance or archival separation. Neither PostgreSQL's tablespaces nor MySQL's tablespace files map exactly onto SQL Server's filegroup concept, though tablespaces serve a broadly similar physical-placement role in PostgreSQL.

3.5 Recovery Models

Model	Behavior
Simple	The transaction log is automatically truncated after each checkpoint; no transaction log backups possible. Simplest to manage, but only point-in-time recovery to the last full/differential backup.
Full	The log retains every transaction until explicitly backed up, enabling point-in-time recovery (Part 37) and transaction log backups — the standard choice for production databases.
Bulk-logged	A middle ground, minimally logging certain bulk operations (`BULK INSERT`, index rebuilds) for performance, while still supporting log backups.

Recovery models are a distinctly SQL Server administrative concept without a direct one-to-one equivalent in PostgreSQL (which uses WAL — Write-Ahead Logging — with continuous archiving configured separately) or MySQL (binary logging, configured independently of a single "recovery model" setting). Choosing the recovery model deliberately (Full for anything requiring point-in-time recovery) is a foundational production decision covered further in Part 37.

3.6 Professional Database Organization Strategy

- One database per logical application/bounded context, rather than one giant shared database for unrelated systems.
- Schemas used deliberately to group related objects (`Sales`, `HR`, `Audit`) rather than dumping everything into the default `dbo` schema.
- `tempdb`-heavy workloads (Part 40) considered when sizing and placing files/filegroups.
- Recovery model chosen explicitly per database based on actual recovery requirements (Part 37), never left as an accidental default.

---

4. SQL Server Data Types

4.1 Numeric Types

Type	Storage	Range/Notes
`tinyint`	1 byte	0 to 255 (unsigned only — no negative values, unlike most other integer types)
`smallint`	2 bytes	-32,768 to 32,767
`int`	4 bytes	~-2.1 billion to 2.1 billion
`bigint`	8 bytes	Very large range; use for surrogate keys expected to exceed `int`'s range
`decimal(p,s)` / `numeric(p,s)`	Varies by precision	Exact, fixed-point — the standard choice for money and precise calculations; `decimal` and `numeric` are fully synonymous in SQL Server
`money`	8 bytes	Fixed 4 decimal places; **generally avoided professionally** in favor of `decimal(19,4)` for clearer precision control and portability
`smallmoney`	4 bytes	Smaller-range version of `money`, same caveats
`float`	4 or 8 bytes	Approximate, floating-point — not appropriate for exact financial values
`real`	4 bytes	Approximate, single-precision floating-point

[SQL Server vs PostgreSQL/MySQL]: `tinyint` being unsigned-only (0-255) is a genuine SQL Server quirk — PostgreSQL has no `tinyint` at all (its smallest integer is `smallint`), and

MySQL's `tinyint` supports an explicit signed range (-128 to 127) by default. Always double-check `tinyint`'s range assumption when porting schemas.

4.2 Character Types

Type	Storage	Notes
`char(n)`	Fixed, n bytes	Fixed-length, space-padded; rarely the right default choice.
`varchar(n)`	Variable, up to n bytes	Variable-length, non-Unicode (ASCII/extended ASCII per collation).
`nchar(n)`	Fixed, 2n bytes	Fixed-length Unicode.
`nvarchar(n)`	Variable, up to 2n bytes	Variable-length Unicode — the standard professional default for any text that might contain non-English characters.
`text`	Deprecated	Legacy large-text type; **avoid** in new development — use `varchar(max)`/`nvarchar(max)` instead (Section 4.2.1).

4.2.1 Why text Is Avoided

`text` (and its counterparts `ntext`, `image`) are deprecated legacy types predating `varchar(max)`/`nvarchar(max)`/`varbinary(max)`, which were introduced specifically to replace them with full support for standard string functions, indexing flexibility, and modern T-SQL features that `text` lacks or handles awkwardly. **Always use `varchar(max)`/`nvarchar(max)` for large text in new SQL Server development.**

[SQL Server vs PostgreSQL/MySQL]: PostgreSQL's `text` type is the modern, fully-supported, unlimited-length standard — completely unrelated to SQL Server's deprecated `text`; don't let the shared name cause confusion. MySQL's TEXT (uppercase-conventional) is also actively used and not deprecated. This is purely a SQL Server-specific naming trap.

4.3 Date/Time Types

Type	Precision	Storage	Notes
`date`	Day	3 bytes	Date only, no time component.
`time`	Up to 100ns	3-5 bytes	Time only, no date.
`datetime`	~3.33ms	8 bytes	Legacy type with an odd rounding precision and a limited range (1753-9999); prefer

Type	Precision	Storage	Notes
			`datetime2` in new development.
`datetime2`	Up to 100ns	6-8 bytes	Modern, higher-precision, wider-range replacement for `datetime` — the standard professional default.
`smalldatetime`	1 minute	4 bytes	Compact, low-precision legacy type.
`datetimeoffset`	Up to 100ns	8-10 bytes	`datetime2` plus a UTC offset — the correct choice for any application spanning multiple time zones.

[SQL Server vs PostgreSQL/MySQL]: PostgreSQL's `timestamp`/`timestamptz` map roughly onto SQL Server's `datetime2`/`datetimeoffset` respectively; MySQL's `DATETIME`/`TIMESTAMP` distinction is different again (MySQL's `TIMESTAMP` is UTC-converting and range-limited to 2038, unlike SQL Server's `datetimeoffset`). Always re-verify these mappings explicitly when porting schemas rather than assuming a name match implies identical behavior.

4.4 Other Types

Type	Purpose
`bit`	Boolean-equivalent: 0, 1, or NULL. SQL Server has no native `boolean` type — `bit` is the standard substitute.
`uniqueidentifier`	A 128-bit GUID, SQL Server's native globally-unique identifier type — see `NEWID()`/`NEWSEQUENTIALID()` (Section 5.4).
`binary(n)` / `varbinary(n)`	Fixed/variable-length raw binary data; `varbinary(max)` for large binary objects (files, images).
`xml`	Native XML storage and querying (Part 32).
JSON	SQL Server has **no dedicated JSON data type** — JSON is stored as `nvarchar(max)` and manipulated with JSON *functions* (Part 31), a notable difference from PostgreSQL's native `json`/`jsonb` types.
Spatial types (`geometry`, `geography`)	Native spatial/GIS data types, broadly analogous to PostgreSQL's PostGIS extension,

Type	Purpose
	but built into the engine directly.

[SQL Server vs PostgreSQL]: The absence of a true JSON data type is one of the most consequential differences from PostgreSQL for application developers used to `jsonb`'s native indexing and operators — SQL Server's JSON support (Part 31) is function-based over plain `nvarchar(max)`, which changes both the syntax and the indexing strategy (typically via computed columns, Section 31.6) needed for performant JSON querying.

4.5 SQL Server Data-Type Selection Guide

Business Need	Recommended Type
Primary/surrogate key	`int IDENTITY` or `bigint IDENTITY` (Section 5.3); `uniqueidentifier` when values must be generated outside the database or merged across systems
Money/financial amounts	`decimal(19,4)`, not `money` or `float`
Names, emails, short text	`nvarchar(n)` sized to a sensible realistic maximum
Large free text	`nvarchar(max)`
Yes/No flags	`bit`
Timestamps (single time zone)	`datetime2`
Timestamps (multi-time-zone)	`datetimeoffset`
Date only (birthdate, order date)	`date`
JSON payloads	`nvarchar(max)` with `ISJSON()` constraint (Part 31)

5. Table Design

5.1 CREATE TABLE / ALTER TABLE / DROP TABLE / TRUNCATE TABLE

```
CREATE TABLE Sales.Customers (
    CustomerID INT IDENTITY(1,1) PRIMARY KEY,
    FullName NVARCHAR(100) NOT NULL,
    Email NVARCHAR(255) UNIQUE,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSDATETIME()
);

ALTER TABLE Sales.Customers ADD Phone NVARCHAR(20) NULL;
ALTER TABLE Sales.Customers DROP COLUMN Phone;
TRUNCATE TABLE Sales.Customers;   -- fast, minimally-logged delete of
ALL rows; resets IDENTITY
DROP TABLE Sales.Customers;
```

`TRUNCATE TABLE` behaves the same conceptually as in PostgreSQL/MySQL — dramatically faster than `DELETE` for clearing an entire table since it deallocates data pages rather than logging row-by-row deletes, but it cannot be filtered with `WHERE` and resets `IDENTITY` (Section 5.3) back to its seed value.

5.2 Keys — Primary, Foreign, Composite, Candidate, Surrogate, Natural

Key Type	Meaning
Primary key	The column(s) uniquely identifying each row; enforced with a unique index automatically.
Foreign key	A column referencing another table's primary key, enforcing referential integrity.
Composite key	A primary/unique key made of more than one column.
Candidate key	Any column (or set of columns) that *could* validly serve as the primary key.
Surrogate key	An artificial, meaningless identifier (typically `IDENTITY` or `uniqueidentifier`) with no business meaning — the standard professional choice for primary keys.
Natural key	A real-world, business-meaningful column (an email, a national ID) used as a key — appropriate as a `UNIQUE` constraint, but generally avoided as the *primary* key since business values can change.

```
CREATE TABLE Sales.OrderItems (
    OrderID INT NOT NULL,
    LineNumber INT NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
```

```
    CONSTRAINT PK_OrderItems PRIMARY KEY (OrderID, LineNumber),   -- composite key
    CONSTRAINT FK_OrderItems_Orders FOREIGN KEY (OrderID) REFERENCES Sales.Orders(OrderID)
);
```

5.3 IDENTITY

```
CREATE TABLE Sales.Orders (
    OrderID INT IDENTITY(1,1) PRIMARY KEY,   -- seed 1, increment 1
    OrderDate DATE NOT NULL
);

INSERT INTO Sales.Orders (OrderDate) VALUES ('2026-01-05');
SELECT SCOPE_IDENTITY();   -- returns the just-inserted IDENTITY value
```

`IDENTITY(seed, increment)` is SQL Server's **[T-SQL]** auto-incrementing column feature — conceptually equivalent to PostgreSQL's `SERIAL`/`GENERATED ALWAYS AS IDENTITY` and MySQL's `AUTO_INCREMENT`, but with its own retrieval function: `SCOPE_IDENTITY()` returns the last identity value generated in the *current session and scope*, deliberately safer than the older `@@IDENTITY` (which can return a value from an unrelated trigger-fired insert on a different table).

5.4 SQL Server SEQUENCE

```
CREATE SEQUENCE Sales.OrderNumberSeq
    START WITH 1000
    INCREMENT BY 1;

INSERT INTO Sales.Orders (OrderID, OrderDate)
VALUES (NEXT VALUE FOR Sales.OrderNumberSeq, '2026-01-05');
```

A `SEQUENCE` is a standalone, schema-scoped number generator, independent of any single table — usable across multiple tables, reservable in advance, and controllable (`CACHE`, `CYCLE`, `MINVALUE`/`MAXVALUE`) with more flexibility than `IDENTITY`. This maps closely onto PostgreSQL's `SEQUENCE` object (which `SERIAL` is itself built on internally); MySQL has no native sequence object at all, historically relying purely on `AUTO_INCREMENT`.

5.5 IDENTITY vs SEQUENCE — When to Use Each

Scenario	Prefer
Simple auto-incrementing single-table primary key	`IDENTITY`
A shared number series needed across multiple tables	`SEQUENCE`
Values needed *before* the `INSERT` executes (e.g. to use in related records first)	`SEQUENCE` (`NEXT VALUE FOR` can be called independently)
Need to control caching/cycling behavior explicitly	`SEQUENCE`

5.6 DEFAULT, NOT NULL, UNIQUE, CHECK

```
CREATE TABLE HR.Employees (
    EmployeeID INT IDENTITY PRIMARY KEY,
    Salary DECIMAL(10,2) NOT NULL CHECK (Salary > 0),
    Status NVARCHAR(20) NOT NULL DEFAULT 'Active',
    Email NVARCHAR(255) NOT NULL UNIQUE
);
```

These constraint types behave essentially identically to PostgreSQL/MySQL and are not re-taught from first principles here; the syntax shown is standard T-SQL, directly portable in concept from either database you already know.

5.7 A Professional Table-Design Example

```
CREATE TABLE Sales.Orders (
    OrderID       INT IDENTITY(1,1) NOT NULL,
    CustomerID    INT NOT NULL,
    OrderDate     DATE NOT NULL DEFAULT CAST(SYSDATETIME() AS DATE),
    Status        NVARCHAR(20) NOT NULL DEFAULT 'Pending',
    TotalAmount   DECIMAL(12,2) NOT NULL DEFAULT 0 CHECK (TotalAmount
>= 0),
    CreatedAt     DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT PK_Orders PRIMARY KEY (OrderID),
    CONSTRAINT FK_Orders_Customers FOREIGN KEY (CustomerID)
        REFERENCES Sales.Customers(CustomerID),
    CONSTRAINT CK_Orders_Status CHECK (Status IN
('Pending','Shipped','Cancelled','Delivered'))
);
```

Every business rule expressible as a constraint (valid status values, non-negative totals) is enforced directly at the schema level — a professional habit that prevents invalid data regardless of which application, script, or careless ad-hoc query ultimately writes to the table.

---

6. CRUD Operations

6.1 INSERT — Single Row, Multiple Rows, From Another Table

```
-- Single row
INSERT INTO Sales.Customers (FullName, Email) VALUES ('Ada Lovelace',
'ada@example.com');

-- Multiple rows in one statement [T-SQL row-constructor syntax, also
standard SQL:2008+]
INSERT INTO Sales.Customers (FullName, Email) VALUES
    ('Grace Hopper', 'grace@example.com'),
    ('Alan Turing', 'alan@example.com');

-- Insert from another table (or query result)
INSERT INTO Sales.ArchivedCustomers (FullName, Email)
SELECT FullName, Email FROM Sales.Customers WHERE Status = 'Inactive';
```

6.2 UPDATE — Multiple Records, Conditional

```
UPDATE Sales.Customers
SET Status = 'Inactive'
WHERE LastOrderDate < DATEADD(YEAR, -1, SYSDATETIME());

-- Update joining another table [T-SQL UPDATE ... FROM syntax]
UPDATE c
SET c.TotalSpent = o.OrderTotal
FROM Sales.Customers AS c
JOIN (SELECT CustomerID, SUM(TotalAmount) AS OrderTotal FROM
Sales.Orders GROUP BY CustomerID) AS o
    ON c.CustomerID = o.CustomerID;
```

`UPDATE ... FROM` is **[T-SQL-specific]** syntax for updating one table based on a join to another; PostgreSQL uses `UPDATE ... SET ... FROM ...` similarly, while MySQL uses `UPDATE t1 JOIN t2 ON ... SET ...` — same capability, genuinely different syntax across all three.

6.3 DELETE — Safe Deletion, Related Records

```
-- Always confirm scope with a SELECT first, in production practice
SELECT * FROM Sales.Orders WHERE Status = 'Cancelled' AND OrderDate <
'2024-01-01';

DELETE FROM Sales.Orders WHERE Status = 'Cancelled' AND OrderDate <
'2024-01-01';

-- Deleting related child records first (or rely on ON DELETE CASCADE
if defined)
DELETE FROM Sales.OrderItems WHERE OrderID IN (SELECT OrderID FROM
Sales.Orders WHERE Status = 'Cancelled');
DELETE FROM Sales.Orders WHERE Status = 'Cancelled';
```

Common Mistake: Running an `UPDATE`/`DELETE` without a `WHERE` clause, updating/deleting every row in the table. Professional practice: always run the equivalent `SELECT` with the same `WHERE` clause first to confirm the affected row count before executing the write, and wrap ad-hoc production changes in an explicit transaction (Part 23) that can be rolled back if the row count looks wrong.

6.4 OUTPUT — Capturing Inserted, Updated, or Deleted Rows

```
-- [SQL Server / T-SQL feature — no direct PostgreSQL/MySQL equivalent
syntax, though
--  PostgreSQL's RETURNING clause serves a similar purpose]
INSERT INTO Sales.Orders (CustomerID, OrderDate)
OUTPUT inserted.OrderID, inserted.OrderDate
VALUES (101, '2026-01-05');

UPDATE Sales.Customers
SET Status = 'Inactive'
OUTPUT deleted.Status AS OldStatus, inserted.Status AS NewStatus,
inserted.CustomerID
```

```
WHERE LastOrderDate < DATEADD(YEAR, -1, SYSDATETIME());

DELETE FROM Sales.Orders
OUTPUT deleted.OrderID, deleted.CustomerID
WHERE Status = 'Cancelled';
```

`OUTPUT` exposes two special pseudo-tables during a DML statement: `inserted` (the new row values, for `INSERT`/`UPDATE`) and `deleted` (the old row values, for `UPDATE`/`DELETE`) — letting a single statement both modify data *and* return exactly what changed, without a separate follow-up `SELECT`. This is directly comparable in purpose to PostgreSQL's `RETURNING` clause, though the `inserted`/`deleted` pseudo-table syntax, and the ability to reference *both* old and new values simultaneously in an `UPDATE`, is distinctly T-SQL. MySQL has no equivalent feature at all.

Business use case: capturing exactly which rows were archived/cancelled/updated for audit logging, without a race-condition-prone separate `SELECT` before or after the write.

---

7. SELECT and T-SQL Querying

7.1 Core Clauses

```
SELECT DISTINCT TOP 10 CustomerID, TotalAmount
FROM Sales.Orders
WHERE Status = 'Shipped'
ORDER BY TotalAmount DESC;
```

`SELECT`, `DISTINCT`, `WHERE`, `ORDER BY`, `GROUP BY`, `HAVING` behave the same as in PostgreSQL/MySQL and are not re-taught here.

7.2 TOP — Deep Dive and Comparison

```
-- [T-SQL] TOP with a fixed count
SELECT TOP 5 CustomerID, TotalAmount FROM Sales.Orders ORDER BY
TotalAmount DESC;

-- TOP with PERCENT
SELECT TOP 10 PERCENT CustomerID, TotalAmount FROM Sales.Orders ORDER
BY TotalAmount DESC;

-- TOP WITH TIES - includes additional rows tied with the last
included row
SELECT TOP 5 WITH TIES CustomerID, TotalAmount
FROM Sales.Orders
ORDER BY TotalAmount DESC;
```

Feature	SQL Server	PostgreSQL	MySQL
Syntax	`SELECT TOP (n) ...` (before column list)	`SELECT ... LIMIT n` (after everything)	`SELECT ... LIMIT n` (after everything)

Feature	SQL Server	PostgreSQL	MySQL
Percentage-based limiting	`TOP n PERCENT`	Not built in (compute manually)	Not built in
Include ties at the boundary	`TOP n WITH TIES`	Not built in directly (`FETCH ... WITH TIES` in modern PostgreSQL)	Not built in
Offset/pagination	`OFFSET ... FETCH` (Section 7.3)	`LIMIT n OFFSET m`	`LIMIT m, n`

`TOP` must always be paired with `ORDER BY` to be meaningful — without it, "top" rows are returned in whatever arbitrary order the engine happens to produce them, which is a common, subtle correctness bug when porting `LIMIT`-based PostgreSQL/MySQL queries that happened to omit `ORDER BY`.

7.3 OFFSET / FETCH — Standard SQL Pagination

```
-- [Standard SQL:2008, available in SQL Server, PostgreSQL, and modern MySQL 8+]
SELECT CustomerID, TotalAmount
FROM Sales.Orders
ORDER BY OrderDate DESC
OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;
```

Unlike `TOP`, `OFFSET...FETCH` is standard SQL and directly portable — the recommended choice for pagination logic in code intended to work across databases, or simply as the more modern, standards-based idiom even within a SQL-Server-only codebase.

7.4 Logical Query Processing Order

```
1. FROM        (including JOINs)
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT      (including DISTINCT's column list is evaluated here)
6. DISTINCT
7. ORDER BY
8. TOP / OFFSET-FETCH
```

This logical order — identical in concept across PostgreSQL, MySQL, and SQL Server, since it derives from the SQL standard, not any vendor — explains many common beginner confusions: a column alias defined in `SELECT` cannot be referenced in `WHERE` (since `WHERE` is logically evaluated *before* `SELECT`), but *can* be referenced in `ORDER BY` (evaluated *after* `SELECT`). **Physical execution** (Part 28-29) does not necessarily follow this order literally — SQL Server's optimizer is free to reorder operations for performance as long as the *logical result* is identical — but reasoning about correctness should always follow the logical order above.

8. Operators and Expressions

8.1 Arithmetic, Comparison, and Logical Operators

```
SELECT OrderID, TotalAmount, TotalAmount * 1.08 AS WithTax
FROM Sales.Orders
WHERE TotalAmount > 100 AND Status IN ('Shipped', 'Delivered') AND NOT
(CustomerID = 5);
```

`+ - * / %`, `= <> != < > <= >=`, and `AND OR NOT` all behave as expected from PostgreSQL/MySQL. Note: `<>` and `!=` are both valid "not equal" operators in T-SQL; `<>` is the ANSI-standard form and generally preferred for portability.

8.2 IN, BETWEEN, LIKE, IS NULL

```
SELECT * FROM Sales.Orders WHERE Status IN ('Pending', 'Shipped');
SELECT * FROM Sales.Orders WHERE OrderDate BETWEEN '2026-01-01' AND
'2026-01-31';
SELECT * FROM Sales.Customers WHERE Email LIKE '%@gmail.com';
SELECT * FROM Sales.Customers WHERE Phone IS NULL;
```

8.3 Wildcard Patterns

Pattern	Meaning
`%`	Any sequence of zero or more characters.
`_`	Any single character.
`[abc]`	Any single character within the listed set — **[T-SQL-specific]**; PostgreSQL/MySQL `LIKE` has no bracket-set syntax at all (PostgreSQL uses `SIMILAR TO` or regex `~` for this).
`[^abc]`	Any single character **not** in the listed set — also **[T-SQL-specific]**.
`[a-f]`	Any single character in the given range.

```
SELECT * FROM Sales.Products WHERE ProductCode LIKE 'A[0-9][0-9]';
-- 'A' + exactly two digits
SELECT * FROM Sales.Products WHERE ProductCode LIKE '[^A-C]%';
-- does not start with A, B, or C
```

T-SQL's bracket-set wildcards inside `LIKE` are a genuinely distinctive feature — a middle ground between plain `%`/`_` wildcards and full regular expressions, useful for simple structured-pattern matching without needing `PATINDEX` (Part 9) or a full regex engine (which T-SQL's native `LIKE` does not support at all, unlike PostgreSQL's `~`).

---

9. T-SQL Functions

9.1 String Functions

```
SELECT
    LEN(FullName)                          AS CharLength,        --
excludes trailing spaces; DATALENGTH gives raw bytes
```

```
    DATALENGTH(FullName)                AS ByteLength,
    LEFT(FullName, 3)                   AS FirstThree,
    RIGHT(FullName, 3)                  AS LastThree,
    SUBSTRING(FullName, 1, 4)           AS Sub,
    CHARINDEX('@', Email)               AS AtPosition,
    PATINDEX('%[0-9]%', FullName)       AS FirstDigitPosition,  --
pattern-based search, supports wildcards
    REPLACE(Email, 'old.com', 'new.com') AS UpdatedEmail,
    CONCAT(FullName, ' <', Email, '>')  AS Display,
    CONCAT_WS(', ', City, State, Zip)   AS Address,             --
concat with separator, skips NULLs
    TRIM(FullName)                      AS Trimmed,
    UPPER(FullName)                     AS Upper,
    LOWER(Email)                        AS Lower
FROM Sales.Customers;
```

Function	Notes
`LEN`	Character count, trailing spaces ignored — differs subtly from PostgreSQL's `LENGTH()`, which does not trim.
`CHARINDEX(substr, str)`	Like PostgreSQL's `POSITION(substr IN str)` / MySQL's `LOCATE`; finds a literal substring's position.
`PATINDEX('%pattern%', str)`	**[T-SQL-specific]** — finds a `LIKE`-style *pattern* (with wildcards) position, not just a literal substring.
`TRANSLATE(str, from, to)`	**[T-SQL, SQL Server 2017+]** — character-by-character substitution, like a compact multi-character `REPLACE` in one call.
`STRING_AGG(expr, separator)`	**[T-SQL, SQL Server 2017+]** — aggregates values across rows into one delimited string, directly equivalent to PostgreSQL's `STRING_AGG` (same name!) and MySQL's `GROUP_CONCAT`.
`STRING_SPLIT(str, separator)`	**[T-SQL, SQL Server 2016+]** — splits a delimited string into rows (a table-valued function); comparable to PostgreSQL's `unnest(string_to_array(...))` or MySQL's lack of a direct built-in equivalent.

```
SELECT CustomerID, STRING_AGG(ProductName, ', ') AS ProductsOrdered
FROM Sales.OrderItems oi JOIN Sales.Products p ON oi.ProductID =
p.ProductID
GROUP BY CustomerID;

SELECT value AS Tag FROM STRING_SPLIT('red,green,blue', ',');
```

9.2 Numeric Functions

```
SELECT ABS(-5), CEILING(4.1), FLOOR(4.9), ROUND(3.14159, 2), POWER(2,
10), SQRT(144), RAND();
```

Standard across all three databases in concept; `CEILING` (SQL Server's spelling) vs `CEIL` (PostgreSQL/MySQL both accept `CEIL`, and PostgreSQL also accepts `CEILING`) is a minor naming difference worth remembering when porting.

9.3 Date Functions

```
SELECT
    GETDATE()                                   AS Now_Legacy,        --
datetime, legacy but still common
    SYSDATETIME()                               AS Now_Modern,        --
datetime2, higher precision, preferred
    CURRENT_TIMESTAMP                           AS Now_Standard,      --
ANSI standard, equivalent to GETDATE()
    DATEADD(DAY, 30, OrderDate)                 AS ThirtyDaysLater,
    DATEDIFF(DAY, OrderDate, SYSDATETIME()) AS DaysSinceOrder,
    DATEDIFF_BIG(SECOND, OrderDate, SYSDATETIME()) AS
SecondsSinceOrder,  -- avoids DATEDIFF's int overflow
    DATEPART(MONTH, OrderDate)                  AS OrderMonth,
    DATENAME(WEEKDAY, OrderDate)                AS OrderDayName,
    EOMONTH(OrderDate)                          AS EndOfOrderMonth,
    DATEFROMPARTS(2026, 1, 5)                   AS ConstructedDate,
    DATETIMEFROMPARTS(2026, 1, 5, 14, 30, 0, 0) AS ConstructedDateTime
FROM Sales.Orders;
```

Function	Notes
`DATEADD(unit, n, date)`	Comparable to PostgreSQL's `date + interval '30 days'` or MySQL's `DATE_ADD`; T-SQL's unit-keyword-first syntax is its own.
`DATEDIFF(unit, start, end)`	Returns an *integer count* of unit boundaries crossed — a common source of off-by-one surprises (e.g. `DATEDIFF(YEAR, '2025-12-31', '2026-01-01')` returns 1, despite only one day apart, since it counts calendar-year boundaries crossed, not elapsed time).
`DATEDIFF_BIG`	Same as `DATEDIFF` but returns `bigint`, avoiding overflow when counting small units (seconds, milliseconds) across a large date range.
`EOMONTH`	**[T-SQL, SQL Server 2012+]** — end-of-month date, directly useful for monthly reporting boundaries without manual date arithmetic.
`DATEFROMPARTS/DATETIMEFROMPARTS`	**[T-SQL]** — construct a date/datetime from

Function	Notes
	separate integer components, safer and clearer than string concatenation + `CAST`.

9.4 Conversion Functions — CAST, CONVERT, TRY_CAST, TRY_CONVERT

```
SELECT CAST('2026-01-05' AS DATE);
SELECT CONVERT(DATE, '2026-01-05');
SELECT CONVERT(VARCHAR(10), SYSDATETIME(), 120);   -- style code 120 =
ISO 8601 'yyyy-mm-dd hh:mi:ss'

SELECT TRY_CAST('not a date' AS DATE);        -- returns NULL instead
of raising an error
SELECT TRY_CONVERT(DECIMAL(10,2), 'abc');      -- returns NULL instead
of raising an error
```

`CAST` is standard SQL; `CONVERT` is **[T-SQL-specific]**, functionally similar but additionally accepting a numeric *style* code (like `120` above) controlling exact output formatting — particularly useful for date/time-to-string conversions, an area PostgreSQL handles instead with `TO_CHAR()`'s format-string approach.

Why TRY_CAST/TRY_CONVERT matter for data cleaning: a plain `CAST`/`CONVERT` on invalid input **raises an error and aborts the batch**; `TRY_CAST`/`TRY_CONVERT` instead return `NULL` on failure, letting a data-cleaning query (Part 33) process an entire messy dataset in one pass — flagging or filtering unconvertible rows via `WHERE TRY_CAST(...) IS NULL` — rather than crashing on the first bad row. PostgreSQL has no exact equivalent (community extensions aside); MySQL's implicit, often-silent type coercion behaves quite differently again, making `TRY_CAST`/`TRY_CONVERT` a distinctly valuable, explicit T-SQL tool.

9.5 NULL Handling — ISNULL, COALESCE, NULLIF

```
SELECT ISNULL(Phone, 'N/A') FROM Sales.Customers;         -- [T-SQL-
specific]
SELECT COALESCE(Phone, MobilePhone, 'N/A') FROM Sales.Customers;   --
[Standard SQL]
SELECT NULLIF(Discount, 0) FROM Sales.Orders;              -- returns
NULL if Discount = 0, else Discount
```

Aspect	`ISNULL`	`COALESCE`
Standard?	**[T-SQL-specific]**	**[Standard SQL]**, portable to PostgreSQL/MySQL
Number of arguments	Exactly 2	2 or more
Return type	Determined by the *first* argument's type	Determined by the *highest-precedence* type among all arguments
Performance	Occasionally marginally faster (simpler internally)	Negligible difference in practice

Professional recommendation: prefer `COALESCE` for portability and its more predictable type-resolution rules, reserving `ISNULL` for cases specifically needing its particular type-preservation behavior — a subtlety that occasionally matters when the fallback value's implied type differs from the primary expression's.

---

10. Conditional Logic

10.1 Simple and Searched CASE

```
-- Simple CASE (compares one expression against several values)
SELECT OrderID,
    CASE Status
        WHEN 'Pending' THEN 1
        WHEN 'Shipped' THEN 2
        WHEN 'Delivered' THEN 3
        ELSE 0
    END AS StatusRank
FROM Sales.Orders;

-- Searched CASE (arbitrary boolean conditions per branch)
SELECT CustomerID, TotalSpent,
    CASE
        WHEN TotalSpent >= 10000 THEN 'Platinum'
        WHEN TotalSpent >= 5000  THEN 'Gold'
        WHEN TotalSpent >= 1000  THEN 'Silver'
        ELSE 'Bronze'
    END AS CustomerTier
FROM Sales.Customers;
```

10.2 Nested CASE, CASE in Aggregation, CASE in ORDER BY

```
-- CASE inside an aggregate - a common, powerful "conditional
counting" pattern
SELECT
    COUNT(CASE WHEN Status = 'Delivered' THEN 1 END) AS
DeliveredCount,
    SUM(CASE WHEN Status = 'Cancelled' THEN TotalAmount ELSE 0 END) AS
CancelledRevenue
FROM Sales.Orders;

-- CASE inside ORDER BY - dynamic sort priority
SELECT * FROM Sales.Orders
ORDER BY CASE WHEN Status = 'Pending' THEN 0 ELSE 1 END, OrderDate
DESC;
```

`CASE` behaves identically across PostgreSQL, MySQL, and SQL Server — genuinely standard SQL, not a T-SQL extension — so this pattern of "conditional aggregation" (`COUNT`/`SUM` combined with `CASE`) transfers directly from prior PostgreSQL/MySQL experience.

10.3 Business Examples

```sql
-- Risk classification (banking)
SELECT AccountID, CreditScore,
    CASE
        WHEN CreditScore >= 750 THEN 'Low Risk'
        WHEN CreditScore >= 650 THEN 'Medium Risk'
        ELSE 'High Risk'
    END AS RiskCategory
FROM Banking.Accounts;

-- Employee performance categorization
SELECT EmployeeID, SalesTotal,
    CASE
        WHEN SalesTotal >= (SELECT AVG(SalesTotal) * 1.2 FROM
HR.EmployeeSales) THEN 'Exceeds Expectations'
        WHEN SalesTotal >= (SELECT AVG(SalesTotal) * 0.8 FROM
HR.EmployeeSales) THEN 'Meets Expectations'
        ELSE 'Needs Improvement'
    END AS PerformanceRating
FROM HR.EmployeeSales;
```

11. Joins

11.1 Join Types Overview

Join mechanics (`INNER`, `LEFT`, `RIGHT`, `FULL OUTER`, `CROSS`, self-joins) are conceptually identical across PostgreSQL, MySQL, and SQL Server — the relational algebra doesn't change — so this part focuses on T-SQL syntax specifics and behavior worth confirming explicitly, rather than re-deriving joins from scratch.

```
-- INNER JOIN: only matching rows from both sides
SELECT c.FullName, o.OrderID, o.TotalAmount
FROM Sales.Customers c
INNER JOIN Sales.Orders o ON c.CustomerID = o.CustomerID;

-- LEFT JOIN: all customers, even those with zero orders (unmatched
columns are NULL)
SELECT c.FullName, o.OrderID
FROM Sales.Customers c
LEFT JOIN Sales.Orders o ON c.CustomerID = o.CustomerID;

-- RIGHT JOIN: the mirror image of LEFT JOIN (less commonly used
stylistically; a LEFT JOIN
-- with tables reversed is usually preferred for readability)
SELECT c.FullName, o.OrderID
FROM Sales.Orders o
RIGHT JOIN Sales.Customers c ON c.CustomerID = o.CustomerID;

-- FULL OUTER JOIN: every row from both sides, matched where possible
SELECT c.FullName, o.OrderID
FROM Sales.Customers c
FULL OUTER JOIN Sales.Orders o ON c.CustomerID = o.CustomerID;

-- CROSS JOIN: the full Cartesian product (every row paired with every
row)
SELECT p.ProductName, s.StoreName FROM Sales.Products p CROSS JOIN
Sales.Stores s;

-- SELF JOIN: a table joined to itself, e.g. finding an employee's
manager
SELECT e.FullName AS Employee, m.FullName AS Manager
FROM HR.Employees e
LEFT JOIN HR.Employees m ON e.ManagerID = m.EmployeeID;
```

MySQL note: MySQL did not support `FULL OUTER JOIN` directly until relatively recent behavior workarounds (traditionally simulated via `UNION` of `LEFT JOIN` and `RIGHT JOIN`); SQL Server has always supported it natively, same as PostgreSQL.

11.2 Relationship Cardinality and Duplicate Rows

A **one-to-many** join (one customer, many orders) naturally produces one output row per matching order — entirely expected. A **many-to-many** join (through an intermediate table, like `OrderItems` linking `Orders` and `Products`) or an unintentionally imprecise join condition can silently multiply rows in ways that inflate `SUM()`/`COUNT()` aggregates — one of the most common real-world query correctness bugs, entirely independent of which SQL dialect is used. Always sanity-check aggregate results against an independent row count when joins are involved.

11.3 NULL Behavior in Joins

`LEFT/RIGHT/FULL OUTER JOIN` populate unmatched columns with `NULL` — and critically, filtering on those `NULL`-able columns in `WHERE` (rather than in the `ON` clause) can silently convert an outer join back into an inner join, since `WHERE SomeOuterColumn = 'X'` is never true when `SomeOuterColumn IS NULL`. Move such filters into the `ON` clause when the intent is genuinely to preserve unmatched rows.

```
-- WRONG - silently becomes an inner join because the WHERE eliminates
unmatched (NULL) rows
SELECT c.FullName, o.OrderID
FROM Sales.Customers c LEFT JOIN Sales.Orders o ON c.CustomerID =
o.CustomerID
WHERE o.Status = 'Shipped';

-- RIGHT - preserves customers with no matching order at all
SELECT c.FullName, o.OrderID
FROM Sales.Customers c
LEFT JOIN Sales.Orders o ON c.CustomerID = o.CustomerID AND o.Status =
'Shipped';
```

---

12. Subqueries

12.1 Scalar, Single-Row, and Multi-Row Subqueries

```
-- Scalar subquery - must return exactly one value
SELECT CustomerID, TotalAmount,
    (SELECT AVG(TotalAmount) FROM Sales.Orders) AS OverallAverage
FROM Sales.Orders;

-- Multi-row subquery with IN
SELECT * FROM Sales.Customers
WHERE CustomerID IN (SELECT CustomerID FROM Sales.Orders WHERE
TotalAmount > 1000);
```

12.2 Correlated Subqueries

```
-- References the outer query's row (c.CustomerID) on each evaluation
SELECT c.FullName,
    (SELECT COUNT(*) FROM Sales.Orders o WHERE o.CustomerID =
```

```
c.CustomerID) AS OrderCount
FROM Sales.Customers c;
```

A correlated subquery is logically re-evaluated once per outer row — the same concept, same performance considerations, and same syntax across PostgreSQL/MySQL/SQL Server.

12.3 EXISTS / NOT EXISTS

```
SELECT c.FullName
FROM Sales.Customers c
WHERE EXISTS (SELECT 1 FROM Sales.Orders o WHERE o.CustomerID =
c.CustomerID);

SELECT c.FullName
FROM Sales.Customers c
WHERE NOT EXISTS (SELECT 1 FROM Sales.Orders o WHERE o.CustomerID =
c.CustomerID);
```

12.4 Why NOT IN Produces Unexpected Results With NULLs

```
-- If Sales.Orders.CustomerID contains even ONE NULL value, this
returns ZERO rows,
-- even for customers genuinely absent from every non-NULL CustomerID!
SELECT * FROM Sales.Customers
WHERE CustomerID NOT IN (SELECT CustomerID FROM Sales.Orders);

-- SAFE alternative - NOT EXISTS is unaffected by NULLs in the
subquery
SELECT * FROM Sales.Customers c
WHERE NOT EXISTS (SELECT 1 FROM Sales.Orders o WHERE o.CustomerID =
c.CustomerID);
```

This `NOT IN` + `NULL` trap is universal SQL behavior (three-valued logic: `x <> NULL` evaluates to `UNKNOWN`, not `TRUE`, causing the entire `NOT IN` list comparison to become `UNKNOWN` when any list member is `NULL`) — identical across PostgreSQL, MySQL, and SQL Server, not a SQL Server quirk, but worth re-emphasizing since it's a genuinely common, silent production bug. **Professional rule: prefer `NOT EXISTS` over `NOT IN` whenever the subquery's column might contain `NULL`.**

12.5 JOIN vs EXISTS vs IN vs Subquery — When to Use Each

Technique	Best For
`JOIN`	Need columns from *both* tables in the result.
`EXISTS/NOT EXISTS`	Only checking for presence/absence of a related row; safest with `NULL`s; often optimizes very well.
`IN`	Simple, short, known-safe (non-`NULL`) static or subquery lists.
Correlated scalar subquery	Need a single computed value per outer row, expressed inline in `SELECT`.

13. CTEs

13.1 Basic CTE — WITH

```
WITH HighValueOrders AS (
    SELECT CustomerID, TotalAmount FROM Sales.Orders WHERE TotalAmount
> 1000
)
SELECT CustomerID, COUNT(*) AS OrderCount
FROM HighValueOrders
GROUP BY CustomerID;
```

A CTE (Common Table Expression) is standard SQL, identical in syntax and purpose to PostgreSQL/MySQL 8+ — a named, temporary result set scoped to the single statement that follows it, primarily used for readability by breaking a complex query into named logical steps.

13.2 Multiple CTEs and CTE Chains

```
WITH RegionalSales AS (
    SELECT Region, SUM(TotalAmount) AS Revenue FROM Sales.Orders GROUP
BY Region
),
RankedRegions AS (
    SELECT Region, Revenue, RANK() OVER (ORDER BY Revenue DESC) AS
RevenueRank
    FROM RegionalSales
)
SELECT * FROM RankedRegions WHERE RevenueRank <= 3;
```

Each CTE can reference any CTE defined *earlier* in the same `WITH` clause, chaining transformations into clearly named stages — dramatically more readable than deeply nested subqueries for multi-step analytical logic.

13.3 Recursive CTEs

```
-- Organizational hierarchy: find every employee reporting up to a
given manager
WITH OrgChart AS (
    -- Anchor member: the top-level manager
    SELECT EmployeeID, FullName, ManagerID, 0 AS Level
    FROM HR.Employees WHERE ManagerID IS NULL

    UNION ALL

    -- Recursive member: joins back to OrgChart itself
    SELECT e.EmployeeID, e.FullName, e.ManagerID, oc.Level + 1
    FROM HR.Employees e
    INNER JOIN OrgChart oc ON e.ManagerID = oc.EmployeeID
)
SELECT * FROM OrgChart ORDER BY Level, FullName;
```

Part	Explanation
Anchor member	The starting point — rows with no manager (the top of the hierarchy).
`UNION ALL`	Combines the anchor with each successive recursive pass; must be `UNION ALL`, not `UNION`, in SQL Server's recursive CTE syntax.
Recursive member	References `OrgChart` itself, joining each pass's newly found rows to find the *next* level down; SQL Server automatically repeats this until no new rows are found.

Recursive CTEs are standard SQL, working nearly identically in PostgreSQL (`WITH RECURSIVE`, note the required `RECURSIVE` keyword PostgreSQL needs that SQL Server does not) — a genuinely important syntax difference to remember when porting. MySQL added recursive CTE support only in version 8.0+.

Business use cases: organizational charts (above), bill-of-materials explosion (a product made of sub-parts, themselves made of sub-parts), date/number sequence generation, and graph-like relationship traversal (finding all paths between connected nodes).

```
-- Generating a date sequence with a recursive CTE
WITH DateSeries AS (
    SELECT CAST('2026-01-01' AS DATE) AS TheDate
    UNION ALL
    SELECT DATEADD(DAY, 1, TheDate) FROM DateSeries WHERE TheDate <
'2026-01-31'
)
SELECT TheDate FROM DateSeries OPTION (MAXRECURSION 100);
```

`OPTION (MAXRECURSION n)` is **[T-SQL-specific]** — SQL Server defaults to a maximum of 100 recursion levels as a safety limit against runaway/infinite recursive CTEs; this must be raised explicitly (or set to `0` for unlimited, with caution) for a recursion depth genuinely expected to exceed 100.

13.4 CTE vs Subquery vs Temporary Table vs Table Variable vs View

Construct	Scope/Lifetime	Reusable Across Statements?	Can Be Indexed?
CTE	Single statement only	No	No
Subquery	Single statement only	No	No
Temporary table (`#Table`)	Session (or until dropped)	Yes, within the session	Yes
Table variable (`@Table`)	Batch/procedure scope	Yes, within that scope	Limited (Part 15)
View	Permanent database object	Yes, across sessions/users	Yes, if an **indexed view** (Part 19)

Covered further with concrete guidance on choosing between these in Part 15.

14. Window Functions

14.1 OVER, PARTITION BY, ORDER BY

```
SELECT CustomerID, OrderDate, TotalAmount,
    SUM(TotalAmount) OVER (PARTITION BY CustomerID ORDER BY OrderDate)
AS RunningTotal
FROM Sales.Orders;
```

Window function syntax and behavior are standard SQL, essentially identical across PostgreSQL, MySQL 8+, and SQL Server — `OVER (PARTITION BY ... ORDER BY ...)` defines the "window" of rows each row's calculation considers, without collapsing rows the way `GROUP BY` does.

14.2 Ranking Functions

```
SELECT CustomerID, TotalAmount,
    ROW_NUMBER() OVER (ORDER BY TotalAmount DESC) AS RowNum,
    RANK()       OVER (ORDER BY TotalAmount DESC) AS Rnk,
    DENSE_RANK() OVER (ORDER BY TotalAmount DESC) AS DenseRnk,
    NTILE(4)     OVER (ORDER BY TotalAmount DESC) AS Quartile
FROM Sales.Orders;
```

Function	Behavior on Ties
`ROW_NUMBER()`	Always unique, sequential — ties broken arbitrarily (by physical order unless `ORDER BY` fully disambiguates).
`RANK()`	Ties share the same rank; the *next* rank skips accordingly (1, 2, 2, 4).
`DENSE_RANK()`	Ties share the same rank; the next rank does **not** skip (1, 2, 2, 3).
`NTILE(n)`	Divides rows into `n` roughly equal buckets — useful for quartiles/deciles.

14.3 LAG, LEAD, FIRST_VALUE, LAST_VALUE

```
SELECT CustomerID, OrderDate, TotalAmount,
    LAG(TotalAmount, 1) OVER (PARTITION BY CustomerID ORDER BY
OrderDate) AS PreviousOrderAmount,
    LEAD(TotalAmount, 1) OVER (PARTITION BY CustomerID ORDER BY
OrderDate) AS NextOrderAmount,
    FIRST_VALUE(TotalAmount) OVER (PARTITION BY CustomerID ORDER BY
OrderDate) AS FirstOrderAmount,
    LAST_VALUE(TotalAmount) OVER (
        PARTITION BY CustomerID ORDER BY OrderDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS LastOrderAmount
FROM Sales.Orders;
```

Common Mistake: `LAST_VALUE` without an explicit frame (`ROWS BETWEEN ... AND ...`) defaults to `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, meaning it

returns the *current row's own value*, not the true last value in the partition — one of the most frequently mis-used window functions across all three databases; always specify the frame explicitly when using LAST_VALUE.

14.4 Running Totals, Moving Averages, Percentages

```
SELECT OrderDate, TotalAmount,
    SUM(TotalAmount) OVER (ORDER BY OrderDate ROWS UNBOUNDED
PRECEDING) AS RunningTotal,
    AVG(TotalAmount) OVER (ORDER BY OrderDate ROWS BETWEEN 6 PRECEDING
AND CURRENT ROW) AS SevenDayMovingAvg,
    TotalAmount * 100.0 / SUM(TotalAmount) OVER () AS PercentOfTotal
FROM Sales.Orders;
```

14.5 Advanced Business Analytics Examples

```
-- Top customers by total spend
SELECT TOP 10 CustomerID, SUM(TotalAmount) AS TotalSpent,
    RANK() OVER (ORDER BY SUM(TotalAmount) DESC) AS SpendRank
FROM Sales.Orders GROUP BY CustomerID ORDER BY TotalSpent DESC;

-- Monthly revenue ranking
SELECT
    FORMAT(OrderDate, 'yyyy-MM') AS OrderMonth,
    SUM(TotalAmount) AS MonthlyRevenue,
    RANK() OVER (ORDER BY SUM(TotalAmount) DESC) AS MonthRank
FROM Sales.Orders GROUP BY FORMAT(OrderDate, 'yyyy-MM');

-- Employee salary ranking within department
SELECT EmployeeID, Department, Salary,
    RANK() OVER (PARTITION BY Department ORDER BY Salary DESC) AS
DeptSalaryRank
FROM HR.Employees;

-- Month-over-month growth
WITH MonthlyRevenue AS (
    SELECT FORMAT(OrderDate, 'yyyy-MM') AS OrderMonth,
SUM(TotalAmount) AS Revenue
    FROM Sales.Orders GROUP BY FORMAT(OrderDate, 'yyyy-MM')
)
SELECT OrderMonth, Revenue,
    LAG(Revenue) OVER (ORDER BY OrderMonth) AS PreviousMonthRevenue,
    (Revenue - LAG(Revenue) OVER (ORDER BY OrderMonth)) * 100.0
        / NULLIF(LAG(Revenue) OVER (ORDER BY OrderMonth), 0) AS
MoMGrowthPercent
FROM MonthlyRevenue;

-- Year-over-year growth
WITH YearlyRevenue AS (
    SELECT YEAR(OrderDate) AS OrderYear, SUM(TotalAmount) AS Revenue
    FROM Sales.Orders GROUP BY YEAR(OrderDate)
)
```

```sql
SELECT OrderYear, Revenue,
    Revenue - LAG(Revenue) OVER (ORDER BY OrderYear) AS YoYChange,
    (Revenue - LAG(Revenue) OVER (ORDER BY OrderYear)) * 100.0
        / NULLIF(LAG(Revenue) OVER (ORDER BY OrderYear), 0) AS
YoYGrowthPercent
FROM YearlyRevenue;

-- Customer retention: did this customer also order in the prior
month?
WITH MonthlyCustomers AS (
    SELECT DISTINCT CustomerID, FORMAT(OrderDate, 'yyyy-MM') AS
OrderMonth
    FROM Sales.Orders
)
SELECT CustomerID, OrderMonth,
    CASE WHEN LAG(OrderMonth) OVER (PARTITION BY CustomerID ORDER BY
OrderMonth)
              = FORMAT(DATEADD(MONTH, -1, CAST(OrderMonth + '-01' AS
DATE)), 'yyyy-MM')
         THEN 'Retained' ELSE 'New/Returning' END AS RetentionStatus
FROM MonthlyCustomers;
```

Note `NULLIF(..., 0)` guarding every growth-percentage calculation's denominator — a direct, practical application of Part 9's NULL-handling functions preventing a `ZeroDivisionError`-equivalent (`Msg 8134: Divide by zero error`) when a prior period had exactly zero revenue.

---

15. Temporary Objects

15.1 Local Temporary Tables — #Table

```sql
CREATE TABLE #TempOrders (OrderID INT, TotalAmount DECIMAL(10,2));
INSERT INTO #TempOrders SELECT OrderID, TotalAmount FROM Sales.Orders
WHERE Status = 'Pending';
SELECT * FROM #TempOrders;
DROP TABLE #TempOrders;   -- optional; auto-dropped at session end (or
procedure end, if created inside one)
```

A `#Table` is **[T-SQL-specific]** syntax (the # prefix) — visible only within the creating session (or, if created inside a stored procedure, that procedure's scope and anything it calls), physically stored in `tempdb` (Part 40), and automatically dropped when the session/scope ends. PostgreSQL uses `CREATE TEMP TABLE`/`CREATE TEMPORARY TABLE` (no special prefix, session-scoped); MySQL uses `CREATE TEMPORARY TABLE` similarly.

15.2 Global Temporary Tables — ##Table

```sql
CREATE TABLE ##SharedStaging (ID INT, Value NVARCHAR(100));
```

A `##Table` (double-hash prefix) is visible to **every** session on the instance, and persists until the creating session ends *and* no other session is still referencing it — a genuinely SQL Server-

specific concept with no direct PostgreSQL/MySQL equivalent; used rarely, mainly for cross-session coordination in specific administrative scenarios.

15.3 Table Variables

```
DECLARE @TempOrders TABLE (OrderID INT, TotalAmount DECIMAL(10,2));
INSERT INTO @TempOrders SELECT OrderID, TotalAmount FROM Sales.Orders
WHERE Status = 'Pending';
SELECT * FROM @TempOrders;
```

A table variable (`@Name TABLE (...)`) is scoped to the current batch/procedure, cannot have most explicit indexes added after creation (only inline `PRIMARY KEY/UNIQUE` constraints, which do create supporting indexes), and — importantly — has historically been assumed by the optimizer to hold exactly one row for cardinality estimation purposes in older SQL Server versions (improved by "table variable deferred compilation" in SQL Server 2019+, but still generally less statistics-aware than a temp table).

15.4 Temporary Table Indexing

```
CREATE TABLE #TempOrders (OrderID INT PRIMARY KEY, CustomerID INT,
TotalAmount DECIMAL(10,2));
CREATE INDEX IX_TempOrders_CustomerID ON #TempOrders (CustomerID);
```

Temp tables support full indexing, statistics, and constraints just like permanent tables — a key practical advantage over table variables for larger intermediate result sets where the optimizer's query plan quality genuinely benefits from real statistics.

15.5 Temporary Table vs Table Variable vs CTE — When Professionals Use Each

Situation	Prefer
Small, simple intermediate set, single statement	CTE
Intermediate set reused across multiple statements, small-to-medium size	Table variable
Large intermediate set, needs indexing, benefits from real statistics for the optimizer	Temp table (`#Table`)
Result set needs to survive and be queried across multiple batches within a stored procedure with complex logic	Temp table
Rollback behavior matters: table variables are **not** rolled back by an enclosing transaction rollback (a genuinely distinctive, sometimes-useful SQL Server behavior), while temp tables *are* rolled back	Table variable, when audit/logging data must survive a rolled-back transaction

16. APPLY Operators

16.1 CROSS APPLY and OUTER APPLY — SQL Server-Specific

```
-- [SQL Server / T-SQL feature - no direct PostgreSQL/MySQL
equivalent]
-- CROSS APPLY: like INNER JOIN, but the right side can reference
columns from the left
SELECT c.CustomerID, c.FullName, recent.OrderID, recent.OrderDate
FROM Sales.Customers c
CROSS APPLY (
    SELECT TOP 1 OrderID, OrderDate
    FROM Sales.Orders o
    WHERE o.CustomerID = c.CustomerID
    ORDER BY o.OrderDate DESC
) AS recent;

-- OUTER APPLY: like LEFT JOIN - keeps customers with NO matching
rows, NULLs filled in
SELECT c.CustomerID, c.FullName, recent.OrderID
FROM Sales.Customers c
OUTER APPLY (
    SELECT TOP 1 OrderID, OrderDate FROM Sales.Orders o
    WHERE o.CustomerID = c.CustomerID ORDER BY o.OrderDate DESC
) AS recent;
```

16.2 How APPLY Differs From Ordinary JOINs

A standard `JOIN`'s `ON` condition cannot reference a *derived, per-row computed table* on its right side — `APPLY` specifically allows the right-hand table expression to be a subquery (or table-valued function) that is **re-evaluated once per row of the left side**, with full access to that row's columns. This is precisely what makes the "latest order per customer" pattern above possible in a single query — a plain `JOIN` to a `TOP 1 ... ORDER BY` subquery cannot correlate that `TOP 1` to each specific customer the way `CROSS APPLY` can. PostgreSQL's closest equivalent is a `LATERAL JOIN` (`JOIN LATERAL (...)`), which serves the same purpose with different syntax; MySQL added `LATERAL` derived table support only in version 8.0.14+, and it remains less commonly used there than `CROSS APPLY` is in SQL Server code.

16.3 Advanced Examples

```
-- Table-valued function combined with APPLY
CREATE FUNCTION dbo.GetTopProducts(@CustomerID INT, @N INT)
RETURNS TABLE
AS
RETURN (
    SELECT TOP (@N) p.ProductName, SUM(oi.Quantity) AS TotalQty
    FROM Sales.OrderItems oi
    JOIN Sales.Orders o ON oi.OrderID = o.OrderID
    JOIN Sales.Products p ON oi.ProductID = p.ProductID
    WHERE o.CustomerID = @CustomerID
    GROUP BY p.ProductName
    ORDER BY TotalQty DESC
```

```
);

SELECT c.CustomerID, t.ProductName, t.TotalQty
FROM Sales.Customers c
CROSS APPLY dbo.GetTopProducts(c.CustomerID, 3) AS t;

-- Top-N-per-group: top 2 orders by amount, per customer
SELECT c.CustomerID, top2.OrderID, top2.TotalAmount
FROM Sales.Customers c
CROSS APPLY (
    SELECT TOP 2 OrderID, TotalAmount FROM Sales.Orders o
    WHERE o.CustomerID = c.CustomerID ORDER BY TotalAmount DESC
) AS top2;

-- Extracting values from a JSON array column, per row (see also Part
31)
SELECT o.OrderID, item.value AS ItemJson
FROM Sales.OrdersWithJson o
CROSS APPLY OPENJSON(o.ItemsJson) AS item;
```

`CROSS APPLY`/`OUTER APPLY` combined with `OPENJSON` (Part 31) or a table-valued function are the standard, idiomatic T-SQL way to "unnest" or "explode" one row into several related rows computed per-row — a pattern PostgreSQL developers would reach for `LATERAL` to accomplish, and MySQL developers would often resort to a correlated subquery or application-side logic for, historically.

---

17. PIVOT and UNPIVOT

17.1 PIVOT — Rows Into Columns

```
-- [SQL Server / T-SQL feature]
SELECT ProductCategory, [2024], [2025], [2026]
FROM (
    SELECT ProductCategory, YEAR(OrderDate) AS OrderYear, TotalAmount
    FROM Sales.Orders o JOIN Sales.Products p ON o.ProductID =
p.ProductID
) AS SourceData
PIVOT (
    SUM(TotalAmount) FOR OrderYear IN ([2024], [2025], [2026])
) AS PivotTable;
```

`PIVOT` transforms distinct values from one column (`OrderYear`) into separate output columns, aggregating (`SUM`) the measure for each — turning "tall" data (one row per order) into a "wide" report layout (one row per category, one column per year), directly useful for financial/BI reporting tables. PostgreSQL has no native `PIVOT` keyword at all (achieved instead via `CASE`-based conditional aggregation, or the `crosstab()` function from the `tablefunc` extension); MySQL likewise has no native `PIVOT`, relying on the same `CASE`-based manual approach.

17.2 UNPIVOT — Columns Into Rows

```
SELECT ProductCategory, OrderYear, Revenue
FROM (
    SELECT ProductCategory, [2024], [2025], [2026] FROM
Sales.YearlySummary
) AS SourceData
UNPIVOT (
    Revenue FOR OrderYear IN ([2024], [2025], [2026])
) AS UnpivotTable;
```

`UNPIVOT` performs the reverse transformation — converting several columns back into rows, useful for normalizing a wide reporting table back into a "tall", analysis-friendly shape.

17.3 Dynamic PIVOT Concepts

```
DECLARE @columns NVARCHAR(MAX), @sql NVARCHAR(MAX);

SELECT @columns = STRING_AGG(QUOTENAME(CAST(OrderYear AS NVARCHAR)),
',')
FROM (SELECT DISTINCT YEAR(OrderDate) AS OrderYear FROM Sales.Orders)
AS Years;

SET @sql = N'
SELECT ProductCategory, ' + @columns + N'
FROM (SELECT ProductCategory, YEAR(OrderDate) AS OrderYear,
TotalAmount
      FROM Sales.Orders o JOIN Sales.Products p ON o.ProductID =
p.ProductID) AS src
PIVOT (SUM(TotalAmount) FOR OrderYear IN (' + @columns + N')) AS
pvt;';

EXEC sp_executesql @sql;
```

Because `PIVOT`'s column list must be known at parse time, a **dynamic** pivot (where the actual set of column values isn't known until runtime — e.g. "however many distinct years exist in the data") requires building and executing the query as a string via dynamic SQL (Part 36) — `QUOTENAME()` here safely brackets each generated column name, and `sp_executesql` is the safe way to execute the constructed statement.

Business use cases: monthly/yearly financial summary reports, sales-by-region-by-quarter dashboards, survey-response cross-tabulations — any report whose natural presentation is a matrix/grid rather than a tall table.

---

18. Set Operations

18.1 UNION, UNION ALL, INTERSECT, EXCEPT

```
-- UNION: combines results, removes duplicates (requires a sort/dedup
pass - has a cost)
SELECT CustomerID FROM Sales.Customers WHERE Region = 'North'
```

```sql
UNION
SELECT CustomerID FROM Sales.Loyalty WHERE Tier = 'Gold';

-- UNION ALL: combines results, KEEPS duplicates (no dedup pass -
typically faster)
SELECT CustomerID FROM Sales.Customers WHERE Region = 'North'
UNION ALL
SELECT CustomerID FROM Sales.Loyalty WHERE Tier = 'Gold';

-- INTERSECT: only rows present in BOTH result sets
SELECT CustomerID FROM Sales.Customers WHERE Region = 'North'
INTERSECT
SELECT CustomerID FROM Sales.Loyalty WHERE Tier = 'Gold';

-- EXCEPT: rows in the first result set, absent from the second (like
a "set subtraction")
SELECT CustomerID FROM Sales.Customers
EXCEPT
SELECT CustomerID FROM Sales.Orders;   -- customers who have never
placed an order
```

18.2 Duplicate Handling, Column Compatibility, Performance

Every set operation requires the same number of columns, in compatible types, across all combined queries — identical requirement across all three databases.
`UNION`/`INTERSECT`/`EXCEPT` (without `ALL`) all perform an implicit deduplication pass (conceptually similar to `DISTINCT`), which has a genuine performance cost on large result sets; prefer `UNION ALL` whenever duplicates are already known to be impossible or simply don't matter for the use case.

18.3 SQL Server vs PostgreSQL vs MySQL

Operation	SQL Server	PostgreSQL	MySQL
`UNION` / `UNION ALL`	Supported	Supported	Supported
`INTERSECT`	Supported	Supported	Supported (MySQL 8.0.31+; earlier versions lack it entirely)
`EXCEPT`	Supported (keyword `EXCEPT`)	Supported (keyword `EXCEPT`)	Supported since 8.0.31 (keyword `EXCEPT`; MySQL never used the older `MINUS` alias some other databases use)

Set-operation *syntax* is essentially uniform across modern versions of all three; the main practical gotcha when porting is confirming the target MySQL version actually supports `INTERSECT`/`EXCEPT` at all, since support arrived comparatively recently there.

19. Views

19.1 CREATE / ALTER / DROP VIEW

```
CREATE VIEW Sales.vw_CustomerOrderSummary AS
SELECT c.CustomerID, c.FullName, COUNT(o.OrderID) AS OrderCount,
SUM(o.TotalAmount) AS TotalSpent
FROM Sales.Customers c
LEFT JOIN Sales.Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.FullName;

ALTER VIEW Sales.vw_CustomerOrderSummary AS
SELECT c.CustomerID, c.FullName, COUNT(o.OrderID) AS OrderCount,
SUM(o.TotalAmount) AS TotalSpent,
       MAX(o.OrderDate) AS LastOrderDate
FROM Sales.Customers c LEFT JOIN Sales.Orders o ON c.CustomerID =
o.CustomerID
GROUP BY c.CustomerID, c.FullName;

DROP VIEW Sales.vw_CustomerOrderSummary;
```

19.2 Why Views Exist

Views provide **security** (granting access to a view without exposing underlying base tables' full column set or row set — see Part 35's row-level security for a more granular alternative), **abstraction** (hiding complex join/aggregation logic behind a simple `SELECT * FROM view`), **reusability** (one definition, queried by many reports/applications), and simplified **reporting** access for consumers (like Power BI, Part 52) who should query a stable, well-named interface rather than raw, potentially-changing base tables.

19.3 Standard Views vs Indexed Views

```
-- [SQL Server / T-SQL feature] - an indexed (materialized) view
CREATE VIEW Sales.vw_DailyRevenue
WITH SCHEMABINDING
AS
SELECT OrderDate, SUM(TotalAmount) AS Revenue, COUNT_BIG(*) AS
OrderCount
FROM Sales.Orders
GROUP BY OrderDate;
GO

CREATE UNIQUE CLUSTERED INDEX IX_vw_DailyRevenue ON
Sales.vw_DailyRevenue (OrderDate);
```

A standard view is just stored SQL text, re-executed against current data every time it's queried. An **indexed view** (created with `WITH SCHEMABINDING`, then given a `UNIQUE CLUSTERED INDEX`) physically materializes and persists its result set on disk, automatically kept in sync as underlying data changes — SQL Server's equivalent of PostgreSQL's `MATERIALIZED VIEW`, with one crucial difference: **a SQL Server indexed view updates automatically and**

incrementally with every underlying data change, while a PostgreSQL materialized view requires an explicit `REFRESH MATERIALIZED VIEW` command and is stale until then. MySQL has no materialized/indexed view feature at all.

19.4 Requirements and Limitations of Indexed Views

`SCHEMABINDING` locks the view's definition to the exact current structure of its underlying tables (preventing those tables from being altered in incompatible ways without first dropping the view); the view's `SELECT` cannot use `SELECT *`, non-deterministic functions, outer joins, or several other constructs; `COUNT(*)` must be written as `COUNT_BIG(*)`; and the first index created must be a `UNIQUE CLUSTERED` index specifically. These restrictions exist because the engine must be able to maintain the materialized result incrementally and deterministically on every underlying write — a genuine engineering trade-off between query-time speed and both write-time overhead and definitional flexibility.

---

20. Stored Procedures

20.1 CREATE / ALTER PROCEDURE, EXEC

```
CREATE PROCEDURE Sales.usp_CreateCustomer
    @FullName NVARCHAR(100),
    @Email NVARCHAR(255),
    @NewCustomerID INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO Sales.Customers (FullName, Email) VALUES (@FullName,
@Email);
    SET @NewCustomerID = SCOPE_IDENTITY();
END;
GO

DECLARE @NewID INT;
EXEC Sales.usp_CreateCustomer @FullName = 'Ada Lovelace', @Email =
'ada@example.com', @NewCustomerID = @NewID OUTPUT;
SELECT @NewID;
```

`SET NOCOUNT ON` suppresses the "(1 row affected)" messages SQL Server sends after each statement — a small but genuine performance and clean-output habit in professional procedure code, since those messages otherwise generate unnecessary network round-trip chatter.

20.2 Parameters — Input, Output, Defaults, Return Values

```
CREATE PROCEDURE Sales.usp_GetOrdersByStatus
    @Status NVARCHAR(20) = 'Pending',   -- default parameter value
    @OrderCount INT OUTPUT
AS
BEGIN
```

```
    SET NOCOUNT ON;
    SELECT * FROM Sales.Orders WHERE Status = @Status;
    SET @OrderCount = @@ROWCOUNT;
    RETURN 0;   -- integer return code, conventionally used for
status/error codes, not data
END;
```

RETURN in a T-SQL procedure can only return a single `int` status code (by strong convention, `0` for success) — actual result data comes back via SELECT result sets or OUTPUT parameters, not RETURN, which is a genuine mechanical difference from how a Python or application-level function returns a value.

20.3 Dynamic SQL, Error Handling, Transactions Inside Procedures

Covered in full depth in Parts 25 (error handling) and 36 (dynamic SQL); a production-grade procedure typically combines TRY/CATCH, an explicit transaction, and input validation together, as shown in the realistic examples below.

20.4 Realistic Stored Procedures

```
-- Process a transaction, with proper error handling and transaction
control
CREATE PROCEDURE Banking.usp_ProcessTransaction
    @AccountID INT,
    @Amount DECIMAL(12,2),
    @TransactionType NVARCHAR(10)   -- 'Deposit' or 'Withdrawal'
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;

        IF @TransactionType = 'Withdrawal'
        BEGIN
            IF (SELECT Balance FROM Banking.Accounts WHERE AccountID =
@AccountID) < @Amount
                THROW 50001, 'Insufficient funds', 1;

            UPDATE Banking.Accounts SET Balance = Balance - @Amount
WHERE AccountID = @AccountID;
        END
        ELSE
        BEGIN
            UPDATE Banking.Accounts SET Balance = Balance + @Amount
WHERE AccountID = @AccountID;
        END

        INSERT INTO Banking.TransactionLog (AccountID, Amount,
TransactionType, CreatedAt)
        VALUES (@AccountID, @Amount, @TransactionType, SYSDATETIME());
```

```
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;

-- Generate a monthly sales report
CREATE PROCEDURE Sales.usp_MonthlySalesReport
    @Year INT, @Month INT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT p.ProductCategory, SUM(oi.Quantity * oi.UnitPrice) AS
Revenue, COUNT(DISTINCT o.OrderID) AS OrderCount
    FROM Sales.Orders o
    JOIN Sales.OrderItems oi ON o.OrderID = oi.OrderID
    JOIN Sales.Products p ON oi.ProductID = p.ProductID
    WHERE YEAR(o.OrderDate) = @Year AND MONTH(o.OrderDate) = @Month
    GROUP BY p.ProductCategory
    ORDER BY Revenue DESC;
END;

-- Calculate employee bonus
CREATE PROCEDURE HR.usp_CalculateBonus
    @EmployeeID INT, @Bonus DECIMAL(10,2) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @Sales DECIMAL(12,2) = (SELECT SUM(SalesAmount) FROM
HR.EmployeeSales WHERE EmployeeID = @EmployeeID);
    SET @Bonus = CASE
        WHEN @Sales > 100000 THEN @Sales * 0.05
        WHEN @Sales > 50000  THEN @Sales * 0.03
        ELSE 0
    END;
END;

-- Detect suspicious transactions (simple rule-based flagging)
CREATE PROCEDURE Banking.usp_DetectSuspiciousTransactions
AS
BEGIN
    SET NOCOUNT ON;
    SELECT t.TransactionID, t.AccountID, t.Amount, t.CreatedAt
    FROM Banking.TransactionLog t
    WHERE t.Amount > 10000
       OR (SELECT COUNT(*) FROM Banking.TransactionLog t2
           WHERE t2.AccountID = t.AccountID
             AND t2.CreatedAt >= DATEADD(MINUTE, -10, t.CreatedAt)) >
```

```
5;
END;
```

21. User-Defined Functions

21.1 Scalar Functions

```
CREATE FUNCTION dbo.CalculateAge (@BirthDate DATE)
RETURNS INT
AS
BEGIN
    RETURN DATEDIFF(YEAR, @BirthDate, GETDATE())
        - CASE WHEN (MONTH(@BirthDate) > MONTH(GETDATE()))
                  OR (MONTH(@BirthDate) = MONTH(GETDATE()) AND
DAY(@BirthDate) > DAY(GETDATE()))
               THEN 1 ELSE 0 END;
END;

SELECT FullName, dbo.CalculateAge(BirthDate) AS Age FROM HR.Employees;
```

21.2 Inline Table-Valued Functions

```
CREATE FUNCTION Sales.GetCustomerOrders (@CustomerID INT)
RETURNS TABLE
AS
RETURN (
    SELECT OrderID, OrderDate, TotalAmount FROM Sales.Orders WHERE
CustomerID = @CustomerID
);

SELECT * FROM Sales.GetCustomerOrders(101);
```

An inline TVF is essentially a parameterized view — a single `RETURN (SELECT ...)` statement, no `BEGIN/END` body — and the query optimizer can **inline** its logic directly into the calling query's execution plan, generally performing very well, comparable to a hand-written subquery.

21.3 Multi-Statement Table-Valued Functions

```
CREATE FUNCTION Sales.GetCustomerSummary (@CustomerID INT)
RETURNS @Summary TABLE (OrderCount INT, TotalSpent DECIMAL(12,2))
AS
BEGIN
    INSERT INTO @Summary
    SELECT COUNT(*), SUM(TotalAmount) FROM Sales.Orders WHERE
CustomerID = @CustomerID;
    RETURN;
END;

SELECT * FROM Sales.GetCustomerSummary(101);
```

A multi-statement TVF explicitly declares a return table variable and populates it procedurally — more flexible (multiple statements, branching logic) but historically optimized poorly, since the engine has traditionally assumed a fixed, small row-count estimate for its output regardless of actual data volume, which can produce poor execution plans when the result set is actually

large (improved in recent SQL Server versions but still worth being cautious about for performance-critical paths).

21.4 Functions vs Stored Procedures

Aspect	Scalar/Table-Valued Function	Stored Procedure
Usable directly inside `SELECT/WHERE/JOIN`	Yes	No
Can perform data modification (`INSERT/UPDATE/DELETE`)	No	Yes
Can use `TRY/CATCH`	No (functions cannot)	Yes
Can execute dynamic SQL	No	Yes
Transaction control (`BEGIN TRAN`, etc.)	No	Yes
Typical use	Reusable computed value/table logic embedded in queries	Business-process logic, data modification, orchestration

When to use functions: any reusable calculation or reusable filtered/derived row set meant to be composed directly into other queries. **When to use procedures:** anything that modifies data, needs transaction/error-handling control, or represents a complete business operation (Part 20's examples) rather than a pure, composable calculation.

---

22. Triggers

22.1 DML Triggers — AFTER and INSTEAD OF

```
-- AFTER trigger: fires after the triggering action completes
CREATE TRIGGER Sales.trg_Orders_Audit
ON Sales.Orders
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO Sales.OrderAuditLog (OrderID, OldStatus, NewStatus,
ChangedAt)
    SELECT i.OrderID, d.Status, i.Status, SYSDATETIME()
    FROM inserted i JOIN deleted d ON i.OrderID = d.OrderID
    WHERE i.Status <> d.Status;
END;

-- INSTEAD OF trigger: REPLACES the triggering action entirely -
common on views
CREATE TRIGGER Sales.trg_CustomerView_Insert
ON Sales.vw_ActiveCustomers
```

```
INSTEAD OF INSERT
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO Sales.Customers (FullName, Email, Status)
    SELECT FullName, Email, 'Active' FROM inserted;
END;
```

Triggers use the same `inserted/deleted` pseudo-tables introduced with `OUTPUT` in Part 6 — a nice syntactic consistency within T-SQL. `AFTER` triggers (the more common type) run once the triggering statement's changes are already applied; `INSTEAD OF` triggers intercept the action entirely, running their own logic in its place — most commonly used to make an otherwise non-updatable view (one involving joins or aggregation) support `INSERT/UPDATE/DELETE`.

22.2 DDL Triggers

```
CREATE TRIGGER trg_PreventTableDrop
ON DATABASE
FOR DROP_TABLE
AS
BEGIN
    RAISERROR ('Dropping tables is not permitted outside of
maintenance windows.', 16, 1);
    ROLLBACK;
END;
```

A DDL trigger fires on schema-changing statements (`CREATE/ALTER/DROP`) rather than data-changing ones — used for governance: preventing accidental structural changes, or logging every schema modification for compliance auditing.

22.3 Practical Use Cases

- **Audit trails:** recording who changed what and when (Section 22.1's example), often superseded in modern SQL Server by Temporal Tables (Part 42) or Change Data Capture (Part 43) for this specific purpose.
- **Change tracking:** propagating a change in one table to a derived/denormalized value in another.
- **Security:** DDL triggers restricting schema changes; DML triggers enforcing row-level business rules beyond what a `CHECK` constraint can express.
- **Business rules:** enforcing cross-table invariants a single-table `CHECK` constraint cannot see (e.g. "an order's total must equal the sum of its line items").

22.4 Why Excessive Trigger Usage Can Make Systems Difficult to Maintain

Triggers execute *implicitly* — invisible in the calling application code or ad-hoc query that triggered them — which means a simple-looking `UPDATE` statement can silently cascade into arbitrarily complex hidden logic, additional writes, or even other triggers firing recursively. This makes debugging unexpected behavior and reasoning about performance significantly harder than equivalent logic made *explicit* in a stored procedure or application layer.
Professional guidance: prefer explicit logic in stored procedures/application code where

practical; reserve triggers for genuine cross-cutting concerns (audit, enforced invariants) that must apply unconditionally regardless of which code path performs the write, and keep trigger logic itself as simple and fast as possible.

---

23. Transactions and ACID

23.1 ACID Properties

Property	Meaning
Atomicity	A transaction's changes either all happen, or none do — no partial completion.
Consistency	A transaction moves the database from one valid state to another, never violating constraints.
Isolation	Concurrent transactions behave (to the degree configured, Part 24) as if executed one at a time.
Durability	Once committed, a transaction's changes survive a subsequent crash or power loss.

These properties are identical in concept across PostgreSQL, MySQL (InnoDB), and SQL Server — foundational relational database theory, not vendor-specific.

23.2 BEGIN TRANSACTION, COMMIT, ROLLBACK, SAVE TRANSACTION

```
BEGIN TRANSACTION;

UPDATE Banking.Accounts SET Balance = Balance - 500 WHERE AccountID =
1;
UPDATE Banking.Accounts SET Balance = Balance + 500 WHERE AccountID =
2;

IF @@ERROR <> 0
BEGIN
    ROLLBACK TRANSACTION;
END
ELSE
BEGIN
    COMMIT TRANSACTION;
END

-- SAVE TRANSACTION - a named savepoint within a larger transaction
BEGIN TRANSACTION;
    UPDATE Banking.Accounts SET Balance = Balance - 100 WHERE
AccountID = 1;
    SAVE TRANSACTION BeforeSecondUpdate;

    UPDATE Banking.Accounts SET Balance = Balance + 100 WHERE
```

```
AccountID = 999;  -- suppose this is wrong
    ROLLBACK TRANSACTION BeforeSecondUpdate;   -- undo only back to
the savepoint, not the whole transaction

    UPDATE Banking.Accounts SET Balance = Balance + 100 WHERE
AccountID = 2;   -- corrected
COMMIT TRANSACTION;
```

`SAVE TRANSACTION` (**[T-SQL syntax]**, `SAVEPOINT` in PostgreSQL/MySQL — same concept, different keyword) allows partial rollback to a named point within a larger transaction, without abandoning the entire unit of work — useful for complex multi-step procedures where one sub-step's failure shouldn't necessarily discard everything already done.

---

24. Transaction Isolation

24.1 Isolation Levels

Level	Prevents Dirty Reads?	Prevents Non-Repeatable Reads?	Prevents Phantom Reads?	Mechanism
Read Uncommitted	No	No	No	No locking on reads at all
Read Committed (SQL Server default)	Yes	No	No	Shared locks held only during the individual read
Repeatable Read	Yes	Yes	No	Shared locks held until end of transaction
Serializable	Yes	Yes	Yes	Range locks preventing any concurrent modification of the read range
Snapshot	Yes	Yes	Yes	Row versioning (Section 24.4), not locking
Read Committed Snapshot Isolation (RCSI)	Yes	No	No	Row versioning at the statement level

```
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
BEGIN TRANSACTION;
    SELECT Balance FROM Banking.Accounts WHERE AccountID = 1;
COMMIT TRANSACTION;
```

24.2 Dirty Reads, Non-Repeatable Reads, Phantom Reads

Anomaly	Description
Dirty read	Reading another transaction's *uncommitted* changes, which might later be rolled back.
Non-repeatable read	Re-reading the same row within one transaction returns *different* data because another transaction committed a change in between.
Phantom read	Re-running the same query within one transaction returns *additional or fewer rows* because another transaction inserted/deleted matching rows in between.

24.3 Blocking, Lock Escalation, Deadlocks

Blocking occurs when one transaction holds a lock another transaction needs, forcing it to wait. **Lock escalation** is SQL Server automatically converting many fine-grained row/page locks into a single coarser table lock once a threshold is exceeded, trading concurrency for lock-management overhead. A **deadlock** occurs when two transactions each hold a lock the other needs, forming a cycle with no possible resolution — SQL Server automatically detects deadlocks and terminates one transaction (the "deadlock victim", chosen by estimated rollback cost) to break the cycle, returning error 1205 to the terminated transaction's caller, whose application code should be prepared to retry.

24.4 SQL Server Locking and Row-Versioning — In Depth

By default (`Read Committed`), SQL Server uses **pessimistic, lock-based** concurrency control: readers take shared locks, writers take exclusive locks, and readers can block behind writers. Enabling **Read Committed Snapshot Isolation (RCSI)** at the database level, or explicitly using the `SNAPSHOT` isolation level, switches to **optimistic, row-versioning-based** concurrency: readers see a consistent, appropriately-timestamped *version* of each row from `tempdb`'s version store (Part 40) instead of waiting on a writer's lock at all — dramatically reducing reader/writer blocking, at the cost of `tempdb` version-store overhead and the possibility of write-write conflicts under `SNAPSHOT` isolation specifically (raising error 3960, requiring a retry).

```
ALTER DATABASE Sales SET READ_COMMITTED_SNAPSHOT ON;
```

[SQL Server vs PostgreSQL/MySQL]: PostgreSQL uses MVCC (Multi-Version Concurrency Control) — row versioning — as its *default*, always-on concurrency model, meaning PostgreSQL readers essentially never block behind writers out of the box; SQL Server's traditional default (`Read Committed` without RCSI) is lock-based and *can* experience reader/writer blocking unless RCSI is explicitly enabled — a genuinely important operational difference to know when moving between the two systems. MySQL/InnoDB also defaults to MVCC-based `REPEATABLE READ` isolation, again differing from SQL Server's out-of-the-box locking behavior.

25. Error Handling

25.1 TRY / CATCH

```
BEGIN TRY
    BEGIN TRANSACTION;
    UPDATE Banking.Accounts SET Balance = Balance - 1000000 WHERE
AccountID = 1;
    IF (SELECT Balance FROM Banking.Accounts WHERE AccountID = 1) < 0
        THROW 50001, 'Insufficient funds', 1;
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

    SELECT
        ERROR_NUMBER()    AS ErrorNumber,
        ERROR_MESSAGE()   AS ErrorMessage,
        ERROR_LINE()      AS ErrorLine,
        ERROR_PROCEDURE() AS ErrorProcedure,
        ERROR_SEVERITY()  AS ErrorSeverity,
        ERROR_STATE()     AS ErrorState;
END CATCH
```

`BEGIN TRY/BEGIN CATCH` (**[T-SQL-specific]**) is structurally similar to PostgreSQL's `EXCEPTION` block within a `DO`/function body, and conceptually parallel to application-level `try/except` (as covered for Python in the Testing and Debugging handbook's Part 9) — but scoped specifically to catchable T-SQL errors, and requiring explicit `@@TRANCOUNT` checking and rollback, unlike PostgreSQL's automatic transaction rollback on an unhandled exception within a block.

25.2 Error Information Functions

Function	Returns
`ERROR_NUMBER()`	The numeric error code.
`ERROR_MESSAGE()`	The full error message text.
`ERROR_LINE()`	The line number within the batch/procedure where the error occurred.
`ERROR_PROCEDURE()`	The name of the stored procedure/function where the error occurred (`NULL` if not inside one).
`ERROR_SEVERITY()`	The error's severity level (16 is a common "user-correctable" default).
`ERROR_STATE()`	A caller-defined state value, useful for distinguishing multiple raise points sharing the same error number.

These are only valid **inside a `CATCH` block** — calling them elsewhere returns `NULL`.

25.3 THROW vs Older Patterns

```
-- Modern, preferred [T-SQL, SQL Server 2012+]
THROW 50001, 'Insufficient funds for this withdrawal.', 1;

-- Re-throwing the original caught error, preserving its original
details
BEGIN CATCH
    THROW;   -- no arguments - re-raises the exact error that was
caught
END CATCH

-- Older pattern, still seen in legacy code
RAISERROR ('Insufficient funds for this withdrawal.', 16, 1);
```

Why `THROW` is generally preferred: a bare `THROW` (no arguments) inside a `CATCH` block re-raises the *original* error with its original line number, procedure name, and severity fully intact — `RAISERROR` cannot do this at all, only generate a *new* error referencing the `CATCH` block's own location, losing the original context. `THROW` with arguments also always terminates the batch (severity is effectively always 16, "error"), giving simpler, more predictable behavior than `RAISERROR`'s configurable severity levels, most of which are rarely appropriate for application error signaling. Use `RAISERROR` only when its specific legacy features (like `WITH NOWAIT` for immediate message flushing, useful in some long-running-job logging scenarios) are genuinely required.

---

26. Indexes

26.1 Heap

A table with no clustered index is called a **heap** — rows are stored in no particular physical order, referenced by other indexes via a physical row ID. Generally, professional guidance recommends every table have a clustered index (Section 26.2) rather than remaining a heap, since a heap can suffer from page fragmentation and lacks the ordered-access benefits a clustered index provides.

26.2 Clustered Index

```
CREATE CLUSTERED INDEX IX_Orders_OrderDate ON Sales.Orders
(OrderDate);
```

A clustered index determines the **physical storage order** of the table's data itself — the table's rows *are* the index's leaf level; there can be at most **one** clustered index per table (since data can only be physically sorted one way). By default, `PRIMARY KEY` creates a clustered index automatically unless `NONCLUSTERED` is specified.

26.3 Nonclustered Index

```
CREATE NONCLUSTERED INDEX IX_Orders_CustomerID ON Sales.Orders
(CustomerID);
```

A nonclustered index is a separate structure, holding a copy of the indexed column(s) plus a pointer back to the full row (the clustering key, if the table has a clustered index, or a physical row locator if it's a heap) — a table can have many nonclustered indexes (up to 999), each optimized for a different query pattern.

26.4 Clustered vs Nonclustered — And Comparison With PostgreSQL/MySQL

Aspect	SQL Server	PostgreSQL	MySQL (InnoDB)
Default table storage	Heap, unless a clustered index is created	Heap-like (unless `CLUSTER` is run once, non-maintained afterward)	Always clustered by primary key (InnoDB's fundamental storage model)
Clustered index maintenance	Automatically, continuously maintained	`CLUSTER` command is a one-time physical reorder, not continuously maintained	Automatic and continuous (built into InnoDB)
Max clustered indexes per table	1	N/A (no true continuously-maintained clustered index concept)	Always exactly 1 (the primary key)

This is a genuinely important conceptual difference: MySQL/InnoDB tables are *always* clustered by primary key with no choice in the matter; SQL Server gives you the explicit choice of clustering key (commonly, but not necessarily, the primary key) and continuously maintains it; PostgreSQL's `CLUSTER` is a one-time physical reorganization that immediately begins drifting out of sync as new rows are inserted, unlike SQL Server's always-current clustered index.

26.5 Unique, Composite, Included Columns, Filtered, Covering Indexes

```
-- Unique index
CREATE UNIQUE NONCLUSTERED INDEX IX_Customers_Email ON Sales.Customers
(Email);

-- Composite (multi-column) index - column ORDER matters greatly for
usefulness (Section 26.8)
CREATE NONCLUSTERED INDEX IX_Orders_Customer_Date ON Sales.Orders
(CustomerID, OrderDate);

-- Included columns [T-SQL/SQL Server-specific] - stored at the leaf
level but NOT part of the key
CREATE NONCLUSTERED INDEX IX_Orders_Covering
ON Sales.Orders (CustomerID)
INCLUDE (OrderDate, TotalAmount, Status);

-- Filtered index [T-SQL/SQL Server-specific] - indexes only a SUBSET
of rows
```

```
CREATE NONCLUSTERED INDEX IX_Orders_PendingOnly
ON Sales.Orders (OrderDate)
WHERE Status = 'Pending';
```

Included columns let a nonclustered index become a **covering index** for a specific query — one where every column the query needs (in `SELECT`, `WHERE`, `JOIN`) is present directly in the index itself, letting the engine satisfy the entire query from the (typically much smaller) index alone, without a **key lookup** (Section 26.6) back to the full table row. PostgreSQL added a comparable `INCLUDE` clause for indexes in version 11+; MySQL has no direct equivalent concept, though a composite index can serve a similar covering purpose if all needed columns happen to be part of the key itself.

Filtered indexes index only rows matching a `WHERE` predicate — smaller, faster to maintain, and more selective for queries that always filter on that same condition (like "only pending orders"). PostgreSQL's `CREATE INDEX ... WHERE ...` (a "partial index") is functionally identical; MySQL has no equivalent feature at all.

26.6 Index Seek vs Index Scan vs Key Lookup

Operation	Meaning
Index seek	The engine navigates the index's B-tree structure directly to the needed row(s) — efficient, typically `O(log n)`.
Index scan	The engine reads the *entire* index (or table) sequentially — can still be efficient for large result-set queries, but expensive when only a few rows are actually needed.
Key lookup	After a nonclustered index seek finds matching rows, the engine must go back to the clustered index (or heap) to retrieve any *additional* columns not included in the nonclustered index — an extra, potentially costly random-access step per row, exactly what covering indexes (Section 26.5) eliminate.

Seeing many key lookups in an execution plan (Part 29) for a query touching many rows is a strong, actionable signal that adding `INCLUDE` columns to make the index "covering" would likely improve performance meaningfully.

26.7 B-Tree Structure and Index Selectivity

Both clustered and nonclustered indexes in SQL Server are implemented as **B-trees** — balanced tree structures allowing `O(log n)` seeks — conceptually identical to PostgreSQL's default B-tree index type and MySQL/InnoDB's index structure. **Selectivity** measures what fraction of a table's rows a given predicate typically matches; a highly selective predicate (`WHERE Email = 'specific@example.com'`, matching ~1 row) benefits enormously from an index seek, while a low-selectivity predicate (`WHERE IsActive = 1`, matching 95% of

rows) often performs *better* with a table scan than an index seek plus enormous numbers of key lookups — the optimizer's cost-based decision here is exactly why "just add an index" is not always the right answer (Part 30).

26.8 SARGability and Composite Index Column Ordering

Covered fully in Part 30; briefly: a predicate is **SARGable** (Search ARGument-able) when the engine can use an index to seek directly to matching rows, rather than needing to evaluate a function/expression against every row. Composite index column order should generally place the most selective and/or most commonly-filtered-with-equality column first, since a composite index can only be seek-efficient on a *left-to-right prefix* of its columns — an index on `(CustomerID, OrderDate)` seeks efficiently for `WHERE CustomerID = 5`, and for `WHERE CustomerID = 5 AND OrderDate > '2026-01-01'`, but **not** efficiently for `WHERE OrderDate > '2026-01-01'` alone.

26.9 Over-Indexing and Missing Indexes

Every additional index speeds up *some* reads but slows down **every** `INSERT/UPDATE/DELETE` that touches indexed columns (since each index must also be maintained on every write) and consumes additional storage — over-indexing a heavily-written table is a genuine, common performance mistake (Part 61). SQL Server's Dynamic Management Views (`sys.dm_db_missing_index_details` and related DMVs) surface indexes the optimizer *would* have used had they existed, based on actual query history — a valuable, data-driven starting point for index design, though missing-index suggestions should always be evaluated critically (Section 30) rather than applied blindly, since they don't account for the write-cost trade-off above.

---

27. Columnstore

27.1 Rowstore vs Columnstore

Traditional indexes (Part 26) are **rowstore**: all of a row's column values are stored physically together, efficient for retrieving/modifying whole rows (typical OLTP access patterns). A **columnstore** index instead stores each *column's* values together, contiguously, across all rows — dramatically more efficient for analytical queries that aggregate over millions of rows but touch only a handful of columns, since only the needed columns' data need be read from disk at all, and highly repetitive column data compresses far better stored this way.

27.2 Clustered and Nonclustered Columnstore Indexes

```
-- Clustered columnstore - the table's PRIMARY storage becomes
columnar
CREATE CLUSTERED COLUMNSTORE INDEX CCI_FactSales ON
Warehouse.FactSales;

-- Nonclustered columnstore - an additional columnar copy alongside
normal rowstore storage
```

```
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_Orders ON Sales.Orders
(CustomerID, TotalAmount, OrderDate);
```

A **clustered columnstore index** replaces the table's entire storage with columnar format — the standard choice for dedicated fact tables (Part 51) in a data warehouse. A **nonclustered columnstore index** adds a columnar copy of specified columns *alongside* a table's normal rowstore storage, useful for enabling fast analytical queries against an otherwise transactional (rowstore) table without migrating its primary storage format.

27.3 Compression and Why Columnstore Matters for Analytics

Columnstore indexes use aggressive compression (often 5-10x smaller than equivalent rowstore data) since column-aligned data tends to be highly repetitive (a `Status` column with only 4 distinct values compresses far better stored column-wise than interleaved with unrelated row data), and SQL Server executes analytical operations against columnstore data using **batch mode** processing (operating on ~900-row batches at a time, vectorized, rather than row-by-row) — the combination of far less I/O (only needed columns read, heavily compressed) and vectorized execution is what makes columnstore indexes routinely deliver order-of-magnitude speedups for `GROUP BY`/`SUM`/`AVG`-heavy analytical workloads compared to rowstore, at the cost of being comparatively inefficient for single-row lookups and frequent small updates — precisely why columnstore is recommended for data-warehouse fact tables (Part 51) and analytical workloads, not OLTP tables with heavy transactional row-level activity.

[SQL Server vs PostgreSQL/MySQL]: Neither PostgreSQL nor MySQL includes a native columnstore index type in their core engine (PostgreSQL has third-party columnar extensions like `cstore_fdw`/Citus; MySQL has no first-party equivalent at all) — this is one of SQL Server's most significant built-in analytical-performance advantages over the two, and a major reason SQL Server is frequently chosen as the engine underneath an in-house data warehouse (Part 51).

28. Query Performance

28.1 Execution Plans — Estimated vs Actual

An **estimated execution plan** shows what the optimizer *plans* to do, based on statistics, without actually running the query; an **actual execution plan** runs the query and additionally reports real row counts and timings alongside the same plan shape — always prefer the actual plan when diagnosing a genuinely slow query, since large gaps between *estimated* and *actual* row counts (Section 29's "poor cardinality estimates") are themselves one of the most common root causes of bad plans.

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT CustomerID, SUM(TotalAmount) FROM Sales.Orders GROUP BY
CustomerID;
```

```
Table 'Orders'. Scan count 1, logical reads 4521, physical reads 0,
read-ahead reads 0.
SQL Server Execution Times: CPU time = 156 ms, elapsed time = 203 ms.
```

`SET STATISTICS IO ON` reports **logical reads** (pages read from the buffer cache/memory — the most important number to watch, since it's a stable, hardware-independent measure of query cost) and **physical reads** (pages actually read from disk, indicating the buffer cache didn't already have the data). `SET STATISTICS TIME ON` reports actual CPU and elapsed time. Together, these are the standard first diagnostic step for any "why is this query slow" investigation, cheaper and often more immediately actionable than opening a full graphical execution plan.

28.2 Query Cost and Cardinality Estimation

The optimizer assigns every candidate execution plan an estimated **cost** (a unitless number combining estimated I/O and CPU work) and selects the cheapest plan it can find within a bounded search — it does not guarantee the *globally* optimal plan, only a "good enough, found quickly" one. **Cardinality estimation** — predicting how many rows each operation will produce — drives nearly every downstream decision (which join algorithm, whether to use an index, how much memory to grant); accurate cardinality estimation depends entirely on up-to-date **statistics** (Section 28.3).

28.3 Statistics

```
UPDATE STATISTICS Sales.Orders;
CREATE STATISTICS ST_Orders_Status ON Sales.Orders (Status);
```

Statistics are compact summaries (histograms) of a column's data distribution, automatically created and maintained (`AUTO_CREATE_STATISTICS`/`AUTO_UPDATE_STATISTICS`, on by default) as data changes — stale statistics (common after a large bulk load without a subsequent update) are one of the single most common root causes of a previously-fast query suddenly performing poorly, since the optimizer's cardinality estimates become wrong.

28.4 Join Algorithms

Algorithm	Best Suited For
Nested Loops	Small outer input joined to an index-seekable inner input — very efficient when applicable.
Hash Join	Large, unsorted inputs with no useful index — builds an in-memory hash table from the smaller input, probes it with the larger.
Merge Join	Both inputs already sorted on the join key (e.g. via an index) — very efficient, avoids building a hash table entirely.

The optimizer chooses among these automatically based on estimated input sizes and available indexes/sort order; seeing an unexpected Hash Join where a Nested Loops join was expected (or vice versa) in an execution plan is often a symptom, not a cause — usually traceable back to missing indexes or poor cardinality estimates (Part 29) rather than something to "fix" by forcing a different join type directly.

28.5 Sort Operations, Memory Grants, Spills, Parallelism

A **sort** operation (for `ORDER BY`, some `GROUP BY` strategies, or merge joins needing sorted input) requires memory, requested in advance as a **memory grant** based on cardinality estimates; when the actual data is larger than the granted memory (often because the underlying cardinality estimate was wrong), the sort **spills** to `tempdb` (Part 40) — visible in an actual execution plan as a "Sort Warning" and a dramatic performance cliff, since disk-based spilling is vastly slower than an in-memory sort. **Parallelism** (multiple CPU cores cooperating on one query) can speed up large analytical queries substantially but carries its own coordination overhead, making it inappropriate (and, by default configuration, avoided by the optimizer) for small, fast OLTP-style queries.

28.6 Diagnosing Slow Queries — A Practical Methodology

1. Confirm the query is actually slow in isolation (rule out application-side or network factors first).
2. Capture the **actual** execution plan (Section 28.1), not just the estimated one.
3. Compare estimated vs actual row counts at each operator — large discrepancies point to stale statistics or non-SARGable predicates (Part 30).
4. Check `SET STATISTICS IO ON` output for unexpectedly high logical reads on any one table.
5. Look for key lookups (Section 26.6), sort spills (Section 28.5), or unexpected scans where a seek would be expected.
6. Only then consider specific fixes: new/adjusted indexes, updated statistics, or query rewriting (Part 30) — in that order of investigation, never index changes as a first, unverified guess.

---

29. Explaining Execution Plans

29.1 Reading a Plan From Right to Left, Top to Bottom

SQL Server execution plans are read **right to left**, with data logically "flowing" from the rightmost (typically a table/index access) operators toward the leftmost (final `SELECT`) operator — each operator's **relative cost percentage** (shown as a percentage of the total batch cost) is the first place to look for the actual bottleneck within a larger plan.

29.2 A Good Execution Plan Example

```
SELECT OrderID, TotalAmount FROM Sales.Orders WHERE CustomerID =
12345;
-- With an index on CustomerID: Index Seek (cost ~2%) -> Key Lookup
(if not covering) -> SELECT
-- With a COVERING index (CustomerID INCLUDE (OrderID, TotalAmount)):
Index Seek ONLY, no lookup at all
```

A "good" plan for a highly selective, point-lookup-style query shows an **Index Seek** with a small estimated row count matching the actual row count closely, minimal or no key lookups, and low relative cost.

29.3 A Bad Execution Plan Example — Missing Index

```
SELECT OrderID, TotalAmount FROM Sales.Orders WHERE CustomerID =
12345;
-- With NO index on CustomerID: Clustered Index Scan / Table Scan
(cost ~98%)
```

Without a usable index, the engine must scan the entire table (or clustered index), evaluating the `WHERE` predicate against every single row — the relative cost is overwhelmingly dominated by this one operator, and the fix (adding an index on `CustomerID`) is directly suggested by SQL Server's "Missing Index" recommendation, often shown directly in the graphical plan.

29.4 Non-SARGable Predicates and Implicit Conversions

```
-- BAD - wrapping the indexed column in a function defeats seekability
entirely
SELECT * FROM Sales.Orders WHERE YEAR(OrderDate) = 2026;

-- GOOD - SARGable equivalent, can use an index on OrderDate via a
seek
SELECT * FROM Sales.Orders WHERE OrderDate >= '2026-01-01' AND
OrderDate < '2027-01-01';

-- BAD - implicit conversion: comparing an nvarchar column to a
varchar literal forces
-- SQL Server to convert EVERY row's value for comparison, defeating
an index seek
SELECT * FROM Sales.Customers WHERE Email = 'ada@example.com';  --
fine if literal type matches column
SELECT * FROM Sales.Customers WHERE NationalID = 12345;          --
BAD if NationalID is actually nvarchar!
```

`YEAR(OrderDate) = 2026` forces the engine to evaluate `YEAR()` against *every row* before it can compare — the column being wrapped in a function makes the predicate **non-SARGable**, and the plan shows a scan instead of a seek even with a perfectly good index on `OrderDate` present. This is directly visible in the execution plan as a scan operator with a `Predicate` (post-filter) rather than a `Seek Predicate`. Implicit conversions (comparing mismatched types, especially `varchar` vs `nvarchar`, or numeric vs string) cause the same problem and are frequently *invisible* unless specifically looked for — SQL Server's plan XML/warnings do flag them, and it's a standard, high-value item to check when a seemingly well-indexed query still scans.

29.5 Parameter-Sensitive Behavior ("Parameter Sniffing")

A stored procedure's execution plan is typically compiled once and cached based on the **first** parameter value(s) it happened to be called with — if that first call was highly atypical (e.g. a customer with an unusually huge order history), the resulting cached plan can perform very poorly for subsequent, more typical calls (or vice versa) — a genuinely SQL-Server-specific operational concern known as "parameter sniffing." Mitigations include `OPTION (RECOMPILE)` (accepting recompilation cost for a plan tailored to each call's actual parameters), `OPTIMIZE FOR` hints, or (in modern SQL Server) Parameter-Sensitive Plan (PSP) optimization, which can automatically maintain multiple cached plan variants for different parameter value ranges.

29.6 Practical Optimization Methodology, Not Just "Add an Index"

Genuine query optimization methodology, in order: (1) confirm the query is logically correct and returns the minimum necessary data (Part 30); (2) verify statistics are current; (3) identify non-SARGable predicates and implicit conversions and rewrite them; (4) *then* evaluate whether an index change would help, informed by the actual plan's bottleneck, not a reflexive guess; (5) re-measure with `SET STATISTICS IO/TIME ON` after each change to confirm genuine improvement rather than assuming it.

---

30. Query Optimization

30.1 SARGability — Practical Rewrites

```
-- Non-SARGable                                  -- SARGable equivalent
WHERE YEAR(OrderDate) = 2026                     WHERE OrderDate >= '2026-
01-01' AND OrderDate < '2027-01-01'
WHERE ISNULL(Status, '') = 'Shipped'             WHERE Status = 'Shipped'
-- (redesign to avoid needing ISNULL here)
WHERE SUBSTRING(Email, 1, 4) = 'ada@'             WHERE Email LIKE 'ada@%'
WHERE Price * 1.08 > 100                         WHERE Price > 100 / 1.08
-- move the calculation OFF the column
```

The general rule: **never wrap an indexed column in a function or arithmetic expression** if the predicate needs to remain seekable; instead, transform the constant/comparison side of the expression to isolate the raw column.

30.2 Selecting Only Required Columns; Avoiding Unnecessary DISTINCT

```
-- WASTEFUL
SELECT * FROM Sales.Orders WHERE CustomerID = 101;

-- BETTER - only what's actually needed, potentially enabling a
covering index (Section 26.5)
SELECT OrderID, OrderDate, TotalAmount FROM Sales.Orders WHERE
CustomerID = 101;

-- DISTINCT masking an underlying join/data problem, rather than
fixing it
SELECT DISTINCT c.CustomerID, c.FullName FROM Sales.Customers c JOIN
Sales.Orders o ON c.CustomerID = o.CustomerID;
-- Often better addressed with EXISTS if the goal is genuinely
"customers WHO HAVE orders":
SELECT c.CustomerID, c.FullName FROM Sales.Customers c
WHERE EXISTS (SELECT 1 FROM Sales.Orders o WHERE o.CustomerID =
c.CustomerID);
```

`DISTINCT` used to paper over row duplication caused by a join (Section 11.2) rather than addressing the join itself both hides the real issue and forces an expensive deduplication pass (Section 18.2) that `EXISTS` avoids entirely.

30.3 Join Optimization, Statistics Maintenance, Parameterization

Ensuring join columns are indexed appropriately (Part 26), keeping statistics current (`UPDATE STATISTICS`, or relying on `AUTO_UPDATE_STATISTICS`), and using **parameterized queries** (both for the SQL-injection-prevention reasons covered in Part 36, and because parameterized queries allow the same cached execution plan to be reused across calls instead of triggering a fresh, costly compilation for every literal-value variation) are all foundational, broadly-applicable optimization practices.

30.4 Temp Table Strategies and Set-Based Operations vs Cursors

```
-- ROW-BY-ROW (cursor) approach - almost always slower for set-based
work
DECLARE @CustomerID INT;
DECLARE cur CURSOR FOR SELECT CustomerID FROM Sales.Customers;
OPEN cur;
FETCH NEXT FROM cur INTO @CustomerID;
WHILE @@FETCH_STATUS = 0
BEGIN
    UPDATE Sales.Customers SET Status = 'Reviewed' WHERE CustomerID =
@CustomerID;
    FETCH NEXT FROM cur INTO @CustomerID;
END
CLOSE cur; DEALLOCATE cur;

-- SET-BASED equivalent - dramatically faster, and simpler
UPDATE Sales.Customers SET Status = 'Reviewed';
```

SQL Server (like PostgreSQL and MySQL) is fundamentally optimized for **set-based** operations — processing many rows at once as a single relational operation — not row-by-row procedural iteration. A cursor forces the engine to process one row at a time, with per-row overhead, and is almost always dramatically slower than an equivalent set-based rewrite for anything that *can* be expressed set-based (which is the large majority of business logic, once the habit of thinking in sets rather than loops is developed). Cursors remain occasionally appropriate for genuinely sequential, stateful administrative tasks (iterating databases for a maintenance script) but should be treated as a last resort for ordinary business-logic queries.

30.5 Before/After Optimization Example

```
-- BEFORE: non-SARGable, SELECT *, cursor-based status update,
unindexed join
-- (illustrative combination of the anti-patterns above)

-- AFTER
CREATE NONCLUSTERED INDEX IX_Orders_CustomerID_Covering
ON Sales.Orders (CustomerID) INCLUDE (OrderID, OrderDate,
TotalAmount);

SELECT OrderID, OrderDate, TotalAmount
FROM Sales.Orders
WHERE CustomerID = 101 AND OrderDate >= '2026-01-01' AND OrderDate <
'2027-01-01';

UPDATE Sales.Customers SET Status = 'Reviewed' WHERE Region = 'North';
```

Measuring `SET STATISTICS IO` logical reads before and after such a rewrite is the concrete, quantifiable way to confirm and communicate the actual improvement achieved — "logical reads dropped from 4,521 to 8" is a far more convincing (and diagnosable) claim than "it feels faster."

31. SQL Server JSON

31.1 JSON Storage Concepts — No Native JSON Type

As noted in Section 4.4, SQL Server stores JSON as plain `nvarchar(max)`, manipulated entirely through JSON *functions* rather than a dedicated type with native operators (unlike PostgreSQL's `jsonb`). Validating that a column actually contains well-formed JSON is a deliberate, explicit step:

```
ALTER TABLE Sales.Orders ADD ItemsJson NVARCHAR(MAX)
    CONSTRAINT CK_Orders_ItemsJson_Valid CHECK (ISJSON(ItemsJson) =
1);
```

31.2 ISJSON, JSON_VALUE, JSON_QUERY

```
SELECT ISJSON(ItemsJson) AS IsValidJson FROM Sales.Orders;

-- JSON_VALUE extracts a SCALAR value (string/number/bool) from a JSON
path
SELECT JSON_VALUE(ItemsJson, '$.customer.name') AS CustomerName FROM
Sales.Orders;

-- JSON_QUERY extracts an OBJECT or ARRAY fragment (not a scalar)
SELECT JSON_QUERY(ItemsJson, '$.items') AS ItemsArray FROM
Sales.Orders;
```

`JSON_VALUE` and `JSON_QUERY` are complementary, not interchangeable: `JSON_VALUE` returns `NULL` (or errors, depending on settings) if the target path is an object/array rather than a scalar, and vice versa for `JSON_QUERY` — a common early mistake is using the wrong one for the target's actual shape.

31.3 JSON_MODIFY — Updating JSON In Place

```
UPDATE Sales.Orders
SET ItemsJson = JSON_MODIFY(ItemsJson, '$.customer.name', 'Ada
Lovelace')
WHERE OrderID = 1001;

-- Append to a JSON array
UPDATE Sales.Orders
SET ItemsJson = JSON_MODIFY(ItemsJson, 'append $.items',
JSON_QUERY('{"sku":"X1","qty":2}'))
WHERE OrderID = 1001;
```

31.4 OPENJSON — Converting JSON Into Relational Rows

```
DECLARE @json NVARCHAR(MAX) = '{"customer":
{"name":"Ada","id":101},"items":[{"sku":"A1","qty":2},
{"sku":"B2","qty":1}]}';

-- Shred the top-level object into key/value/type rows
SELECT * FROM OPENJSON(@json);
```

```
-- Shred a nested ARRAY with an explicit schema (WITH clause)
SELECT item.sku, item.qty
FROM OPENJSON(@json, '$.items')
WITH (sku NVARCHAR(20) '$.sku', qty INT '$.qty') AS item;
```

`OPENJSON` is the T-SQL mechanism for turning a JSON document into a proper relational rowset — directly comparable in purpose to PostgreSQL's `jsonb_to_recordset/jsonb_array_elements`, though the syntax (a table-valued function with an explicit `WITH` schema) is distinctly its own. This is commonly paired with `CROSS APPLY` (Section 16.3) to "explode" a JSON array column into one row per array element, joined back to its parent row.

31.5 FOR JSON — Returning Query Results as JSON

```
SELECT CustomerID, FullName, Email
FROM Sales.Customers
FOR JSON PATH, ROOT('customers');

SELECT OrderID, CustomerID,
    (SELECT ProductID, Quantity FROM Sales.OrderItems oi WHERE
oi.OrderID = o.OrderID FOR JSON PATH) AS Items
FROM Sales.Orders o
FOR JSON PATH;
```

`FOR JSON PATH` (or the simpler `FOR JSON AUTO`) converts an ordinary relational result set directly into a JSON document — the standard T-SQL mechanism for an API backend (Part 53) to return JSON straight from a query, without needing an intermediate application-layer serialization step for straightforward cases. PostgreSQL's equivalent is `row_to_json/json_agg`; MySQL provides `JSON_ARRAYAGG/JSON_OBJECTAGG` for a broadly similar purpose.

31.6 Indexing JSON Data — Computed Columns

```
ALTER TABLE Sales.Orders ADD CustomerName AS JSON_VALUE(ItemsJson,
'$.customer.name') PERSISTED;
CREATE INDEX IX_Orders_CustomerNameFromJson ON Sales.Orders
(CustomerName);
```

Because SQL Server's JSON columns are plain text, they cannot be indexed directly the way PostgreSQL's `jsonb` supports GIN indexes natively; the standard workaround is a **computed column** (`AS JSON_VALUE(...) PERSISTED`, physically storing the extracted value) with an ordinary index built on top of it — effective, but requiring one such computed column per JSON path that needs efficient querying, unlike `jsonb`'s more general-purpose native indexing.

31.7 Backend/API Example

```
-- An API endpoint's query: return a customer's full order history as
one JSON document
SELECT
    c.CustomerID, c.FullName,
    (SELECT o.OrderID, o.OrderDate, o.TotalAmount
     FROM Sales.Orders o WHERE o.CustomerID = c.CustomerID
     FOR JSON PATH) AS Orders
```

```
FROM Sales.Customers c
WHERE c.CustomerID = 101
FOR JSON PATH, WITHOUT_ARRAY_WRAPPER;
```

---

32. XML

32.1 The XML Data Type

```
CREATE TABLE Config.Settings (SettingID INT PRIMARY KEY, SettingXml
XML);

INSERT INTO Config.Settings VALUES (1,
'<settings><timeout>30</timeout><retries>3</retries></settings>');
```

The native `xml` data type stores well-formed XML with built-in validation, and — unlike SQL Server's JSON support — comes with dedicated storage optimizations and querying methods, reflecting XML's earlier (SQL Server 2005+) and more deeply integrated support in the engine relative to JSON's later, function-based approach.

32.2 XML Querying Methods

```
SELECT SettingXml.value('(/settings/timeout)[1]', 'INT') AS
TimeoutSeconds
FROM Config.Settings;

SELECT SettingXml.query('/settings/retries') AS RetriesNode
FROM Config.Settings;

UPDATE Config.Settings
SET SettingXml.modify('replace value of (/settings/timeout/text())[1]
with "60"')
WHERE SettingID = 1;
```

`.value()`, `.query()`, `.modify()`, `.exist()`, and `.nodes()` are XML data-type methods using XQuery/XPath expressions — a genuinely more mature and standardized query language than SQL Server's JSON path syntax, reflecting XML's head start in the engine.

32.3 FOR XML

```
SELECT CustomerID, FullName FROM Sales.Customers FOR XML AUTO;
SELECT CustomerID, FullName FROM Sales.Customers FOR XML
PATH('Customer'), ROOT('Customers');
```

The direct XML analogue of `FOR JSON` (Section 31.5) — converting a relational result set into an XML document.

32.4 Where XML Is Still Relevant in Enterprise Systems

Despite JSON's dominance in modern web APIs, XML remains heavily entrenched in enterprise integration contexts predating JSON's rise: SOAP-based web services, many legacy ERP/EDI (Electronic Data Interchange) systems, certain regulatory/compliance reporting formats (some

financial and healthcare standards are XML-based by mandate), and configuration formats in older enterprise software. A SQL Server professional working in banking, healthcare, or long-established enterprise environments should expect to encounter and need to query XML data considerably more often than in a greenfield, JSON-native startup context.

---

33. Data Cleaning With T-SQL

33.1 NULL Handling and Missing Values

```
SELECT CustomerID, ISNULL(Phone, 'Unknown') AS Phone,
COALESCE(MobilePhone, Phone, 'Unknown') AS BestContact
FROM Sales.Customers;

-- Identify the SCALE of missingness before deciding how to handle it
SELECT
    COUNT(*) AS TotalRows,
    COUNT(Phone) AS NonNullPhone,
    COUNT(*) - COUNT(Phone) AS MissingPhone
FROM Sales.Customers;
```

33.2 Duplicate Detection and Safe Removal

```
-- Detect duplicates by a business key
SELECT Email, COUNT(*) AS DuplicateCount
FROM Sales.Customers
GROUP BY Email
HAVING COUNT(*) > 1;

-- Safely remove duplicates, keeping the earliest-created row per
Email
WITH Ranked AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY Email ORDER BY CreatedAt
ASC) AS rn
    FROM Sales.Customers
)
DELETE FROM Ranked WHERE rn > 1;
```

Using `ROW_NUMBER()` inside a CTE, then deleting `WHERE rn > 1` directly against the CTE (T-SQL allows DML directly against a CTE referencing the base table) is the standard, professional pattern for deterministic duplicate removal — always run the equivalent `SELECT` first to review exactly which rows would be deleted.

33.3 Invalid Dates and Numeric Values

```
SELECT RawDateText, TRY_CAST(RawDateText AS DATE) AS ParsedDate
FROM Staging.RawImport
WHERE TRY_CAST(RawDateText AS DATE) IS NULL AND RawDateText IS NOT
NULL;   -- flag unparseable rows

SELECT RawAmountText, TRY_CONVERT(DECIMAL(12,2), RawAmountText) AS
```

```
ParsedAmount
FROM Staging.RawImport;
```

33.4 Text Normalization, Phone Cleaning, Email Validation Concepts

```
-- Normalize whitespace and casing
UPDATE Staging.RawImport SET FullName = UPPER(LTRIM(RTRIM(FullName)));

-- Strip non-digit characters from a phone number
UPDATE Staging.RawImport
SET CleanPhone = (
    SELECT STRING_AGG(value, '') FROM STRING_SPLIT(Phone, '') WHERE
value LIKE '[0-9]'
);
-- (In practice, a CLR function or a simple loop is often clearer than
the STRING_SPLIT trick above
--  for character-by-character filtering; T-SQL has no native regex
REPLACE.)

-- Basic email STRUCTURE validation (format only - see Part 39's
format-vs-verification distinction
-- from the Regular Expressions handbook for the same caveat,
applicable here too)
SELECT Email FROM Sales.Customers
WHERE Email NOT LIKE '_%@_%.__%' OR Email LIKE '%@%@%';
```

Note on regex in T-SQL: unlike PostgreSQL (`~`, `SIMILAR TO`) or MySQL (`REGEXP`), standard T-SQL has **no native regular-expression support** at all for pattern matching or replacement — `LIKE` with wildcards (Part 8) is the built-in alternative, considerably less expressive. Full regex capability in SQL Server requires either a CLR (Common Language Runtime) integration function, calling out to an external service, or handling that logic in the application layer (e.g. Python, per Part 49) instead.

33.5 Outlier Identification

```
WITH Stats AS (
    SELECT AVG(TotalAmount) AS AvgAmount, STDEV(TotalAmount) AS
StdAmount FROM Sales.Orders
)
SELECT o.OrderID, o.TotalAmount
FROM Sales.Orders o CROSS JOIN Stats s
WHERE ABS(o.TotalAmount - s.AvgAmount) > 3 * s.StdAmount;
```

33.6 A Complete Data-Cleaning Project

```
-- Stage 1: land raw data as-is (all text columns, no constraints) -
see Part 39
-- Stage 2: identify and quarantine bad rows
SELECT * INTO Staging.QuarantinedRows
FROM Staging.RawImport
WHERE TRY_CAST(OrderDateText AS DATE) IS NULL
   OR TRY_CONVERT(DECIMAL(12,2), AmountText) IS NULL
   OR CustomerEmail NOT LIKE '_%@_%.__%';
```

```sql
-- Stage 3: transform and load the clean remainder
INSERT INTO Sales.Orders (CustomerID, OrderDate, TotalAmount)
SELECT c.CustomerID, TRY_CAST(r.OrderDateText AS DATE),
TRY_CONVERT(DECIMAL(12,2), r.AmountText)
FROM Staging.RawImport r
JOIN Sales.Customers c ON c.Email = r.CustomerEmail
WHERE r.OrderDateText NOT IN (SELECT OrderDateText FROM
Staging.QuarantinedRows WHERE OrderDateText IS NOT NULL);

-- Stage 4: report on data quality
SELECT
    (SELECT COUNT(*) FROM Staging.RawImport) AS TotalRawRows,
    (SELECT COUNT(*) FROM Staging.QuarantinedRows) AS QuarantinedRows,
    (SELECT COUNT(*) FROM Sales.Orders) AS SuccessfullyLoaded;
```

---

34. Database Design and Normalization

34.1 Entities, Attributes, Relationships, Keys, ER Diagrams

These foundational relational-modeling concepts (identical across any RDBMS, including PostgreSQL and MySQL) are assumed already familiar; this part focuses on how they inform practical SQL Server table design decisions rather than re-deriving ER modeling from first principles.

34.2 Normal Forms — 1NF, 2NF, 3NF, BCNF

Form	Requirement
1NF	Every column holds a single, atomic value; no repeating groups or arrays in one column.
2NF	1NF, plus every non-key column depends on the **whole** primary key (relevant only for composite keys).
3NF	2NF, plus no non-key column depends on *another non-key column* (no "transitive" dependency).
BCNF	A stricter version of 3NF, addressing certain edge cases involving multiple overlapping candidate keys.

34.3 Normalization vs Denormalization

Normalization minimizes data redundancy and update anomalies by splitting data into well-structured related tables — the standard approach for OLTP systems (Section 34.4) where data integrity and efficient, small transactional writes matter most. **Denormalization** deliberately reintroduces redundancy (duplicating data across tables, or pre-aggregating values) to reduce the number of joins a *read*-heavy analytical query needs — the standard approach for

OLAP/data-warehouse design (Part 51), where read performance for large aggregate queries outweighs the write-side and storage costs of redundancy.

34.4 OLTP Design vs OLAP Design

Aspect	OLTP (Transactional)	OLAP (Analytical)
Primary workload	Many small, fast reads/writes	Fewer, large, complex aggregate reads
Schema shape	Normalized (3NF+)	Denormalized (star/snowflake schema, Part 51)
Typical index strategy	Many targeted nonclustered indexes for specific lookups	Fewer, wider indexes; heavy columnstore use (Part 27)
Example	An e-commerce checkout system	A sales data warehouse used for BI reporting

SQL Server is routinely used for **both** roles — often literally the same product, configured and modeled differently — a genuinely practical reason to understand this distinction deeply as a SQL Server professional working across both backend development (Part 53) and analytics/warehousing (Parts 47, 51) contexts.

35. SQL Server Security

35.1 Logins, Users, Roles, Schemas, Permissions

```
-- Server level
CREATE LOGIN app_service WITH PASSWORD = 'Str0ng!Passw0rd123';

-- Database level - map the login to a user within a specific database
USE Sales;
CREATE USER app_service FOR LOGIN app_service;

-- Roles group permissions for easier management
CREATE ROLE ReportReaders;
GRANT SELECT ON SCHEMA::Sales TO ReportReaders;
ALTER ROLE ReportReaders ADD MEMBER app_service;
```

35.2 GRANT, DENY, REVOKE

```
GRANT SELECT, INSERT ON Sales.Orders TO app_service;
DENY DELETE ON Sales.Orders TO app_service;     -- explicit denial
overrides any GRANT, even from role membership
REVOKE INSERT ON Sales.Orders FROM app_service;  -- removes a
previously granted permission (neutral, not a denial)
```

`DENY` is worth calling out specifically: it **always wins** over any `GRANT`, from any role or source — a genuinely important behavior for enforcing hard security boundaries that cannot be accidentally overridden by adding a user to some other permissive role later. `REVOKE` merely removes a specific grant/denial, returning to a neutral "no explicit permission" state, which is different from actively denying.

35.3 Server-Level vs Database-Level vs Object-Level Security, Least Privilege

Level	Example
Server-level	`CREATE LOGIN`, server roles (`sysadmin`, `dbcreator`) — controls what a login can do across the *entire instance*.
Database-level	Database roles (`db_datareader`, `db_datawriter`, custom roles) — controls access *within one specific database*.
Object-level	`GRANT SELECT ON Sales.Orders` — the finest-grained permission, on a specific table/view/procedure.

Least privilege — granting a login/user only the exact permissions its role genuinely requires, nothing more — is the foundational security principle underlying all of Part 35; an application's service account, for instance, should almost never hold `sysadmin` or `db_owner`, even though it would technically "work" with those broad permissions.

35.4 Row-Level Security

```
CREATE FUNCTION Security.fn_CustomerAccessPredicate(@RegionID INT)
RETURNS TABLE
```

```
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS AccessResult
WHERE @RegionID = CAST(SESSION_CONTEXT(N'UserRegionID') AS INT) OR
IS_MEMBER('db_owner') = 1;

CREATE SECURITY POLICY Security.CustomerFilter
ADD FILTER PREDICATE Security.fn_CustomerAccessPredicate(RegionID) ON
Sales.Customers;
```

Row-Level Security (**[SQL Server feature]**, introduced 2016) transparently filters (or blocks writes to) rows based on a predicate function evaluated per query — a salesperson querying `Sales.Customers` automatically sees only their own region's rows, with **no application code changes required at all**, since the filtering happens inside the engine itself for every query, regardless of which application or ad-hoc tool issues it. PostgreSQL has a very similar native `ROW LEVEL SECURITY` feature (`CREATE POLICY`); MySQL has no native equivalent, typically requiring views or application-level filtering instead.

35.5 Dynamic Data Masking

```
ALTER TABLE Banking.Customers
ALTER COLUMN SSN ADD MASKED WITH (FUNCTION = 'partial(0,"XXX-
XX-",4)');

ALTER TABLE Banking.Customers
ALTER COLUMN Email ADD MASKED WITH (FUNCTION = 'email()');
```

Dynamic Data Masking (**[SQL Server feature]**) obscures sensitive column values in query *results* for users without an explicit `UNMASK` permission, while leaving the actual stored data untouched — useful for letting support staff or reporting tools see "enough" of a value (like the last 4 digits of an SSN) without exposing the full sensitive value, without needing separate masked copies of the data maintained elsewhere.

35.6 Encryption Concepts

Feature	What It Protects Against
Transparent Data Encryption (TDE)	Encrypts the database's physical files at rest — protects against someone stealing the physical `.mdf/.bak` files; the server itself, and any authorized query, sees plain data transparently.
Always Encrypted	Encrypts specific column *values* client-side, before they ever reach the server in plaintext — even a DBA with full server access cannot see the plaintext value; decryption happens only in an authorized client application holding the encryption key.
Column-level (cell) encryption	Older, more manual T-SQL function-based (`ENCRYPTBYKEY`, etc.) column encryption,

Feature	What It Protects Against
	largely superseded in new development by Always Encrypted.

Banking/healthcare example: TDE is a standard, comparatively low-effort baseline (protecting backup files if physically stolen); Always Encrypted is appropriate for the most sensitive fields specifically (SSNs, full card numbers) where even privileged database administrators should never be able to view plaintext values — a materially stronger, "zero server-side trust" security posture increasingly required by financial and healthcare compliance regimes.

---

36. Dynamic SQL

36.1 sp_executesql and Parameterized Dynamic SQL

```
DECLARE @sql NVARCHAR(MAX) = N'SELECT * FROM Sales.Orders WHERE
CustomerID = @CustID AND Status = @Stat';
EXEC sp_executesql @sql,
    N'@CustID INT, @Stat NVARCHAR(20)',
    @CustID = 101, @Stat = 'Shipped';
```

`sp_executesql` executes a dynamically-built SQL string **with proper parameterization** — the parameter values are passed separately from the SQL text itself, exactly like a parameterized query in any application-level driver (Part 49), meaning the values can never be misinterpreted as SQL syntax.

36.2 SQL Injection — Unsafe vs Safe Dynamic SQL

```
-- UNSAFE - directly concatenates user input into the SQL string
DECLARE @Status NVARCHAR(50) = '''; DROP TABLE Sales.Orders; --';
DECLARE @sql NVARCHAR(MAX) = N'SELECT * FROM Sales.Orders WHERE Status
= ''' + @Status + N'''';
EXEC (@sql);   -- the malicious input breaks out of the intended
string literal entirely!

-- SAFE - the value is passed as a genuine parameter, never
concatenated into the SQL text
DECLARE @sql2 NVARCHAR(MAX) = N'SELECT * FROM Sales.Orders WHERE
Status = @Stat';
EXEC sp_executesql @sql2, N'@Stat NVARCHAR(50)', @Stat = '''; DROP
TABLE Sales.Orders; --';
-- Here, the malicious-looking string is treated as ONE LITERAL VALUE
to compare Status against,
-- entirely inert as SQL syntax - the query simply (and correctly)
matches zero rows.
```

This is the exact T-SQL equivalent of the SQL-injection defense pattern covered for Python in the Regular Expressions and Testing handbooks: **never build SQL by string-concatenating**

untrusted input; always pass untrusted values as genuine parameters, whether via `sp_executesql` in T-SQL or a parameterized query from an application driver (Part 49).

36.3 Dynamic Filtering, Reporting, and PIVOT

```
-- Dynamic filtering: build a WHERE clause only from the filters
actually supplied
CREATE PROCEDURE Sales.usp_SearchOrders
    @CustomerID INT = NULL, @Status NVARCHAR(20) = NULL
AS
BEGIN
    DECLARE @sql NVARCHAR(MAX) = N'SELECT * FROM Sales.Orders WHERE
1=1';
    IF @CustomerID IS NOT NULL SET @sql += N' AND CustomerID =
@CustID';
    IF @Status IS NOT NULL SET @sql += N' AND Status = @Stat';

    EXEC sp_executesql @sql,
        N'@CustID INT, @Stat NVARCHAR(20)',
        @CustID = @CustomerID, @Stat = @Status;
END;
```

Dynamic reporting and dynamic `PIVOT` (Section 17.3) follow the same fundamental pattern: build the SQL text programmatically to accommodate a structure not known until runtime (which filters were supplied; which columns should appear), but **always** route the actual data values through `sp_executesql`'s parameter mechanism, never string concatenation, regardless of how dynamic the surrounding query structure needs to be.

---

37. SQL Server Backup and Restore

37.1 Backup Types

Type	Captures
Full backup	The entire database at a point in time — the foundation every other backup type builds on.
Differential backup	Everything changed *since the last full backup* — smaller and faster than another full backup, but grows over time until the next full backup resets the baseline.
Transaction log backup	Every transaction recorded in the log since the last log backup — enables point-in-time recovery (Section 37.3); only possible under the Full (or Bulk-logged) recovery model (Section 3.5).

37.2 BACKUP DATABASE, BACKUP LOG, RESTORE DATABASE, RESTORE LOG

```
BACKUP DATABASE Sales TO DISK = 'D:\Backups\Sales_Full.bak';
BACKUP DATABASE Sales TO DISK = 'D:\Backups\Sales_Diff.bak' WITH
DIFFERENTIAL;
BACKUP LOG Sales TO DISK = 'D:\Backups\Sales_Log.trn';

RESTORE DATABASE Sales FROM DISK = 'D:\Backups\Sales_Full.bak' WITH
NORECOVERY;
RESTORE DATABASE Sales FROM DISK = 'D:\Backups\Sales_Diff.bak' WITH
NORECOVERY;
RESTORE LOG Sales FROM DISK = 'D:\Backups\Sales_Log.trn' WITH
RECOVERY;
```

`WITH NORECOVERY` leaves the database in a "restoring" state, ready to accept the *next* backup file in sequence (differential, then log); the final restore step in the chain uses `WITH RECOVERY` to bring the database fully online. Restoring the wrong order, or forgetting `NORECOVERY` on an intermediate step, is a common practical mistake worth rehearsing in a non-production environment before ever relying on it during a real incident.

37.3 Point-in-Time Recovery

```
RESTORE DATABASE Sales FROM DISK = 'D:\Backups\Sales_Full.bak' WITH
NORECOVERY;
RESTORE LOG Sales FROM DISK = 'D:\Backups\Sales_Log.trn'
    WITH RECOVERY, STOPAT = '2026-01-05T14:30:00';
```

`STOPAT` restores the transaction log only up to a specific moment — the standard recovery technique for "someone ran a bad `UPDATE` at 2:35pm, restore to just before it" scenarios, only possible when the Full recovery model (Section 3.5) has been in effect with regular log backups being taken.

37.4 RPO, RTO, and Disaster Recovery

Term	Meaning
RPO (Recovery Point Objective)	The maximum acceptable amount of *data loss*, measured in time — "we can tolerate losing up to 15 minutes of data" directly determines how frequently log backups must run.
RTO (Recovery Time Objective)	The maximum acceptable *downtime* before service is restored — directly informs whether a slower full-restore-from-backup strategy is adequate, or a High Availability solution (Part 45) with near-instant failover is required instead.

A backup/restore strategy should always be derived explicitly from a stated RPO/RTO, not chosen arbitrarily — these two numbers are a business decision (how much data loss and downtime is genuinely acceptable), which the technical backup schedule and architecture then serve.

38. SQL Server Agent

38.1 Jobs, Job Steps, Schedules, Alerts, Operators

SQL Server Agent is the built-in job scheduler running as a Windows service alongside the Database Engine — a **job** is a named unit of scheduled work made of one or more **job steps** (each a T-SQL script, SSIS package, PowerShell script, or other supported step type), triggered by a **schedule** (time-based) or in response to an **alert** (a specific error/performance condition), with **operators** (people/distribution lists) notified of success/failure via email or other configured methods.

```
EXEC msdb.dbo.sp_add_job @job_name = 'Nightly Sales ETL';

EXEC msdb.dbo.sp_add_jobstep
    @job_name = 'Nightly Sales ETL',
    @step_name = 'Run ETL Procedure',
    @subsystem = 'TSQL',
    @command = 'EXEC Sales.usp_RunNightlyETL;';

EXEC msdb.dbo.sp_add_schedule
    @schedule_name = 'Nightly at 2AM',
    @freq_type = 4, @freq_interval = 1, @active_start_time = 020000;

EXEC msdb.dbo.sp_attach_schedule
    @job_name = 'Nightly Sales ETL', @schedule_name = 'Nightly at
2AM';
```

SQL Server Agent has no direct built-in equivalent in either PostgreSQL (typically relying on the OS's own scheduler, `cron`, or the `pg_cron` extension) or MySQL (the `EVENT` scheduler, a comparatively lighter-weight built-in feature) — Agent's integration with `msdb` (Section 1.8), its rich job-step type system, and its native alerting are a genuinely more mature, SQL-Server-native automation platform.

38.2 Automating Data Imports, ETL, Backups, Reports, Maintenance

Common professional Agent job categories: nightly ETL/data-warehouse loading (Part 50), scheduled full/differential/log backups (Part 37) run on an automated cadence matching the stated RPO, index maintenance (rebuilding/reorganizing fragmented indexes on a regular schedule), automated report generation and distribution, and scheduled data-quality checks (re-running validation queries like Section 33's and alerting an operator if quality thresholds are violated).

39. Data Import and Export

39.1 Import/Export Wizard, BULK INSERT, OPENROWSET, bcp

```
-- BULK INSERT - load a flat file directly into a table from within T-SQL
BULK INSERT Staging.RawImport
FROM 'D:\Imports\sales_january.csv'
WITH (
    FIRSTROW = 2,                -- skip the header row
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '\n',
    CODEPAGE = '65001'            -- UTF-8
);

-- OPENROWSET - query a file's content directly, without a permanent staging table
SELECT * FROM OPENROWSET(
    BULK 'D:\Imports\sales_january.csv',
    FORMATFILE = 'D:\Imports\sales_format.xml'
) AS ImportedData;

# bcp - command-line bulk copy utility, useful for scripted/automated imports and exports
bcp Sales.Orders out D:\Exports\orders.csv -c -t, -S localhost -T
bcp Staging.RawImport in D:\Imports\new_data.csv -c -t, -S localhost -T
```

39.2 Common Data-Import Problems

Problem	Symptom / Fix
Encoding	Non-UTF-8 source files producing garbled special characters; specify `CODEPAGE` explicitly (`BULK INSERT`) or the correct code page in `bcp`.
Delimiters	A field's own content containing the delimiter character (an address with an embedded comma) breaking column alignment; requires proper quoting/escaping support in the source file and import configuration.
NULLs	Distinguishing a genuinely empty string from a true `NULL` — flat files often represent both identically, requiring explicit post-import cleanup (Part 33).
Dates	Ambiguous formats (`01/02/2026` — January 2nd or February 1st?) varying by source-system locale; always confirm the source format explicitly rather than

Problem	Symptom / Fix
	assuming.
Decimal formats	Comma-as-decimal-separator locales (European `1.234,56`) misparsed against a US-locale expectation (`1,234.56`).
Bad records	A single malformed row aborting an entire bulk load; `BULK INSERT`'s `MAXERRORS` and error-file options allow tolerating and logging a bounded number of bad rows instead.
Data-type mismatches	Source values that don't fit the target column's type; landing data in an all-`nvarchar` staging table first (Part 33's data-cleaning pattern) and validating/converting explicitly afterward avoids load-time failures entirely.

40. tempdb

40.1 What tempdb Is

Introduced in Section 1.8: `tempdb` is a system database, shared by every database and session on the instance, recreated fresh on every SQL Server restart, used as scratch space for a wide range of internal operations — not merely explicit `#temp` tables.

40.2 What Uses tempdb

Use	Example
Temporary tables/table variables	`#Table`, `@TableVar` (Part 15)
Sorting	`ORDER BY` operations exceeding available memory grant (Section 28.5)
Hashing	Hash joins and hash aggregates needing a working area
Version store	Row versioning for Snapshot isolation/RCSI (Section 24.4)
Intermediate operations	Query plan operators like spools, and some index-rebuild operations

Because so many unrelated internal operations all compete for the same shared `tempdb`, it is frequently one of the most contended resources on a busy SQL Server instance — **tempdb contention** manifests as blocking specifically on `tempdb`'s internal system pages (visible via `PAGELATCH` waits in monitoring DMVs), a genuinely SQL-Server-specific operational concern with no precise PostgreSQL/MySQL parallel, since neither uses one single shared scratch database this centrally for such a broad range of engine-internal purposes.

40.3 Monitoring and Configuration Concepts

Professional guidance for busy instances: configure **multiple tempdb data files** of equal size (commonly one per CPU core, up to a reasonable cap) specifically to reduce internal allocation-page contention, monitor tempdb space usage and growth (an out-of-space `tempdb` can halt query processing instance-wide, affecting *every* database, not just one), and be alert to any single long-running transaction under Snapshot/RCSI isolation, since it can prevent the version store from being cleaned up, causing `tempdb` to grow unexpectedly large.

---

41. SQL Server System Catalog

41.1 Inspecting Metadata

```
SELECT name, object_id, schema_id, create_date FROM sys.tables WHERE
schema_id = SCHEMA_ID('Sales');

SELECT c.name AS ColumnName, t.name AS DataType, c.max_length,
c.is_nullable
FROM sys.columns c
JOIN sys.types t ON c.user_type_id = t.user_type_id
WHERE c.object_id = OBJECT_ID('Sales.Orders');

SELECT i.name AS IndexName, i.type_desc, c.name AS ColumnName,
ic.key_ordinal
FROM sys.indexes i
JOIN sys.index_columns ic ON i.object_id = ic.object_id AND i.index_id
= ic.index_id
JOIN sys.columns c ON ic.object_id = c.object_id AND ic.column_id =
c.column_id
WHERE i.object_id = OBJECT_ID('Sales.Orders')
ORDER BY i.name, ic.key_ordinal;

SELECT name, type_desc FROM sys.objects WHERE type IN ('U', 'V', 'P',
'FN', 'TR');  -- tables, views, procs, functions, triggers

SELECT name, type_desc, is_disabled FROM sys.database_principals WHERE
type IN ('S', 'U');  -- SQL/Windows users

SELECT definition FROM sys.sql_modules WHERE object_id =
OBJECT_ID('Sales.usp_CreateCustomer');
```

`sys.*` catalog views are SQL Server's equivalent of PostgreSQL's `pg_catalog`/`information_schema` and MySQL's `information_schema` — the standard, portable-in-concept-if-not-in-syntax way to programmatically inspect database metadata rather than relying on GUI tools alone.

41.2 Practical Auditing and Documentation Queries

```
-- Every table and its row count, for a quick database inventory
SELECT s.name AS SchemaName, t.name AS TableName, p.rows AS RowCount
```

```
FROM sys.tables t
JOIN sys.schemas s ON t.schema_id = s.schema_id
JOIN sys.partitions p ON t.object_id = p.object_id AND p.index_id IN
(0, 1)
ORDER BY p.rows DESC;

-- Every foreign key relationship, for documenting the schema's
structure
SELECT
    fk.name AS ForeignKeyName,
    OBJECT_NAME(fk.parent_object_id) AS ChildTable,
    OBJECT_NAME(fk.referenced_object_id) AS ParentTable
FROM sys.foreign_keys fk;

-- Every column, across every table, whose name suggests it might hold
sensitive data
SELECT OBJECT_NAME(object_id) AS TableName, name AS ColumnName
FROM sys.columns
WHERE name LIKE '%ssn%' OR name LIKE '%password%' OR name LIKE
'%creditcard%';
```

These system-catalog queries are the foundation of automated database documentation generation, security auditing (finding tables that should have Row-Level Security or masking applied, Part 35), and impact-analysis before a schema change (which tables/procedures actually reference a column being considered for removal).

42. Temporal Tables

42.1 System-Versioned Temporal Tables

```
-- [SQL Server / T-SQL feature, SQL Server 2016+]
CREATE TABLE HR.Employees (
    EmployeeID INT PRIMARY KEY,
    FullName NVARCHAR(100),
    Salary DECIMAL(10,2),
    ValidFrom DATETIME2 GENERATED ALWAYS AS ROW START,
    ValidTo DATETIME2 GENERATED ALWAYS AS ROW END,
    PERIOD FOR SYSTEM_TIME (ValidFrom, ValidTo)
)
WITH (SYSTEM_VERSIONING = ON (HISTORY_TABLE = HR.EmployeesHistory));
```

A temporal table automatically maintains a complete history of every change to every row — SQL Server transparently creates and maintains a paired **history table** (`HR.EmployeesHistory`), moving the "old" version of any updated or deleted row there automatically, with **zero application code changes required**. This is a genuinely distinctive SQL Server feature; PostgreSQL and MySQL have no equivalent *built into the core engine* (achievable only via manual triggers or third-party extensions), making native temporal tables one of SQL Server's most valuable built-in advantages for audit- and compliance-heavy applications.

42.2 Time-Based Querying

```
-- Current data (identical to querying a normal table)
SELECT * FROM HR.Employees WHERE EmployeeID = 42;

-- The row's value as of a specific point in time
SELECT * FROM HR.Employees FOR SYSTEM_TIME AS OF '2025-06-01' WHERE
EmployeeID = 42;

-- Every historical version within a date range
SELECT * FROM HR.Employees FOR SYSTEM_TIME BETWEEN '2025-01-01' AND
'2025-12-31' WHERE EmployeeID = 42;

-- The complete change history for a row
SELECT * FROM HR.Employees FOR SYSTEM_TIME ALL WHERE EmployeeID = 42
ORDER BY ValidFrom;
```

42.3 Auditing, Financial Applications, and Compliance

Temporal tables are directly applicable wherever regulatory or audit requirements demand answering "what did this record look like on this specific past date" — salary history for compliance audits, account-balance history for financial reconciliation, or any record whose historical state must be reconstructable on demand without custom audit-trigger logic (Section 22.3) maintained by hand.

42.4 A Practical Temporal-Table Project

```
-- Track salary changes over time and calculate the total number of
raises per employee
CREATE TABLE HR.Salaries (
    EmployeeID INT PRIMARY KEY,
    Salary DECIMAL(10,2),
    ValidFrom DATETIME2 GENERATED ALWAYS AS ROW START,
    ValidTo DATETIME2 GENERATED ALWAYS AS ROW END,
    PERIOD FOR SYSTEM_TIME (ValidFrom, ValidTo)
)
WITH (SYSTEM_VERSIONING = ON (HISTORY_TABLE = HR.SalariesHistory));

UPDATE HR.Salaries SET Salary = 85000 WHERE EmployeeID = 42;  -- old
value automatically archived

SELECT EmployeeID, COUNT(*) AS RaiseCount
FROM HR.SalariesHistory
GROUP BY EmployeeID;

SELECT EmployeeID, Salary, ValidFrom, ValidTo
FROM HR.Salaries FOR SYSTEM_TIME ALL
WHERE EmployeeID = 42
ORDER BY ValidFrom;
```

---

43. Change Data Capture and Change Tracking

43.1 Change Data Capture (CDC)

```
EXEC sys.sp_cdc_enable_db;
EXEC sys.sp_cdc_enable_table
    @source_schema = 'Sales', @source_name = 'Orders', @role_name =
NULL;

SELECT * FROM cdc.fn_cdc_get_all_changes_Sales_Orders(
    sys.fn_cdc_get_min_lsn('Sales_Orders'), sys.fn_cdc_get_max_lsn(),
'all'
);
```

CDC captures the **full detail** of every INSERT/UPDATE/DELETE against a tracked table — including old and new column values — into dedicated change tables, by reading the transaction log asynchronously (minimal impact on the source table's transactional workload). This is purpose-built for feeding **ETL pipelines** (Part 50): a downstream data warehouse load process can pull only what changed since its last run, rather than re-scanning the entire source table every time.

43.2 Change Tracking

```
ALTER DATABASE Sales SET CHANGE_TRACKING = ON (CHANGE_RETENTION = 7
DAYS, AUTO_CLEANUP = ON);
ALTER TABLE Sales.Orders ENABLE CHANGE_TRACKING;
```

```sql
SELECT * FROM CHANGETABLE(CHANGES Sales.Orders, 0) AS CT;
```

Change Tracking is a lighter-weight alternative to CDC: it records **that** a row changed (and its primary key) but not the actual old/new column values — sufficient for synchronization scenarios (like "which rows need to be re-fetched by a downstream cache or search index") that don't need full change history, at lower storage and overhead cost than full CDC.

43.3 CDC vs Change Tracking — Choosing

Need	Choose
Full audit trail of exactly what changed, old and new values	CDC
Just need to know *which* rows changed, to re-sync/re-fetch them	Change Tracking
Feeding an incremental ETL/data-warehouse load (Part 50)	CDC (needs the actual changed values)
Keeping a search index or cache in sync with a source table	Change Tracking (usually sufficient, lighter weight)

Neither PostgreSQL nor MySQL bundles a directly equivalent first-party feature at this level of built-in integration (PostgreSQL's logical replication and various CDC tools like Debezium serve an analogous purpose but as separate components, not a single built-in engine feature pair like SQL Server's).

---

44. Partitioning

44.1 Partition Functions and Schemes

```sql
CREATE PARTITION FUNCTION PF_OrderYear (DATE)
AS RANGE RIGHT FOR VALUES ('2024-01-01', '2025-01-01', '2026-01-01');

CREATE PARTITION SCHEME PS_OrderYear
AS PARTITION PF_OrderYear ALL TO ([PRIMARY]);

CREATE TABLE Sales.OrdersPartitioned (
    OrderID INT NOT NULL,
    OrderDate DATE NOT NULL,
    TotalAmount DECIMAL(12,2)
) ON PS_OrderYear (OrderDate);
```

A **partition function** defines the logical boundaries (here, by year) dividing a table's rows; a **partition scheme** maps each resulting partition to a physical filegroup (Section 3.4) — together, they let one logical table be physically stored (and, importantly, *managed*) as several separate underlying partitions, transparent to ordinary querying.

44.2 Range Partitioning and Partition Switching

```
-- Partition switching - moves an ENTIRE partition's data in/out
almost instantly (metadata-only operation)
ALTER TABLE Sales.OrdersArchive_2023 SWITCH TO Sales.OrdersPartitioned
PARTITION 1;
```

Partition switching is the standout capability: moving a whole partition's worth of data into or out of a partitioned table is a near-instantaneous, purely metadata-level operation (no actual data is physically copied or moved), regardless of how many millions of rows the partition contains — the standard technique for efficiently archiving old data out of an active table, or rapidly loading a new bulk data batch in, without the locking/logging cost of an equivalent row-by-row `DELETE`/`INSERT`.

44.3 How Partitioning Helps Large Enterprise Datasets

Partitioning improves **manageability** (index maintenance, backups, and archival can target individual partitions rather than an entire massive table), **query performance** for queries that filter on the partitioning column (**partition elimination** lets the engine skip scanning irrelevant partitions entirely), and enables the near-instant archival pattern above — all without changing how the table is queried from the application's perspective. PostgreSQL's native declarative partitioning (`PARTITION BY RANGE`) and MySQL's `PARTITION BY RANGE` serve a broadly similar purpose, though SQL Server's partition-switching mechanism and its tight integration with filegroups reflect a somewhat more mature, enterprise-oriented feature set historically.

---

45. SQL Server High Availability

45.1 High Availability vs Backup/Disaster Recovery — A Critical Distinction

High Availability (HA) minimizes *downtime* by maintaining one or more synchronized standby copies of a database ready for near-instant failover if the primary fails. **Backup and Disaster Recovery (Part 37)** protects against *data loss* and provides a recovery path after a catastrophic failure (including scenarios HA itself cannot address, like widespread corruption replicated to every HA copy, or an entire data center being destroyed). **These are complementary, not substitutes for each other** — a robust production architecture requires both: HA for routine failures (a single server crashing) and backups for catastrophic or logical failures (corruption, accidental mass deletion, a data-center-level disaster) that HA alone cannot protect against.

45.2 Always On Availability Groups

Always On Availability Groups (**[SQL Server feature]**, 2012+) maintain one or more synchronized secondary replicas of a set of databases, automatically (or manually) failing over to a secondary if the primary becomes unavailable — replicas can also serve **read-only** query workloads (offloading reporting queries from the primary), a commonly-used side benefit beyond pure HA.

45.3 Failover, Log Shipping, Database Mirroring

Mechanism	Summary
Always On Availability Groups	Modern, flexible, multi-replica HA/DR solution; the current recommended approach.
Log Shipping	Periodically ships and restores transaction log backups to one or more secondary servers — simple, reliable, but with higher latency (minutes, not seconds) than Availability Groups.
Database Mirroring	An older (pre-2012) single-secondary HA feature, now deprecated in favor of Availability Groups; understood today mainly for historical/legacy-system context.

45.4 Replication's Role

Replication (Part 46) is not primarily an HA mechanism at all — it is oriented toward *distributing* data to other locations/systems for different purposes (reporting offload, integration with other systems), though it is sometimes loosely associated with availability due to maintaining additional data copies elsewhere.

---

46. Replication

46.1 Publisher, Distributor, Subscriber

Role	Function
Publisher	The source database whose changes are being replicated out.
Distributor	Coordinates the movement of changes from publisher to subscriber(s); can be the same server as the publisher or a dedicated separate server.
Subscriber	The destination database receiving replicated changes.

46.2 Transactional, Snapshot, and Merge Replication

Type	Behavior	Typical Use
Transactional replication	Near-real-time, continuous propagation of individual transactions from publisher to subscriber(s).	Feeding a reporting server or another system with low-latency, up-to-date copies of operational data.
Snapshot replication	Periodically copies the *entire* current dataset, rather than	Smaller, infrequently-changing datasets, or as the

Type	Behavior	Typical Use
	incremental changes.	initial seeding step before transactional/merge replication begins.
Merge replication	Allows changes at *both* publisher and subscriber(s), with conflict-detection/resolution logic reconciling divergent changes.	Distributed/offline scenarios where subscribers (e.g. field sales laptops) must be able to make local changes while disconnected, later synchronized bidirectionally.

46.3 Where Replication Fits Into Enterprise Data Architectures

Replication is most often used to distribute operational data to a **reporting server** (isolating heavy analytical query load away from the transactional primary, a distinct concern from — and often complementary to — Always On's read-replica capability), to feed data into **separate downstream systems** (a data warehouse's staging layer, Part 50), or to support **distributed/branch-office** scenarios via merge replication. In modern architectures, CDC (Part 43) combined with a dedicated ETL/ELT pipeline has increasingly displaced transactional replication for pure data-warehouse-feeding use cases, while replication remains relevant for genuine multi-master or distributed-write scenarios.

47. SQL Server for Data Analytics

47.1 Revenue, Profit, Customers, Products, Regions, Employee Performance

```
-- Revenue and profit by product category
SELECT p.ProductCategory,
    SUM(oi.Quantity * oi.UnitPrice) AS Revenue,
    SUM(oi.Quantity * (oi.UnitPrice - p.UnitCost)) AS Profit
FROM Sales.OrderItems oi
JOIN Sales.Products p ON oi.ProductID = p.ProductID
GROUP BY p.ProductCategory
ORDER BY Revenue DESC;

-- Revenue by region
SELECT c.Region, SUM(o.TotalAmount) AS Revenue
FROM Sales.Orders o JOIN Sales.Customers c ON o.CustomerID =
c.CustomerID
GROUP BY c.Region ORDER BY Revenue DESC;
```

47.2 Customer Retention, Churn, Cohort Analysis

```
-- Cohort analysis: group customers by signup month, track activity in
subsequent months
WITH Cohorts AS (
    SELECT CustomerID, DATEFROMPARTS(YEAR(MIN(OrderDate)),
MONTH(MIN(OrderDate)), 1) AS CohortMonth
    FROM Sales.Orders GROUP BY CustomerID
),
ActivityByMonth AS (
    SELECT o.CustomerID, DATEFROMPARTS(YEAR(o.OrderDate),
MONTH(o.OrderDate), 1) AS ActivityMonth
    FROM Sales.Orders o
)
SELECT
    c.CohortMonth,
    DATEDIFF(MONTH, c.CohortMonth, a.ActivityMonth) AS
MonthsSinceSignup,
    COUNT(DISTINCT a.CustomerID) AS ActiveCustomers
FROM Cohorts c JOIN ActivityByMonth a ON c.CustomerID = a.CustomerID
GROUP BY c.CohortMonth, DATEDIFF(MONTH, c.CohortMonth,
a.ActivityMonth)
ORDER BY c.CohortMonth, MonthsSinceSignup;

-- Simple churn flag: no order in the trailing 90 days
SELECT CustomerID,
    CASE WHEN MAX(OrderDate) < DATEADD(DAY, -90, SYSDATETIME()) THEN 1
ELSE 0 END AS IsChurned
FROM Sales.Orders GROUP BY CustomerID;
```

47.3 Funnel Analysis and Market Basket Analysis Concepts

```
-- Funnel: count of users reaching each stage
SELECT
```

```
    COUNT(DISTINCT CASE WHEN EventType = 'ViewedProduct' THEN UserID
END) AS Viewed,
    COUNT(DISTINCT CASE WHEN EventType = 'AddedToCart' THEN UserID
END) AS AddedToCart,
    COUNT(DISTINCT CASE WHEN EventType = 'Purchased' THEN UserID END)
AS Purchased
FROM Analytics.Events;

-- Market basket: which product pairs are frequently bought together
SELECT a.ProductID AS ProductA, b.ProductID AS ProductB, COUNT(*) AS
TimesBoughtTogether
FROM Sales.OrderItems a
JOIN Sales.OrderItems b ON a.OrderID = b.OrderID AND a.ProductID <
b.ProductID
GROUP BY a.ProductID, b.ProductID
ORDER BY TimesBoughtTogether DESC;
```

47.4 Growth, Rolling Averages, Pareto Analysis, Customer Lifetime Value

```
-- Pareto analysis: what % of customers generate 80% of revenue
WITH CustomerRevenue AS (
    SELECT CustomerID, SUM(TotalAmount) AS Revenue FROM Sales.Orders
GROUP BY CustomerID
),
Ranked AS (
    SELECT *, SUM(Revenue) OVER (ORDER BY Revenue DESC ROWS UNBOUNDED
PRECEDING) AS RunningRevenue,
           SUM(Revenue) OVER () AS TotalRevenue
    FROM CustomerRevenue
)
SELECT CustomerID, Revenue, RunningRevenue * 100.0 / TotalRevenue AS
CumulativePercent
FROM Ranked ORDER BY Revenue DESC;

-- Customer Lifetime Value (simple historical CLV)
SELECT CustomerID, SUM(TotalAmount) AS HistoricalCLV, COUNT(*) AS
OrderCount,
    AVG(TotalAmount) AS AvgOrderValue
FROM Sales.Orders GROUP BY CustomerID;
```

Month-over-month and year-over-year growth (Section 14.5) and rolling averages (Section 14.4) are already covered fully in Part 14's window-function examples, directly applicable here as core Data Analytics deliverables.

---

48. SQL Server for Data Science

48.1 Dataset Preparation, Aggregation, Sampling

```
-- Feature aggregation at the customer level
SELECT CustomerID,
```

```
    COUNT(*) AS OrderCount,
    AVG(TotalAmount) AS AvgOrderValue,
    MAX(OrderDate) AS LastOrderDate,
    DATEDIFF(DAY, MIN(OrderDate), MAX(OrderDate)) AS
CustomerTenureDays
FROM Sales.Orders GROUP BY CustomerID;

-- Random sampling for exploratory analysis
SELECT TOP 1000 * FROM Sales.Orders ORDER BY NEWID();   -- [T-SQL]
NEWID() per row -> effectively random order
SELECT * FROM Sales.Orders TABLESAMPLE (10 PERCENT);      --
[T-SQL/SQL Server feature] - approximate, page-based sampling
```

TABLESAMPLE samples at the **physical page level**, not truly per-row — fast even on huge tables, but the resulting sample can be less statistically uniform than ORDER BY NEWID() (which is truly per-row random but requires a full sort, expensive on large tables). Choose based on whether speed or sampling rigor matters more for the specific analysis.

48.2 Time-Based and Customer-Level Feature Engineering

```
SELECT CustomerID,
    SUM(CASE WHEN OrderDate >= DATEADD(DAY, -30, SYSDATETIME()) THEN
TotalAmount ELSE 0 END) AS Spend_Last30d,
    SUM(CASE WHEN OrderDate >= DATEADD(DAY, -90, SYSDATETIME()) THEN
TotalAmount ELSE 0 END) AS Spend_Last90d,
    DATEDIFF(DAY, MAX(OrderDate), SYSDATETIME()) AS DaysSinceLastOrder
FROM Sales.Orders
GROUP BY CustomerID;
```

48.3 Target Construction

```
-- Building a binary churn LABEL for supervised learning: churned = no
order in the next 90 days
SELECT CustomerID,
    CASE WHEN NOT EXISTS (
        SELECT 1 FROM Sales.Orders o2
        WHERE o2.CustomerID = o1.CustomerID
          AND o2.OrderDate > o1.MaxOrderDate
          AND o2.OrderDate <= DATEADD(DAY, 90, o1.MaxOrderDate)
    ) THEN 1 ELSE 0 END AS Churned
FROM (SELECT CustomerID, MAX(OrderDate) AS MaxOrderDate FROM
Sales.Orders GROUP BY CustomerID) o1;
```

48.4 How SQL and Python Work Together

SQL Server excels at exactly the operations shown above — large-scale aggregation, filtering, and joining directly where the data already lives, minimizing data movement — while Python (Part 49) excels at everything *beyond* relational transformation: statistical modeling, machine learning, and visualization. The professional division of labor: push as much feature engineering and aggregation as reasonably possible **into SQL**, then pull the resulting, already-reduced dataset into Python/pandas for modeling — rather than pulling raw, row-level data

into Python and performing aggregation there, which is typically far slower and more memory-intensive for anything beyond small datasets.

---

49. Python + SQL Server

49.1 pyodbc — Direct Connection and Execution

```
import pyodbc

conn = pyodbc.connect(
    'DRIVER={ODBC Driver 18 for SQL Server};'
    'SERVER=localhost;DATABASE=Sales;'
    'UID=app_user;PWD=YourPassword;'
    'Encrypt=yes;TrustServerCertificate=yes;'
)

cursor = conn.cursor()
cursor.execute("SELECT CustomerID, TotalAmount FROM Sales.Orders WHERE
Status = ?", ('Shipped',))
for row in cursor.fetchall():
    print(row.CustomerID, row.TotalAmount)

conn.close()
```

The `?` placeholder followed by a separate parameters tuple is `pyodbc`'s parameterized-query syntax — directly analogous to the `sp_executesql` parameterization from Part 36, and essential for the exact same SQL-injection-prevention reason (Section 36.2) whenever a query includes any value not fully controlled by the application itself.

49.2 SQLAlchemy

```
from sqlalchemy import create_engine, text

engine = create_engine(
    'mssql+pyodbc://app_user:YourPassword@localhost/Sales'
    '?driver=ODBC+Driver+18+for+SQL+Server&TrustServerCertificate=yes'
)

with engine.connect() as conn:
    result = conn.execute(text("SELECT CustomerID, TotalAmount FROM
Sales.Orders WHERE Status = :status"),
                          {'status': 'Shipped'})
    for row in result:
        print(row.CustomerID, row.TotalAmount)
```

SQLAlchemy provides a database-agnostic abstraction layer over `pyodbc` (and other drivers) — the same application code can, with configuration changes rather than rewritten queries, target PostgreSQL, MySQL, or SQL Server, directly useful given this handbook's cross-database orientation (Parts 55-56).

49.3 pandas — Reading SQL Data Into a DataFrame

```
import pandas as pd
from sqlalchemy import create_engine

engine =
create_engine('mssql+pyodbc://app_user:YourPassword@localhost/Sales?
driver=ODBC+Driver+18+for+SQL+Server')

df = pd.read_sql(
    "SELECT CustomerID, OrderDate, TotalAmount FROM Sales.Orders WHERE
OrderDate >= '2026-01-01'",
    engine
)
print(df.head())
print(df.groupby('CustomerID')['TotalAmount'].sum())
```

49.4 Writing pandas DataFrames Back to SQL Server

```
cleaned_df.to_sql('CleanedOrders', engine, schema='Staging',
if_exists='replace', index=False)
```

49.5 Transactions, Connection Management, Connection Pooling, Error Handling

```
from sqlalchemy import create_engine, text
from sqlalchemy.exc import SQLAlchemyError

engine = create_engine(
    'mssql+pyodbc://app_user:YourPassword@localhost/Sales?
driver=ODBC+Driver+18+for+SQL+Server',
    pool_size=5, max_overflow=10, pool_pre_ping=True
)

try:
    with engine.begin() as conn:   # engine.begin() commits
automatically on success, rolls back on exception
        conn.execute(text("UPDATE Banking.Accounts SET Balance =
Balance - :amt WHERE AccountID = :id"),
                     {'amt': 500, 'id': 1})
        conn.execute(text("UPDATE Banking.Accounts SET Balance =
Balance + :amt WHERE AccountID = :id"),
                     {'amt': 500, 'id': 2})
except SQLAlchemyError as e:
    print(f"Transaction failed and was rolled back: {e}")
```

Concept	Explanation
`pool_size` / `max_overflow`	Connection pooling — reusing a bounded set of already-open connections rather than opening/closing a new physical connection per query, avoiding significant per-connection overhead under load.
`pool_pre_ping=True`	Tests each pooled connection's liveness

Concept	Explanation
	before reuse, gracefully handling connections SQL Server or a network intermediary silently closed.
`engine.begin()`	A context manager that wraps the block in an explicit transaction, auto-committing on success and auto-rolling-back on any exception — the Python-side analogue of T-SQL's `TRY/CATCH` + `COMMIT/ROLLBACK` pattern (Section 25.1).

50. SQL Server for Data Engineering

50.1 ETL vs ELT

ETL (Extract, Transform, Load) transforms data *before* loading it into the target warehouse — transformation logic typically lives in a separate tool/service. **ELT** (Extract, Load, Transform) loads raw data into the target first, then transforms it *using the target database's own processing power* (T-SQL, in SQL Server's case) — increasingly the more common modern pattern, leveraging SQL Server's own set-based processing strength (Part 30) for the transformation step itself rather than an external engine.

50.2 Incremental Loading and Slowly Changing Dimensions

```
-- Incremental load using a high-water-mark pattern
DECLARE @LastLoadTime DATETIME2 = (SELECT MAX(LoadedThroughDate) FROM
ETL.LoadHistory);

INSERT INTO Warehouse.FactOrders
SELECT * FROM Sales.Orders WHERE ModifiedDate > @LastLoadTime;

INSERT INTO ETL.LoadHistory (LoadedThroughDate) VALUES
(SYSDATETIME());

-- Slowly Changing Dimension Type 2: preserve full history of a
changing dimension attribute
UPDATE Warehouse.DimCustomer
SET ValidTo = SYSDATETIME(), IsCurrent = 0
WHERE CustomerID = 101 AND IsCurrent = 1 AND Region <> 'West';  -- the
customer's region changed

INSERT INTO Warehouse.DimCustomer (CustomerID, Region, ValidFrom,
ValidTo, IsCurrent)
VALUES (101, 'West', SYSDATETIME(), '9999-12-31', 1);
```

An SCD Type 2 pattern preserves every historical version of a dimension row (closely related in spirit to Part 42's temporal tables, though SCDs are a data-warehouse *modeling* pattern implemented with ordinary tables, while temporal tables are an *engine feature* automating a

similar outcome) — essential for accurate historical reporting ("what region was this customer in when this order was placed", not just "what region are they in now").

50.3 A Complete ETL Workflow Using SQL Server

```
-- EXTRACT: land raw data into staging (Part 39)
BULK INSERT Staging.RawOrders FROM 'D:\Imports\orders.csv' WITH
(FIRSTROW = 2, FIELDTERMINATOR = ',');

-- TRANSFORM: clean and validate (Part 33)
INSERT INTO Staging.CleanOrders
SELECT CustomerID, TRY_CAST(OrderDateText AS DATE) AS OrderDate,
TRY_CONVERT(DECIMAL(12,2), AmountText) AS TotalAmount
FROM Staging.RawOrders
WHERE TRY_CAST(OrderDateText AS DATE) IS NOT NULL AND
TRY_CONVERT(DECIMAL(12,2), AmountText) IS NOT NULL;

-- LOAD: into the warehouse fact table (Part 51), resolving dimension
keys
INSERT INTO Warehouse.FactOrders (CustomerKey, DateKey, Amount)
SELECT dc.CustomerKey, dd.DateKey, co.TotalAmount
FROM Staging.CleanOrders co
JOIN Warehouse.DimCustomer dc ON co.CustomerID = dc.CustomerID AND
dc.IsCurrent = 1
JOIN Warehouse.DimDate dd ON co.OrderDate = dd.FullDate;
```

---

51. SQL Server Data Warehousing

51.1 Star Schema — Fact and Dimension Tables

```
CREATE TABLE Warehouse.DimCustomer (
    CustomerKey INT IDENTITY PRIMARY KEY,   -- surrogate key
    CustomerID INT,                          -- natural/business key
from the source system
    FullName NVARCHAR(100), Region NVARCHAR(50),
    ValidFrom DATE, ValidTo DATE, IsCurrent BIT
);

CREATE TABLE Warehouse.DimDate (
    DateKey INT PRIMARY KEY, FullDate DATE, Year INT, Quarter INT,
Month INT, DayName NVARCHAR(10)
);

CREATE TABLE Warehouse.FactOrders (
    OrderKey INT IDENTITY PRIMARY KEY,
    CustomerKey INT REFERENCES Warehouse.DimCustomer(CustomerKey),
    DateKey INT REFERENCES Warehouse.DimDate(DateKey),
    Amount DECIMAL(12,2)
);
```

A **star schema** places a central **fact table** (numeric measures — `Amount`) surrounded by **dimension tables** (descriptive context — customer, date, product), joined via surrogate keys — deliberately denormalized (Section 34.3) compared to an OLTP schema, optimized for the large aggregate `GROUP BY` queries typical of BI reporting rather than for minimizing storage redundancy.

51.2 Snowflake Schema

A **snowflake schema** further normalizes dimension tables (e.g. splitting `DimCustomer` into `DimCustomer` + a separate `DimRegion`) — reduces some redundancy at the cost of additional joins for queries, a genuine trade-off against the star schema's simplicity; star schemas are more common in practice for their query simplicity and (especially combined with columnstore indexes, Part 27) strong performance.

51.3 Date Dimensions and Surrogate Keys

A dedicated `DimDate` table (pre-populated with one row per calendar date, often years into the future) is a near-universal data-warehouse convention — it lets date-based grouping/filtering (`Quarter`, `DayName`, fiscal periods) happen via simple joins and `WHERE` clauses against pre-computed columns, rather than repeatedly recomputing `DATEPART`-style expressions (which, per Part 30, are also often non-SARGable) across every fact-table query.

51.4 Building a Complete Sales Data Warehouse

The complete pattern combines Sections 51.1-51.3 with Part 50's ETL/SCD Type 2 loading pattern and Part 27's columnstore indexing on the fact table — together forming the standard architecture of a SQL-Server-based sales data warehouse: normalized OLTP source → staged, cleaned, and conformed via ETL → loaded into a star-schema warehouse with surrogate keys and SCD-tracked dimensions → queried via columnstore-accelerated analytical SQL (Part 47) → surfaced to business users through Power BI (Part 52).

52. SQL Server + Power BI

52.1 Import Mode vs DirectQuery

Mode	Behavior
Import	Power BI copies a snapshot of the data into its own compressed in-memory model — very fast query/interaction performance, but data is only as fresh as the last scheduled refresh.
DirectQuery	Power BI sends live queries back to SQL Server for every interaction — always current data, but performance depends entirely on the underlying SQL Server query performance (Parts 28-30), and the query patterns Power BI itself generates.

52.2 SQL Views for Power BI

```
CREATE VIEW Reporting.vw_SalesSummary WITH SCHEMABINDING AS
SELECT dc.Region, dd.Year, dd.Quarter, SUM(f.Amount) AS TotalRevenue,
COUNT_BIG(*) AS OrderCount
FROM Warehouse.FactOrders f
JOIN Warehouse.DimCustomer dc ON f.CustomerKey = dc.CustomerKey
JOIN Warehouse.DimDate dd ON f.DateKey = dd.DateKey
GROUP BY dc.Region, dd.Year, dd.Quarter;
```

Exposing a stable, well-named, pre-joined **view** (Part 19) — rather than having report authors connect directly to raw warehouse tables — is standard professional practice: it decouples the underlying schema from report definitions (a table rename/restructure doesn't break every downstream report), lets a DBA optimize the view definition centrally, and can itself be an indexed view (Section 19.3) for DirectQuery performance.

52.3 Query Optimization Before Power BI, Star Schema, Aggregations

DirectQuery performance is directly determined by the same optimization principles from Parts 28-30 applied to whatever queries Power BI generates — a star schema (Section 51.1) with appropriate indexes (including columnstore, Part 27) on the fact table is the standard foundation; Power BI's own **aggregation tables** feature (pre-summarized, smaller tables Power BI automatically queries instead of the full fact table when a report doesn't need row-level detail) further reduces the query burden reaching SQL Server.

52.4 Data Refresh and Gateway Concepts

For Import-mode reports, a scheduled **refresh** re-pulls data from SQL Server on a defined cadence; when Power BI Service (cloud) needs to reach an on-premises SQL Server, an **On-premises Data Gateway** — a locally installed service bridging the cloud service and the internal network — is required, since Power BI Service cannot directly reach a database behind a private network's firewall otherwise.

52.5 A Complete SQL Server → Power BI Analytical Architecture

```
OLTP Source (Sales.Orders, etc.)
    → ETL/ELT (Part 50) →
Data Warehouse (star schema, Part 51, columnstore-indexed, Part 27)
    → Reporting views (Section 52.2) →
Power BI (Import mode for most reports; DirectQuery for near-real-time
dashboards)
    → On-premises Data Gateway (if SQL Server is on-prem and Power BI
Service is cloud-hosted)
```

---

53. SQL Server for Backend Development

53.1 SQL Server Behind Web Applications and APIs

Banking, e-commerce, ERP, and HR systems (and most transactional line-of-business applications generally) rely on SQL Server as their backend of record precisely because of the properties covered throughout this handbook: ACID transactions (Part 23), strong referential integrity via constraints (Part 5), and mature security (Part 35) — a backend API layer (Flask, FastAPI, Django) sits between the client and SQL Server, translating HTTP requests into parameterized queries or stored procedure calls.

53.2 Flask Example

```
from flask import Flask, jsonify, request
from sqlalchemy import create_engine, text

app = Flask(__name__)
engine = create_engine('mssql+pyodbc://app_user:pass@localhost/Sales?
driver=ODBC+Driver+18+for+SQL+Server')

@app.route('/customers/<int:customer_id>/orders')
def get_orders(customer_id):
    with engine.connect() as conn:
        result = conn.execute(
            text("SELECT OrderID, OrderDate, TotalAmount FROM
Sales.Orders WHERE CustomerID = :cid"),
            {'cid': customer_id}
        )
        orders = [dict(row._mapping) for row in result]
    return jsonify(orders)
```

53.3 FastAPI Example

```
from fastapi import FastAPI
from sqlalchemy import create_engine, text

app = FastAPI()
engine = create_engine('mssql+pyodbc://app_user:pass@localhost/Sales?
driver=ODBC+Driver+18+for+SQL+Server')
```

```
@app.get("/customers/{customer_id}/orders")
def get_orders(customer_id: int):
    with engine.connect() as conn:
        result = conn.execute(
            text("SELECT OrderID, OrderDate, TotalAmount FROM
Sales.Orders WHERE CustomerID = :cid"),
            {'cid': customer_id}
        )
        return [dict(row._mapping) for row in result]
```

53.4 Django Example (via Django ORM's mssql backend)

```
# models.py
class Order(models.Model):
    order_id = models.AutoField(primary_key=True)
    customer_id = models.IntegerField()
    order_date = models.DateField()
    total_amount = models.DecimalField(max_digits=12,
decimal_places=2)
    class Meta:
        db_table = 'Sales.Orders'
        managed = False   # table already exists, managed outside
Django migrations

# views.py
def customer_orders(request, customer_id):
    orders = Order.objects.filter(customer_id=customer_id).values()
    return JsonResponse(list(orders), safe=False)
```

Django's ORM automatically parameterizes queries it generates, providing the same injection protection as Section 36.2/49.1 without manual parameter handling — a genuine convenience, though understanding the *underlying* parameterization principle (Part 36) remains essential for the raw-SQL escape hatches every ORM eventually requires for complex queries.

53.5 Connection Pooling, Transactions, Authentication, Authorization, CRUD APIs

These backend-integration concerns — connection pooling (Section 49.5), transactions wrapping multi-step API operations, authentication/authorization enforced at the API layer (often backed by SQL Server security principals and roles, Part 35, for defense in depth), parameterized queries preventing SQL injection (Part 36), and standard CRUD endpoint design (Part 6's operations exposed via HTTP verbs) — together form the standard architecture of a production SQL-Server-backed API, directly building on every mechanism covered earlier in this handbook rather than introducing new database concepts.

---

54. SQL Server for AI and Machine Learning

54.1 What Should Be Done in SQL Server vs Python/ML Frameworks

Task	Where It Belongs
Joining, filtering, aggregating raw transactional data	SQL Server (T-SQL)
Feature engineering expressible as SQL (Part 48)	SQL Server, when it reduces data volume before export
Model training itself	Python/ML frameworks (scikit-learn, PyTorch, etc.)
Complex feature engineering (embeddings, text processing, nonlinear transforms)	Python
Storing and serving predictions back to applications	SQL Server (Section 54.4)
Real-time feature lookup for a live prediction request	SQL Server (fast, indexed point lookups)

This division mirrors Section 48.4's SQL/Python division of labor directly, extended specifically to ML workflows: SQL Server is the right tool for *data preparation and serving*, not for the modeling/training step itself, which belongs in a purpose-built ML framework.

54.2 Training Dataset Preparation and Data Pipelines

```
-- Feature extraction query feeding a fraud-detection model's training
set
SELECT
    t.TransactionID, t.Amount,
    DATEDIFF(MINUTE, LAG(t.CreatedAt) OVER (PARTITION BY t.AccountID
ORDER BY t.CreatedAt), t.CreatedAt) AS MinutesSinceLastTxn,
    COUNT(*) OVER (PARTITION BY t.AccountID ORDER BY t.CreatedAt
                   RANGE BETWEEN INTERVAL '1' HOUR PRECEDING AND
CURRENT ROW) AS TxnCountLastHour,
    CASE WHEN t.IsFraud = 1 THEN 1 ELSE 0 END AS Label
FROM Banking.TransactionLog t;
```

(Note: T-SQL's window-frame syntax does not actually support `INTERVAL` literals like PostgreSQL's — a genuine T-SQL/PostgreSQL divergence — the equivalent time-based windowing in T-SQL typically requires a self-join or `CROSS APPLY` on a computed time boundary instead; shown here in illustrative pseudocode form to convey the *feature engineering intent* clearly.)

54.3 Model Scoring Workflows — Batch and Real-Time

```
-- Batch scoring: apply an already-trained model's logic (simplified
as a stored procedure call
-- to an external scoring service, or SQL Server Machine Learning
Services running Python/R in-database)
CREATE PROCEDURE ML.usp_ScoreChurnRisk
AS
```

```
BEGIN
    -- In practice: EXEC sp_execute_external_script @language =
N'Python', @script = N'...'
    -- (SQL Server Machine Learning Services allows running Python/R
scripts directly inside the engine)
    INSERT INTO ML.ChurnPredictions (CustomerID, ChurnProbability,
ScoredAt)
    SELECT CustomerID, /* model output */ 0.0, SYSDATETIME() FROM
Sales.Customers;
END;
```

Batch prediction runs on a schedule (via SQL Server Agent, Part 38) against a full customer/transaction set, writing results to a predictions table for later consumption. **Real-time prediction architecture** typically has the application call an external model-serving endpoint directly (not SQL Server) for the prediction itself, with SQL Server's role being fast feature lookup (indexed point queries retrieving a customer's current feature values) and storing the resulting prediction back (Section 54.4) — SQL Server rarely performs the model inference computation itself in a low-latency real-time path, aside from specific SQL Server Machine Learning Services scenarios.

54.4 Storing Predictions and Model Monitoring Data

```
CREATE TABLE ML.ChurnPredictions (
    PredictionID INT IDENTITY PRIMARY KEY,
    CustomerID INT, ChurnProbability DECIMAL(5,4), ModelVersion
NVARCHAR(20), ScoredAt DATETIME2
);

-- Model monitoring: track prediction distribution drift over time
SELECT ModelVersion, CAST(ScoredAt AS DATE) AS ScoreDate,
    AVG(ChurnProbability) AS AvgPredictedProbability, COUNT(*) AS
PredictionCount
FROM ML.ChurnPredictions
GROUP BY ModelVersion, CAST(ScoredAt AS DATE)
ORDER BY ScoreDate;
```

Storing every prediction with its model version and timestamp (rather than only the latest prediction per entity) supports both **auditability** (what did we predict, and with which model, when a business decision was made) and **model monitoring** — tracking whether the distribution of predictions is drifting over time, an early warning sign of model degradation or underlying data distribution shift.

54.5 AI Application Databases and Retrieval-Oriented Architectures

Modern AI applications (RAG pipelines, agent tool-calling systems — see the Testing and Debugging handbook's Part 23 for the Python-side testing perspective on these) still frequently rely on SQL Server for **structured, transactional data** the AI application needs to retrieve or act upon (customer records, order history, inventory) even when *unstructured* retrieval (embeddings/vector search) is handled by a separate specialized vector store — SQL Server 2025 has introduced native vector data type and search capabilities, narrowing this gap, but a

hybrid architecture (SQL Server for structured data and transactional writes; a dedicated vector store or SQL Server's native vector features for similarity search) remains a common, pragmatic professional pattern.

54.6 Realistic Examples

- **Fraud detection:** real-time feature lookup (Section 54.3) against recent transaction history stored in SQL Server, feeding a model-serving endpoint; flagged transactions written back for case-management review.
- **Customer churn:** batch-scored monthly (Section 54.3), predictions stored (Section 54.4) and joined against CRM data to drive proactive retention outreach.
- **Credit risk:** feature engineering (payment history aggregates, Part 48) computed in SQL Server, feeding an external risk-scoring model; the resulting score stored back onto the customer/account record for underwriting decisions.
- **Sales forecasting:** historical sales data aggregated by the data warehouse (Part 51), exported to Python for time-series modeling, forecasts written back to SQL Server for BI reporting (Part 52).
- **Customer recommendation:** market-basket-style co-occurrence features (Section 47.3) computed in SQL, feeding a recommendation model whose output (top-N recommended products per customer) is stored back for fast retrieval by the application's product pages.

55. PostgreSQL vs MySQL vs SQL Server

55.1 Comprehensive Feature Comparison

Feature	PostgreSQL	MySQL	SQL Server
Auto-increment	`SERIAL` / `GENERATED ALWAYS AS IDENTITY`	`AUTO_INCREMENT`	`IDENTITY(seed, increment)`
Standalone sequences	Native `SEQUENCE`	No native sequence object	Native `SEQUENCE`
String concatenation	double-pipe (two vertical bars) or `CONCAT()`	`CONCAT()` (double-pipe is logical OR unless `PIPES_AS_CONCAT` set)	`+` or `CONCAT()`/`CONCAT_WS()`
Limit/pagination	`LIMIT n OFFSET m`	`LIMIT m, n` or `LIMIT n OFFSET m`	`TOP (n)` or `OFFSET...FETCH`
Boolean type	Native `boolean`	`TINYINT(1)` convention (no true boolean)	No native type; `bit` (0/1/NULL)
NULL-safe equality	`IS NOT DISTINCT FROM`	`<=>`	No direct operator; use `CASE`/`ISNULL` comparison
JSON type	Native `json`/`jsonb`, indexable	Native `JSON` type	No native type; `nvarchar(max)` + JSON functions (Part 31)
Window functions	Full support	Full support (8.0+)	Full support
CTEs	Full support, `WITH RECURSIVE`	Full support (8.0+), `WITH RECURSIVE`	Full support, `WITH` (no `RECURSIVE` keyword needed)
Temporary tables	`CREATE TEMP TABLE`	`CREATE TEMPORARY TABLE`	`#Table` (session) / `##Table` (global)
Default index type	B-tree (heap table storage otherwise)	Clustered by primary key always (InnoDB)	Heap by default; explicit clustered index optional
Materialized/ Indexed views	`MATERIALIZED VIEW` (manual `REFRESH`)	Not supported natively	Indexed views (automatically maintained)
Transactions/ Isolation default	MVCC-based, `Read Committed` default	MVCC-based (InnoDB), `Repeatable Read` default	Lock-based, `Read Committed` default (RCSI optional)
Stored procedures	`CREATE`	`CREATE PROCEDURE`	`CREATE PROCEDURE`

Feature	PostgreSQL	MySQL	SQL Server
	PROCEDURE/FUNCTION (PL/pgSQL)	(limited vs PL/pgSQL/T-SQL)	(T-SQL)
Triggers	Full support, including per-statement	Full support, per-row only	Full support (Part 22)
Backup	pg_dump/ pg_basebackup, continuous WAL archiving	mysqldump/ mysqlpump, binary log	BACKUP DATABASE/BACKUP LOG (Part 37)
Row-level security	Native ROW LEVEL SECURITY/CREATE POLICY	Not supported natively	Native (Part 35)
Columnstore/ analytics	Via extensions (Citus, cstore_fdw)	Not supported natively	Native columnstore indexes (Part 27)
Regex support	Native (~, SIMILAR TO, regex functions)	Native (REGEXP)	Not native (Section 33.4)

55.2 Equivalent Query Translation Table

Task	PostgreSQL	MySQL	SQL Server
Limit to 10 rows	SELECT * FROM t LIMIT 10;	SELECT * FROM t LIMIT 10;	SELECT TOP 10 * FROM t;
Paginate (skip 20, take 10)	SELECT * FROM t ORDER BY id LIMIT 10 OFFSET 20;	SELECT * FROM t ORDER BY id LIMIT 20, 10;	SELECT * FROM t ORDER BY id OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;
Current timestamp	SELECT NOW();	SELECT NOW();	SELECT SYSDATETIME();
String concatenation	SELECT first double-pipe ' ' double-pipe last;	SELECT CONCAT(first, ' ', last);	SELECT CONCAT(first, ' ', last);
Auto-increment PK	id SERIAL PRIMARY KEY	id INT AUTO_INCREMENT PRIMARY KEY	id INT IDENTITY(1,1) PRIMARY KEY
Upsert	INSERT ... ON CONFLICT (id) DO UPDATE SET ...	INSERT ... ON DUPLICATE KEY UPDATE ...	MERGE (Part 61) or explicit IF EXISTS ... UPDATE ELSE INSERT
Group-concat strings	STRING_AGG(col, ',')	GROUP_CONCAT(col SEPARATOR ',')	STRING_AGG(col, ',')
Case-insensitive match	WHERE col ILIKE 'value'	WHERE col LIKE 'value' (often	WHERE col LIKE 'value' (case

Task	PostgreSQL	MySQL	SQL Server
		case-insensitive by default collation)	sensitivity depends on collation)

56. SQL Portability

56.1 Identifying ANSI SQL vs Vendor-Specific Syntax

Category	Examples
ANSI SQL (portable)	`SELECT`, `WHERE`, `JOIN` types, `GROUP BY`/`HAVING`, `CASE`, standard CTEs, `OFFSET...FETCH`, window functions, `UNION`/`INTERSECT`/`EXCEPT`
PostgreSQL-specific	`SERIAL`, `RETURNING`, `ILIKE`, array types, `jsonb` operators, `LATERAL`
MySQL-specific	`AUTO_INCREMENT`, `LIMIT m, n`, `ON DUPLICATE KEY UPDATE`, backtick identifiers
SQL Server-specific	`TOP`, `IDENTITY`, `OUTPUT`, `CROSS APPLY`/`OUTER APPLY`, `PIVOT`/`UNPIVOT`, square-bracket identifiers, `[T-SQL]`-labeled items throughout this handbook

56.2 Writing Portable SQL

```
-- Portable, ANSI-standard pagination - works unmodified in
PostgreSQL, MySQL 8+, and SQL Server
SELECT CustomerID, TotalAmount FROM Sales.Orders
ORDER BY OrderDate DESC
OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;

-- Vendor-specific alternative (SQL Server) - functionally equivalent,
NOT portable
SELECT TOP 10 CustomerID, TotalAmount FROM Sales.Orders ORDER BY
OrderDate DESC;
```

Favoring `OFFSET...FETCH` over `TOP`, standard CTEs over vendor-specific procedural extensions, and `CASE` over any single database's proprietary conditional shortcuts are all small, deliberate, and cumulative choices that keep application-level SQL portable when portability is a genuine project requirement (e.g. a product intended to support multiple database backends, or an organization anticipating a future migration).

56.3 Why Understanding Both Portable and Vendor-Specific SQL Matters

Writing exclusively portable SQL leaves substantial performance and functionality on the table — `MERGE`, `CROSS APPLY`, columnstore indexes, and Row-Level Security (all covered in this handbook) have no portable equivalent, and avoiding them entirely for the sake of theoretical portability is frequently the wrong trade-off for a SQL-Server-only production system. The

professional standard: **default to portable SQL when a project's requirements genuinely call for cross-database compatibility; deliberately and knowingly use vendor-specific features when their concrete benefit (performance, functionality, or genuine developer productivity) outweighs portability for that specific project** — and always know, explicitly, which category any given piece of SQL falls into, rather than discovering it painfully during a future migration attempt.

57. Real-World SQL Server Projects

57.1 Project 1 — Retail Sales Database

Business problem: A retailer needs a database tracking products, customers, and orders, supporting both transactional operations and basic reporting.

Tables: `Customers(CustomerID PK, FullName, Email, Region)`, `Products(ProductID PK, ProductName, Category, UnitPrice, UnitCost)`, `Orders(OrderID PK, CustomerID FK, OrderDate, Status)`, `OrderItems(OrderID FK, ProductID FK, Quantity, UnitPrice, PK(OrderID, ProductID))`.

Key constraints: `CHECK (Quantity > 0)`, `FOREIGN KEY` cascade rules considered carefully (avoid cascading deletes on financial data — Part 61).

Sample queries: revenue by category (Section 47.1), top customers (Section 14.5), stock reconciliation via `OrderItems` aggregation.

Stored procedure: `usp_PlaceOrder` (validates stock, inserts `Orders`/`OrderItems` in one transaction, returns the new `OrderID` via `OUTPUT`).

View: `vw_OrderSummary` joining all four tables for reporting consumption.

Indexing: nonclustered index on `Orders.CustomerID`, covering index on `OrderItems (ProductID) INCLUDE (Quantity)` for stock reports.

Security: a `ReportViewer` role with `SELECT`-only access to views, never base tables.

Deliverables: full schema script, seed data script, 10+ analytical queries, one stored procedure, one view, a documented indexing rationale.

57.2 Project 2 — Banking Transaction System

Business problem: Track accounts and transactions with strict correctness (no lost updates, no partial transfers) and full auditability.

Tables: `Accounts(AccountID PK, CustomerID, Balance, Status)`, `TransactionLog(TransactionID PK, AccountID FK, Amount, TransactionType, CreatedAt)`.

Core logic: `usp_ProcessTransaction` (Section 20.4) with `TRY/CATCH` and explicit transactions; a temporal table (Part 42) on `Accounts` for full balance-history auditability; Row-Level Security (Part 35) restricting each branch's staff to their own branch's accounts.

Analytical queries: suspicious-transaction detection (Section 20.4), daily reconciliation totals, running account balance via window functions (Part 14).

Security considerations: Always Encrypted (Part 35) on account numbers; least-privilege roles separating teller, auditor, and administrator access.

Deliverables: schema with temporal versioning enabled, transaction-processing procedure, RLS policy, reconciliation report query, encryption configuration notes.

57.3 Project 3 — Hospital Management Database

Business problem: Manage patients, appointments, and treatment records with strict privacy requirements.

Tables: `Patients`, `Doctors`, `Appointments`, `TreatmentRecords`, with `Patients.SSN` and similar fields under Dynamic Data Masking (Section 35.5).

Key design point: normalization to 3NF (Part 34) is especially important here to avoid inconsistent patient/doctor data duplicated across records.

Analytical queries: appointment no-show rates, doctor utilization, patient visit frequency (a direct healthcare application of Part 47's retention-analysis patterns).

Security: row-level security restricting each doctor's default view to their own patients; full audit trail via temporal tables (Part 42) on `TreatmentRecords` for compliance.

57.4 Project 4 — HR Management System

Business problem: Track employees, departments, salaries, and organizational hierarchy.

Tables: `Employees(EmployeeID PK, ManagerID FK self-referencing, DepartmentID FK, Salary)`, `Departments`.

Key technique: recursive CTE (Section 13.3) for the organizational chart; temporal table (Part 42) on `Employees` for salary-history auditing (directly reusing Section 42.4's project).

Analytical queries: salary ranking by department (Section 14.5), employees earning more than their manager (Part 59's classic interview question, Section 12.2-style self-join).

57.5 Project 5 — E-commerce Database

Business problem: A full e-commerce backend: catalog, cart, checkout, and order fulfillment.

Tables: `Products`, `Customers`, `Carts`, `CartItems`, `Orders`, `OrderItems`, `Payments`, `Shipments`.

Backend integration: exposed via a FastAPI layer (Section 53.3) with parameterized queries throughout; `OUTPUT` clause (Section 6.4) used to return the created order ID directly from the checkout `INSERT`.

Analytical queries: cart abandonment rate (funnel analysis, Section 47.3), best-selling products (Section 14.5), month-over-month revenue growth (Section 14.5).

Optimization: covering index on `OrderItems (ProductID) INCLUDE (Quantity, UnitPrice)` for fast best-seller aggregation; columnstore index (Part 27) on a reporting-oriented `OrderItems` copy for BI queries.

57.6 Project 6 — Inventory Management System

Business problem: Track stock levels across multiple warehouses, with automatic low-stock alerting.

Tables: `Warehouses`, `Products`, `Inventory(WarehouseID, ProductID, QuantityOnHand, PK composite)`, `StockMovements`.

Automation: a SQL Server Agent job (Part 38) running a nightly low-stock-check procedure, emailing an operator alert when `QuantityOnHand` falls below a `ReorderThreshold`.

Analytical queries: inventory turnover rate, days-of-stock-remaining estimates using average daily consumption (a rolling-average window function, Section 14.4).

57.7 Project 7 — Fraud Detection Database

Business problem: Flag potentially fraudulent transactions in near real time.

Tables: `Transactions`, `FraudFlags`, `Customers`.

Key technique: rule-based detection procedure (Section 20.4's `usp_DetectSuspiciousTransactions` pattern extended); feature engineering queries (Section 54.2) preparing training data for an external ML model; predictions written back (Section 54.4) with model version tracking.

Analytical queries: false-positive rate tracking (comparing `FraudFlags` against later-confirmed legitimate transactions), fraud rate trends over time.

57.8 Project 8 — Customer Churn Analytics System

Business problem: Identify customers at risk of churning and quantify retention program effectiveness.

Tables: `Customers`, `Subscriptions`, `UsageEvents`, `ChurnPredictions`.

Key technique: cohort analysis (Section 47.2), churn-label construction (Section 48.3), feature engineering for a churn model (Section 48.2), predictions stored with model monitoring (Section 54.4).

Analytical queries: cohort retention curves, churn rate by acquisition channel, the financial impact of churn (lost revenue) via `SUM(TotalAmount)` joined against churned-customer flags.

57.9 Project 9 — Financial Data Warehouse

Business problem: Consolidate financial transactions from multiple source systems into a queryable warehouse for finance-team reporting.

Design: star schema (Part 51) with `FactTransactions`, `DimAccount`, `DimDate`, `DimCostCenter`; SCD Type 2 (Section 50.2) on `DimAccount` for historical account-hierarchy changes; CDC (Part 43) from source OLTP systems feeding incremental ETL (Part 50).

Analytical queries: budget-vs-actual variance, quarter-over-quarter trend analysis, Pareto analysis of cost centers (Section 47.4).

Optimization: clustered columnstore index (Part 27) on `FactTransactions`; partitioning (Part 44) by fiscal year with partition-switching for year-end archival.

57.10 Project 10 — Enterprise Business Intelligence Platform

Business problem: A unifying BI platform combining sales, HR, and finance data for executive dashboards.

Design: a conformed-dimension warehouse (shared `DimDate`, `DimEmployee` reused across sales/HR/finance fact tables) — the most architecturally mature project in this set, directly combining Parts 47-52's analytics, warehousing, and Power BI integration content into one system.

Deliverables: full star-schema design across three business domains, ETL pipeline (Part 50) with CDC-based incremental loading (Part 43), Power BI semantic layer (Section 52.2's reporting views), row-level security (Part 35) ensuring each department sees only its own detailed data while executives see all of it, and a documented refresh/gateway architecture (Section 52.4).

---

58. Complete SQL Server Capstone

58.1 Capstone Architecture

```
Raw Data (CSV exports from a retail POS system)
    ↓
SQL Server (Staging schema - Part 39's BULK INSERT)
    ↓
Data Cleaning (Part 33 - TRY_CAST, duplicate removal, outlier
flagging)
    ↓
Data Warehouse (Part 51 - star schema, SCD Type 2, columnstore-indexed
FactSales)
    ↓
Analytical SQL (Part 47 - revenue, cohort, retention, Pareto queries)
    ↓
Python (Part 49 - pandas pulls aggregated features via SQLAlchemy)
    ↓
Machine Learning (scikit-learn - churn/CLV model trained on SQL-
prepared features)
    ↓
Predictions (Part 54 - written back to SQL Server's
ML.ChurnPredictions table)
    ↓
SQL Server (predictions joined with warehouse dimensions for
reporting)
    ↓
```

Power BI (Part 52 - executive dashboard combining actuals and predictions)

58.2 Stage-by-Stage Explanation

Raw Data → SQL Server: Nightly POS export files land via `BULK INSERT` (Part 39) into a `Staging` schema, all-text columns, no constraints — capturing data exactly as received before any assumptions are applied.

Data Cleaning: `TRY_CAST/TRY_CONVERT` (Section 9.4) validate and convert staged data; malformed rows are quarantined (Section 33.6) rather than silently dropped or allowed to break the pipeline; duplicate transactions are detected and removed deterministically (Section 33.2).

Data Warehouse: Cleaned data is loaded into a star schema (Part 51) — `FactSales` (columnstore-indexed, Part 27, for analytical performance) joined to `DimCustomer` (SCD Type 2, tracking customer attribute changes over time), `DimProduct`, and `DimDate`.

Analytical SQL: A library of views (Section 52.2) and parameterized stored procedures compute revenue trends, customer cohorts (Section 47.2), and Pareto/CLV metrics (Section 47.4) directly against the warehouse.

Python: A scheduled Python job connects via SQLAlchemy (Part 49), pulling a **pre-aggregated, feature-engineered** dataset (customer-level RFM — recency, frequency, monetary — features, Section 48.2) rather than raw transaction rows, keeping the expensive aggregation work inside SQL Server where it's most efficient.

Machine Learning: scikit-learn trains a churn-classification model (or a CLV regression model) on the pulled features, following the ML-testing discipline from the Testing and Debugging handbook's Part 22 (train/test split with no leakage, reproducible with a fixed seed).

Predictions → SQL Server: The trained model scores the full customer base; predictions, along with model version and scoring timestamp (Section 54.4), are written back via `to_sql()` (Section 49.4) into a dedicated predictions table.

Power BI: A final reporting view (Section 52.2) joins warehouse actuals with the predictions table, feeding an executive dashboard showing both historical performance and forward-looking churn/CLV risk — combining every layer of this capstone into one decision-support artifact.

58.3 Why This Architecture Reflects Professional Practice

Every stage does the work it is genuinely best suited for: SQL Server handles bulk data movement, cleaning, and set-based aggregation (its core strength, per Part 30); Python/scikit-learn handles the actual statistical modeling (its core strength); and the round-trip back into SQL Server keeps predictions alongside the operational and warehouse data business users already query through Power BI — no separate, disconnected "data science silo" that the rest of the organization can't easily consume.

59. Interview Preparation

59.1 Beginner (Q1-Q10)

Q1. What is the difference between SQL Server, T-SQL, and SQL? SQL is the standard query language; T-SQL is Microsoft's extended dialect adding procedural constructs and SQL-Server-specific syntax; SQL Server is the database engine that executes T-SQL (Section 1.2).

Q2. What is the object hierarchy in SQL Server? Instance → Database → Schema → Objects → Rows/Columns (Section 1.5).

Q3. What is the difference between a login and a user? A login is a server-level identity; a user is a database-level identity mapped to a login (Section 1.6).

Q4. What are the four system databases? `master`, `model`, `msdb`, `tempdb` (Section 1.8).

Q5. What is the difference between `CHAR` and `VARCHAR`? `CHAR` is fixed-length, space-padded; `VARCHAR` is variable-length, storing only the actual content length.

Q6. What does `IDENTITY(1,1)` mean? An auto-incrementing column starting at 1, incrementing by 1 per new row (Section 5.3).

Q7. What is the difference between `DELETE` and `TRUNCATE`? `DELETE` can be filtered with `WHERE` and is fully logged row-by-row; `TRUNCATE` removes all rows, cannot be filtered, and is minimally logged (Section 5.1).

Q8. What does `TOP` do, and how does it differ from `LIMIT`? Restricts the number of returned rows; SQL Server's `TOP` syntax precedes the column list, unlike PostgreSQL/MySQL's trailing `LIMIT` (Section 7.2).

Q9. What is a primary key, and can a table have more than one? A column (or set of columns) uniquely identifying each row; a table can have only one primary key, though it can be composite (multiple columns, Section 5.2).

Q10. What is `NULL`, and how does it differ from an empty string or zero? `NULL` represents an unknown/missing value; it is never equal to an empty string or zero, and any comparison with `NULL` yields `UNKNOWN`, not `TRUE/FALSE`.

59.2 Intermediate (Q11-Q20)

Q11. What is the difference between `INNER JOIN` and `LEFT JOIN`? `INNER JOIN` returns only matching rows; `LEFT JOIN` returns all rows from the left table, with `NULL`s for unmatched right-side columns (Section 11.1).

Q12. Why can `NOT IN` behave unexpectedly with `NULL` values? If the subquery's result set contains any `NULL`, the entire `NOT IN` comparison evaluates to `UNKNOWN` for every row, returning zero rows (Section 12.4).

Q13. What is a CTE, and when would you use one? A named temporary result set (`WITH ... AS (...)`), scoped to one statement, used to improve readability of multi-step queries (Part 13).

Q14. What is the difference between `RANK()` and `DENSE_RANK()`? `RANK()` skips subsequent rank values after a tie; `DENSE_RANK()` does not (Section 14.2).

Q15. What is the difference between a temp table and a table variable? A temp table supports full indexing and better statistics; a table variable has more limited indexing and historically weaker cardinality estimation, but is unaffected by transaction rollback (Part 15).

Q16. What does `CROSS APPLY` do that a regular `JOIN` cannot? Allows the right-hand table expression to be correlated to each row of the left side, such as a per-row `TOP N` subquery (Part 16).

Q17. What is the difference between a view and an indexed view? A standard view is just stored SQL, re-executed on each query; an indexed view physically materializes and automatically maintains its result set (Section 19.3).

Q18. What does `SCOPE_IDENTITY()` return, and why is it preferred over `@@IDENTITY`? The last identity value generated in the current session and scope; `@@IDENTITY` can incorrectly return a value from an unrelated trigger-fired insert (Section 5.3).

Q19. What is the purpose of `TRY_CAST`/`TRY_CONVERT`? They return `NULL` on a failed conversion instead of raising an error, valuable for cleaning messy data in one pass (Section 9.4).

Q20. What is the difference between `ISNULL` and `COALESCE`? `ISNULL` is T-SQL-specific, takes exactly 2 arguments; `COALESCE` is standard SQL, takes 2 or more, with more predictable type resolution (Section 9.5).

59.3 Advanced (Q21-Q30)

Q21. What is a non-SARGable predicate, and how do you fix one? A predicate that prevents index seek usage, typically by wrapping an indexed column in a function; fix by rewriting to isolate the raw column (Section 29.4, Section 30.1).

Q22. What is parameter sniffing? A cached execution plan compiled for one parameter value performing poorly for other, differently-shaped parameter values (Section 29.5).

Q23. What is the difference between a clustered and nonclustered index? A clustered index determines physical row storage order (max one per table); a nonclustered index is a separate structure pointing back to the table (Sections 26.2-26.3).

Q24. What is a covering index? A nonclustered index containing every column a specific query needs, avoiding a key lookup (Section 26.5).

Q25. What is the difference between `RCSI` and `SNAPSHOT` isolation? RCSI provides statement-level row versioning as a drop-in replacement for `Read Committed`'s locking behavior; `SNAPSHOT` provides transaction-level consistency and can raise write-write conflict errors (Section 24.4).

Q26. What causes a deadlock, and how does SQL Server resolve one? Two transactions holding locks the other needs, forming a cycle; SQL Server detects it and terminates the lower-cost "victim" transaction (Section 24.3).

Q27. Why is `THROW` generally preferred over `RAISERROR`? A bare `THROW` in a `CATCH` block re-raises the original error with its original context intact; `RAISERROR` always generates a new error at the `CATCH` block's location (Section 25.3).

Q28. What is the difference between CDC and Change Tracking? CDC captures full old/new column values; Change Tracking only records that a row changed, at lower overhead (Part 43).

Q29. What is partition switching, and why is it fast? Moving a partition's data in/out of a table via a metadata-only operation, with no physical data movement, regardless of row count (Section 44.2).

Q30. What is the difference between High Availability and Disaster Recovery? HA minimizes downtime via synchronized standby replicas; DR protects against data loss and provides recovery after catastrophic failure — complementary, not substitutes (Section 45.1).

59.4 Senior (Q31-Q40)

Q31. How would you diagnose a query that suddenly became slow after months of good performance? Check for stale statistics first, then compare estimated vs actual row counts in the execution plan, then look for non-SARGable predicates or implicit conversions before considering index changes (Sections 28.6, 29.6).

Q32. When would you choose a nonclustered columnstore index over a clustered rowstore index? When a transactional table needs to also support fast, large-scale analytical aggregation without migrating its primary storage format (Section 27.2).

Q33. How do you design a schema to support both OLTP and OLAP workloads on the same SQL Server instance? Keep the OLTP schema normalized for transactional integrity; feed a separately-modeled, denormalized star-schema warehouse via ETL/CDC for analytical workloads, isolating the two access patterns (Sections 34.4, 50, 51).

Q34. What's the risk of over-relying on missing-index DMV suggestions? They don't account for the write-side maintenance cost of additional indexes; blindly applying every suggestion can degrade write performance without properly evaluating the trade-off (Section 26.9).

Q35. How would you implement point-in-time recovery for a critical production database? Use the Full recovery model with regular transaction log backups, then `RESTORE DATABASE ... WITH NORECOVERY` followed by `RESTORE LOG ... WITH RECOVERY, STOPAT` (Section 37.3).

Q36. How does Always On Availability Groups differ architecturally from Log Shipping? Availability Groups provide near-real-time, often automatic failover with optional readable secondaries; Log Shipping is simpler but has higher latency and typically requires manual failover (Section 45.3).

Q37. What are the trade-offs of Always Encrypted vs Transparent Data Encryption? TDE protects data at rest transparently but is fully visible to authorized server-side queries/DBAs;

Always Encrypted protects specific column values even from privileged server-side access, at the cost of requiring client-side key management (Section 35.6).

Q38. How would you safely perform a large-scale data archival without locking a live production table? Use partition switching (Section 44.2) to move an entire partition's data out as a near-instant metadata operation, rather than a row-by-row `DELETE`.

Q39. What's the professional argument against overusing triggers for business logic? Triggers execute implicitly and invisibly to calling code, making systems harder to debug and reason about; explicit logic in stored procedures or application code is generally preferred except for genuine cross-cutting concerns (Section 22.4).

Q40. How would you evaluate whether a query should be rewritten as set-based instead of using a cursor? Nearly any row-by-row procedural loop performing a per-row DML operation can typically be rewritten as a single set-based statement; cursors should be reserved for genuinely sequential administrative tasks, not ordinary business logic (Section 30.4).

59.5 Data Analyst (Q41-Q50)

Q41. How do you calculate month-over-month growth in T-SQL? Using `LAG()` over a monthly-aggregated CTE, comparing each month's value to the previous month's (Section 14.5).

Q42. How do you find the top 3 products per category? `ROW_NUMBER() OVER (PARTITION BY Category ORDER BY Revenue DESC)` filtered to `<= 3`, or `CROSS APPLY` with `TOP 3` per category (Sections 14.2, 16.3).

Q43. How would you calculate a running total of sales? `SUM(Amount) OVER (ORDER BY Date ROWS UNBOUNDED PRECEDING)` (Section 14.4).

Q44. How do you identify customers with no transactions? `LEFT JOIN` with a `WHERE ... IS NULL` filter, or `NOT EXISTS` (Sections 11.1, 12.3).

Q45. What is cohort analysis, and how would you implement it in SQL? Grouping customers by their acquisition period and tracking their activity in subsequent periods, typically via a CTE joining signup month to activity month (Section 47.2).

Q46. How do you calculate customer retention rate? Comparing the set of customers active in one period against those still active in a subsequent period, typically via `INTERSECT` or a self-join on customer ID across periods.

Q47. What is Pareto analysis, and how is it computed in SQL? Identifying what proportion of entities account for a given proportion of a total (the "80/20 rule"), computed via a running-total window function divided by the grand total (Section 47.4).

Q48. How would you find the second-highest salary in a table? `SELECT MAX(Salary) FROM Employees WHERE Salary < (SELECT MAX(Salary) FROM Employees)`, or `DENSE_RANK()` filtered to rank 2 (Section 60's coding challenges).

Q49. What's the difference between `PIVOT` and conditional aggregation with `CASE`? `PIVOT` is a dedicated T-SQL operator for a known, fixed set of columns; conditional aggregation is more flexible and portable but slightly more verbose (Part 17).

Q50. How would you detect gaps in a sequence of dates? Compare each date to the previous date via `LAG()`, flagging where the difference exceeds one day (Section 60's coding challenges).

59.6 Data Scientist (Q51-Q58)

Q51. What kinds of feature engineering are best done in SQL vs Python? Aggregations, joins, and simple derived columns are efficient in SQL; complex nonlinear transforms, embeddings, and text processing are better suited to Python (Section 48.4).

Q52. How would you construct a training label in SQL for a churn model? A `CASE/NOT EXISTS` expression checking whether a customer has any activity within a defined future window after a reference date (Section 48.3).

Q53. How do you prevent data leakage when preparing features in SQL for a time-series-sensitive model? Ensure every feature is computed using only data available *as of* the reference/prediction date, never including future information relative to that point.

Q54. Why might you prefer `TABLESAMPLE` over `ORDER BY NEWID()` for large-table sampling? `TABLESAMPLE` samples at the physical page level without a full table sort, making it far faster on large tables, at some cost to sampling uniformity (Section 48.1).

Q55. How would you version and monitor ML predictions stored in SQL Server? Store `ModelVersion` and `ScoredAt` alongside each prediction, enabling drift monitoring queries over time (Section 54.4).

Q56. What's the professional division of labor between SQL Server and a Python ML framework? SQL Server handles data preparation, aggregation, and serving; Python/ML frameworks handle model training and complex feature transforms (Section 54.1).

Q57. How would you join SQL-extracted features to a pandas DataFrame for modeling? Query the aggregated feature set directly into a DataFrame via `pandas.read_sql()` over a SQLAlchemy engine (Section 49.3).

Q58. Why is it important to push aggregation into SQL rather than pulling raw rows into pandas? SQL Server's set-based engine performs large-scale aggregation far more efficiently than row-by-row processing in Python, and reduces the data volume transferred (Section 48.4).

59.7 Data Engineer (Q59-Q68)

Q59. What's the difference between ETL and ELT? ETL transforms data before loading into the target; ELT loads raw data first, then transforms using the target system's own processing power (Section 50.1).

Q60. How would you implement incremental loading? A high-water-mark pattern tracking the last successfully loaded timestamp, or CDC/Change Tracking for more granular incremental capture (Sections 43, 50.2).

Q61. What is a Slowly Changing Dimension, and when would you use Type 2? A dimension table design pattern preserving history; Type 2 is used when historical accuracy of a changing attribute (e.g. a customer's region at the time of a past order) must be preserved (Section 50.2).

Q62. What is the difference between CDC and transactional replication for ETL purposes? CDC captures changes for consumption by a separate ETL process on a schedule; transactional replication continuously propagates changes in near real time to another database (Sections 43, 46.2).

Q63. Why use a star schema rather than a fully normalized schema for a data warehouse? Denormalization reduces the number of joins needed for large aggregate queries, which matters more for OLAP read performance than storage redundancy does (Section 34.3, Part 51).

Q64. What is partition elimination, and why does it matter for large fact tables? The optimizer's ability to skip scanning irrelevant partitions entirely when a query filters on the partitioning column, improving performance on very large tables (Section 44.3).

Q65. How would you handle a malformed row during a bulk import without aborting the entire load? Use `BULK INSERT`'s `MAXERRORS` and error-file options, or land all data into an all-text staging table and validate/convert afterward (Sections 33.6, 39.2).

Q66. What automation tool would you use to schedule a nightly ETL job in SQL Server? SQL Server Agent, defining a job with appropriate steps and a schedule (Part 38).

Q67. How would you design a conformed dimension shared across multiple fact tables? Build one authoritative `DimDate`/`DimCustomer` table referenced by every relevant fact table, ensuring consistent granularity and keys across subject areas (Section 57.10).

Q68. What role does `OUTPUT` play in an ETL procedure? Capturing exactly which rows were inserted/updated/deleted in a single statement, useful for logging and auditing pipeline behavior (Part 6).

59.8 Backend Developer (Q69-Q78)

Q69. How do you prevent SQL injection when building dynamic queries? Always pass untrusted values as parameters via `sp_executesql` (T-SQL) or a driver's parameterized query mechanism, never string concatenation (Sections 36.2, 49.1).

Q70. What is connection pooling, and why does it matter for a web API? Reusing a bounded set of already-open database connections rather than opening a new one per request, avoiding significant per-connection overhead under load (Section 49.5).

Q71. How would you wrap a multi-step database operation in a transaction from Python? Use `engine.begin()` as a context manager in SQLAlchemy, which commits on success and rolls back on any exception (Section 49.5).

Q72. What's the benefit of `OUTPUT` for an API's create-resource endpoint? Returning the newly created row's generated ID directly from the `INSERT` statement, without a separate follow-up query (Section 6.4).

Q73. How does an ORM like Django's typically prevent SQL injection? It automatically parameterizes generated queries rather than concatenating values into SQL text (Section 53.4).

Q74. What SQL Server security principle should a backend service account follow? Least privilege — granting only the exact permissions the service genuinely needs, never broad roles like `db_owner` by default (Section 35.3).

Q75. How would you design a CRUD API's database access layer for testability? Separate query-building logic from execution, allowing the database connection/session to be injected and mocked in tests (a direct application of the Testing handbook's Part 38 dependency-injection principle).

Q76. What's a common mistake when building dynamic filtering for a search endpoint? Concatenating filter values directly into SQL text instead of building the WHERE clause dynamically but passing values as parameters (Section 36.3).

Q77. Why might a backend prefer stored procedures over inline application-level SQL for certain operations? Centralizing complex business logic and transaction handling in the database, providing a stable interface less coupled to application code changes, and enabling database-level permission grants scoped to specific procedures rather than raw table access.

Q78. How would you handle a transient connection failure to SQL Server from a backend service? Implement retry logic with backoff around the connection/query call, similar in principle to the API retry patterns covered in the Testing handbook's Part 13.

59.9 Database Developer (Q79-Q88)

Q79. When would you choose a scalar function vs an inline table-valued function? Scalar functions for a single computed value; inline TVFs when the reusable logic naturally produces a set of rows, since they inline into the query plan more efficiently than multi-statement TVFs (Part 21).

Q80. What is the difference between a standard view and an indexed view? An indexed view physically materializes and automatically maintains its result set, requiring `SCHEMABINDING` and a unique clustered index (Section 19.3).

Q81. How would you implement a recursive hierarchy query? A recursive CTE, with an anchor member selecting the top-level rows and a recursive member joining back to the CTE itself (Section 13.3).

Q82. What is the purpose of `SET NOCOUNT ON` in a stored procedure? Suppresses row-count messages, reducing unnecessary network chatter — a standard professional habit (Section 20.1).

Q83. How do you safely build a dynamic PIVOT query? Construct the column list dynamically, then execute the assembled query via `sp_executesql`, keeping any actual data values parameterized (Section 17.3).

Q84. What's the danger of using `MERGE` in SQL Server, and what's a safer alternative? `MERGE` has had historical concurrency-related bugs and subtle correctness issues under concurrent access; a safer alternative is separate, explicit `UPDATE` and `INSERT` statements wrapped in a transaction (Part 61).

Q85. How would you design a table to support both current and historical queries without manual audit triggers? Use a system-versioned temporal table, which automatically maintains history with no additional application logic (Part 42).

Q86. What's the difference between a DML trigger and a DDL trigger? A DML trigger fires on data changes (`INSERT/UPDATE/DELETE`); a DDL trigger fires on schema changes (`CREATE/ALTER/DROP`) (Part 22).

Q87. How would you enforce a business rule spanning multiple tables that a `CHECK` constraint cannot express? A trigger, though used sparingly given Section 22.4's maintainability caveat; alternatively, enforce it in a stored procedure that is the sole sanctioned write path.

Q88. What's the professional guidance on function usage in `WHERE` clauses against indexed columns? Avoid wrapping the indexed column itself in a function, since it defeats SARGability; transform the comparison side of the expression instead (Section 30.1).

59.10 Database Administrator (Q89-Q100)

Q89. What's the difference between the Simple and Full recovery models? Simple truncates the log automatically and supports no log backups; Full retains the log for point-in-time recovery via log backups (Section 3.5).

Q90. How would you restore a database to a specific point in time? Restore the last full backup `WITH NORECOVERY`, any differential `WITH NORECOVERY`, then the log backup(s) `WITH RECOVERY, STOPAT` (Section 37.3).

Q91. What is lock escalation, and why might it be undesirable? SQL Server automatically converting many fine-grained locks into a coarser table lock, which can reduce concurrency for other transactions needing access to the same table (Section 24.3).

Q92. How would you diagnose tempdb contention? Monitor `PAGELATCH` waits via DMVs, and consider configuring multiple equally-sized tempdb data files (Sections 40.2-40.3).

Q93. What's the difference between High Availability and a backup strategy, and why do you need both? HA minimizes downtime via standby replicas; backups protect against

data loss and catastrophic scenarios HA cannot address, like widespread corruption (Section 45.1).

Q94. How would you find and address missing indexes on a production system? Query `sys.dm_db_missing_index_details` and related DMVs, but evaluate each suggestion critically against the write-cost trade-off before implementing (Section 26.9).

Q95. What is RPO, and how does it inform backup scheduling? Recovery Point Objective — the maximum acceptable data loss — directly determines how frequently transaction log backups must run (Section 37.4).

Q96. How would you audit who has access to sensitive tables in a database? Query `sys.database_principals`, `sys.database_permissions`, and related system catalog views (Part 41).

Q97. What is the purpose of Dynamic Data Masking, and what does it not protect against? Obscures sensitive values in query results for unauthorized users; it does not encrypt the underlying stored data, and users with sufficient permission (or via certain workarounds) can potentially infer masked values, so it should not be relied upon as the sole protection for highly sensitive data (Section 35.5).

Q98. How would you plan for a planned failover using Always On Availability Groups? Verify replica synchronization state, perform a manual (planned) failover during a maintenance window, and confirm application connection strings correctly redirect to the new primary (Section 45.2).

Q99. What is the risk of over-indexing a heavily-written OLTP table? Every additional index adds write overhead to every `INSERT/UPDATE/DELETE` touching indexed columns, potentially degrading transactional throughput (Section 26.9).

Q100. How would you approach investigating a sudden spike in blocking on a production system? Identify the head blocker via blocking-chain DMVs, examine what it's waiting on and why, and determine whether it's a long-running transaction, a missing index causing lock escalation, or an isolation-level mismatch (Section 24.3-24.4).

60. SQL Server Coding Challenges

60.1 Reference Schema Used Throughout This Part

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY, FullName NVARCHAR(100), Department
NVARCHAR(50),
    Salary DECIMAL(10,2), ManagerID INT, HireDate DATE
);
CREATE TABLE Customers (CustomerID INT PRIMARY KEY, FullName
NVARCHAR(100), Email NVARCHAR(255));
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY, CustomerID INT, OrderDate DATE,
TotalAmount DECIMAL(12,2), Status NVARCHAR(20)
);
CREATE TABLE Products (ProductID INT PRIMARY KEY, ProductName
NVARCHAR(100), Category NVARCHAR(50), Price DECIMAL(10,2));
```

60.2 Beginner (1-15)

1. List all employees in the 'Sales' department.

```
SELECT * FROM Employees WHERE Department = 'Sales';
```

2. Find employees earning more than 50000.

```
SELECT * FROM Employees WHERE Salary > 50000;
```

3. Count the number of customers.

```
SELECT COUNT(*) FROM Customers;
```

4. List orders placed in January 2026.

```
SELECT * FROM Orders WHERE OrderDate >= '2026-01-01' AND OrderDate <
'2026-02-01';
```

5. Find the total revenue from all orders.

```
SELECT SUM(TotalAmount) FROM Orders;
```

6. List distinct departments.

```
SELECT DISTINCT Department FROM Employees;
```

7. Find the 5 most recent orders.

```
SELECT TOP 5 * FROM Orders ORDER BY OrderDate DESC;
```

8. Find employees hired before 2020.

```
SELECT * FROM Employees WHERE HireDate < '2020-01-01';
```

9. List products priced between 10 and 50.

```sql
SELECT * FROM Products WHERE Price BETWEEN 10 AND 50;
```

10. Find customers whose email ends in 'gmail.com'.

```sql
SELECT * FROM Customers WHERE Email LIKE '%gmail.com';
```

11. Sort employees by salary, highest first.

```sql
SELECT * FROM Employees ORDER BY Salary DESC;
```

12. Count orders per status.

```sql
SELECT Status, COUNT(*) AS Cnt FROM Orders GROUP BY Status;
```

13. Find the average order amount.

```sql
SELECT AVG(TotalAmount) FROM Orders;
```

14. Find employees with no manager (top of hierarchy).

```sql
SELECT * FROM Employees WHERE ManagerID IS NULL;
```

15. List all product categories with at least 3 products.

```sql
SELECT Category, COUNT(*) AS ProductCount FROM Products GROUP BY
Category HAVING COUNT(*) >= 3;
```

60.3 Intermediate (16-30)

16. Find the second-highest salary.

```sql
SELECT MAX(Salary) FROM Employees WHERE Salary < (SELECT MAX(Salary)
FROM Employees);
```

17. Find the nth-highest salary (n = 3).

```sql
SELECT DISTINCT Salary FROM (
    SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
FROM Employees
) t WHERE rnk = 3;
```

18. Find duplicate customer emails.

```sql
SELECT Email, COUNT(*) FROM Customers GROUP BY Email HAVING COUNT(*) >
1;
```

19. Remove duplicate customers, keeping the lowest CustomerID per email.

```sql
WITH Ranked AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY Email ORDER BY
CustomerID) AS rn FROM Customers
)
DELETE FROM Ranked WHERE rn > 1;
```

20. Find customers with no orders.

```sql
SELECT c.* FROM Customers c
WHERE NOT EXISTS (SELECT 1 FROM Orders o WHERE o.CustomerID =
c.CustomerID);
```

21. Find the highest-selling product (by revenue).

```sql
SELECT TOP 1 p.ProductName, SUM(oi.Quantity * oi.UnitPrice) AS Revenue
FROM OrderItems oi JOIN Products p ON oi.ProductID = p.ProductID
GROUP BY p.ProductName ORDER BY Revenue DESC;
```

22. Find each customer's latest order.

```sql
SELECT c.CustomerID, latest.OrderID, latest.OrderDate
FROM Customers c
CROSS APPLY (SELECT TOP 1 * FROM Orders o WHERE o.CustomerID =
c.CustomerID ORDER BY OrderDate DESC) AS latest;
```

23. Find employees earning more than their manager.

```sql
SELECT e.FullName AS Employee, e.Salary, m.FullName AS Manager,
m.Salary AS ManagerSalary
FROM Employees e JOIN Employees m ON e.ManagerID = m.EmployeeID
WHERE e.Salary > m.Salary;
```

24. Calculate a running total of order amounts by date.

```sql
SELECT OrderDate, TotalAmount,
    SUM(TotalAmount) OVER (ORDER BY OrderDate ROWS UNBOUNDED
PRECEDING) AS RunningTotal
FROM Orders;
```

25. Find the top 3 products per category.

```sql
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY Category ORDER BY Price
DESC) AS rn FROM Products
) t WHERE rn <= 3;
```

26. Calculate month-over-month order count growth.

```sql
WITH Monthly AS (
    SELECT FORMAT(OrderDate,'yyyy-MM') AS Ym, COUNT(*) AS Cnt FROM
Orders GROUP BY FORMAT(OrderDate,'yyyy-MM')
)
SELECT Ym, Cnt, Cnt - LAG(Cnt) OVER (ORDER BY Ym) AS Growth FROM
Monthly;
```

27. Find employees whose salary is above their department's average.

```sql
SELECT e.* FROM Employees e
WHERE e.Salary > (SELECT AVG(Salary) FROM Employees e2 WHERE
e2.Department = e.Department);
```

28. List customers along with their total number of orders (including zero).

```
SELECT c.CustomerID, c.FullName, COUNT(o.OrderID) AS OrderCount
FROM Customers c LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.FullName;
```

29. Find the department with the highest total payroll.

```
SELECT TOP 1 Department, SUM(Salary) AS TotalPayroll FROM Employees
GROUP BY Department ORDER BY TotalPayroll DESC;
```

30. Find orders placed on a weekend.

```
SELECT * FROM Orders WHERE DATENAME(WEEKDAY, OrderDate) IN
('Saturday', 'Sunday');
```

60.4 Advanced (31-50)

31. Detect consecutive days with orders for a given customer.

```
WITH CustomerDates AS (
    SELECT DISTINCT CustomerID, OrderDate,
        DATEDIFF(DAY, 0, OrderDate) - ROW_NUMBER() OVER (PARTITION BY
CustomerID ORDER BY OrderDate) AS grp
    FROM Orders
)
SELECT CustomerID, MIN(OrderDate) AS StreakStart, MAX(OrderDate) AS
StreakEnd, COUNT(*) AS StreakLength
FROM CustomerDates GROUP BY CustomerID, grp HAVING COUNT(*) >= 2;
```

32. Identify gaps in a sequence of dates (missing days with no orders at all).

```
WITH AllDates AS (
    SELECT CAST(MIN(OrderDate) AS DATE) AS d FROM Orders
    UNION ALL
    SELECT DATEADD(DAY, 1, d) FROM AllDates WHERE d < (SELECT
MAX(OrderDate) FROM Orders)
)
SELECT d AS MissingDate FROM AllDates
WHERE NOT EXISTS (SELECT 1 FROM Orders o WHERE o.OrderDate =
AllDates.d)
OPTION (MAXRECURSION 366);
```

33. Calculate year-over-year revenue growth percentage.

```
WITH Yearly AS (SELECT YEAR(OrderDate) AS Yr, SUM(TotalAmount) AS Rev
FROM Orders GROUP BY YEAR(OrderDate))
SELECT Yr, Rev, (Rev - LAG(Rev) OVER (ORDER BY Yr)) * 100.0 /
NULLIF(LAG(Rev) OVER (ORDER BY Yr), 0) AS YoYGrowth
FROM Yearly;
```

34. Find the median order amount.

```sql
SELECT DISTINCT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY
TotalAmount) OVER () AS MedianAmount FROM Orders;
```

35. Pivot monthly revenue into columns per year.

```sql
SELECT * FROM (
    SELECT YEAR(OrderDate) AS Yr, MONTH(OrderDate) AS Mo, TotalAmount
FROM Orders
) src
PIVOT (SUM(TotalAmount) FOR Mo IN ([1],[2],[3],[4],[5],[6],[7],[8],
[9],[10],[11],[12])) AS pvt;
```

36. Find customers who ordered every product in a given category.

```sql
SELECT c.CustomerID
FROM Customers c
WHERE NOT EXISTS (
    SELECT p.ProductID FROM Products p WHERE p.Category =
'Electronics'
    EXCEPT
    SELECT oi.ProductID FROM Orders o JOIN OrderItems oi ON o.OrderID
= oi.OrderID WHERE o.CustomerID = c.CustomerID
);
```

37. Calculate a 7-day moving average of daily revenue.

```sql
WITH Daily AS (SELECT OrderDate, SUM(TotalAmount) AS Rev FROM Orders
GROUP BY OrderDate)
SELECT OrderDate, AVG(Rev) OVER (ORDER BY OrderDate ROWS BETWEEN 6
PRECEDING AND CURRENT ROW) AS MA7
FROM Daily;
```

38. Rank employees within department, breaking ties by hire date.

```sql
SELECT *, RANK() OVER (PARTITION BY Department ORDER BY Salary DESC,
HireDate ASC) AS Rnk FROM Employees;
```

39. Find the first order date and last order date per customer, and the gap in days.

```sql
SELECT CustomerID, MIN(OrderDate) AS First, MAX(OrderDate) AS Last,
    DATEDIFF(DAY, MIN(OrderDate), MAX(OrderDate)) AS DaySpan
FROM Orders GROUP BY CustomerID;
```

40. Find products never ordered.

```sql
SELECT p.* FROM Products p
WHERE NOT EXISTS (SELECT 1 FROM OrderItems oi WHERE oi.ProductID =
p.ProductID);
```

41. Compute each order's percentage of its customer's total spend.

```sql
SELECT OrderID, CustomerID, TotalAmount,
    TotalAmount * 100.0 / SUM(TotalAmount) OVER (PARTITION BY
```

```
CustomerID) AS PctOfCustomerTotal
FROM Orders;
```

42. Find the longest gap (in days) between consecutive orders for each customer.

```
WITH Gaps AS (
    SELECT CustomerID, OrderDate,
        DATEDIFF(DAY, LAG(OrderDate) OVER (PARTITION BY CustomerID
ORDER BY OrderDate), OrderDate) AS GapDays
    FROM Orders
)
SELECT CustomerID, MAX(GapDays) AS LongestGap FROM Gaps GROUP BY
CustomerID;
```

43. Build a recursive CTE listing every employee's full management chain.

```
WITH Chain AS (
    SELECT EmployeeID, FullName, ManagerID, CAST(FullName AS
NVARCHAR(MAX)) AS Path
    FROM Employees WHERE ManagerID IS NULL
    UNION ALL
    SELECT e.EmployeeID, e.FullName, e.ManagerID, CAST(c.Path + ' > '
+ e.FullName AS NVARCHAR(MAX))
    FROM Employees e JOIN Chain c ON e.ManagerID = c.EmployeeID
)
SELECT * FROM Chain;
```

44. Find customers whose spending increased every month for the last 3 months.

```
WITH Monthly AS (
    SELECT CustomerID, FORMAT(OrderDate,'yyyy-MM') AS Ym,
SUM(TotalAmount) AS Rev
    FROM Orders GROUP BY CustomerID, FORMAT(OrderDate,'yyyy-MM')
),
Trend AS (
    SELECT *, LAG(Rev,1) OVER (PARTITION BY CustomerID ORDER BY Ym) AS
Prev1,
              LAG(Rev,2) OVER (PARTITION BY CustomerID ORDER BY Ym) AS
Prev2
    FROM Monthly
)
SELECT DISTINCT CustomerID FROM Trend WHERE Rev > Prev1 AND Prev1 >
Prev2;
```

45. Use `OUTPUT` to log every row deleted from a cancelled-orders cleanup.

```
DELETE FROM Orders OUTPUT deleted.* INTO Audit.DeletedOrders WHERE
Status = 'Cancelled';
```

46. Find the 3 most improved products (biggest revenue increase year-over-year).

```
WITH Yearly AS (
    SELECT p.ProductID, YEAR(o.OrderDate) AS Yr,
SUM(oi.Quantity*oi.UnitPrice) AS Rev
    FROM OrderItems oi JOIN Orders o ON oi.OrderID=o.OrderID JOIN
Products p ON oi.ProductID=p.ProductID
    GROUP BY p.ProductID, YEAR(o.OrderDate)
)
SELECT TOP 3 ProductID, (Rev - LAG(Rev) OVER (PARTITION BY ProductID
ORDER BY Yr)) AS Increase
FROM Yearly ORDER BY Increase DESC;
```

47. Build a dynamic search procedure with optional filters (safe dynamic SQL).

```
CREATE PROCEDURE usp_SearchOrders @Status NVARCHAR(20)=NULL,
@MinAmount DECIMAL(12,2)=NULL AS
BEGIN
    DECLARE @sql NVARCHAR(MAX)=N'SELECT * FROM Orders WHERE 1=1';
    IF @Status IS NOT NULL SET @sql += N' AND Status=@St';
    IF @MinAmount IS NOT NULL SET @sql += N' AND TotalAmount>=@Amt';
    EXEC sp_executesql @sql, N'@St NVARCHAR(20), @Amt DECIMAL(12,2)',
@St=@Status, @Amt=@MinAmount;
END;
```

48. Compute a customer's rank by lifetime spend, and their percentile.

```
SELECT CustomerID, SUM(TotalAmount) AS LTV,
    PERCENT_RANK() OVER (ORDER BY SUM(TotalAmount)) AS Percentile
FROM Orders GROUP BY CustomerID;
```

49. Find orders where the total doesn't match the sum of its line items (data-quality check).

```
SELECT o.OrderID, o.TotalAmount, SUM(oi.Quantity*oi.UnitPrice) AS
CalculatedTotal
FROM Orders o JOIN OrderItems oi ON o.OrderID = oi.OrderID
GROUP BY o.OrderID, o.TotalAmount
HAVING o.TotalAmount <> SUM(oi.Quantity*oi.UnitPrice);
```

50. Build a query returning each customer's order history as JSON.

```
SELECT CustomerID,
    (SELECT OrderID, OrderDate, TotalAmount FROM Orders o WHERE
o.CustomerID = c.CustomerID FOR JSON PATH) AS OrdersJson
FROM Customers c;
```

60.5 Expert (51-65)

51. Implement SCD Type 2 logic for a customer region change.

```
UPDATE DimCustomer SET ValidTo = SYSDATETIME(), IsCurrent = 0
WHERE CustomerID = 101 AND IsCurrent = 1 AND Region <> 'West';
INSERT INTO DimCustomer (CustomerID, Region, ValidFrom, ValidTo,
```

```
IsCurrent)
SELECT 101, 'West', SYSDATETIME(), '9999-12-31', 1
WHERE NOT EXISTS (SELECT 1 FROM DimCustomer WHERE CustomerID=101 AND
IsCurrent=1 AND Region='West');
```

52. Write a query using `OPENJSON` to bulk-insert orders from a JSON array parameter.

```
DECLARE @json NVARCHAR(MAX) = N'[{"CustomerID":1,"Amount":50.0},
{"CustomerID":2,"Amount":75.0}]';
INSERT INTO Orders (CustomerID, TotalAmount, OrderDate, Status)
SELECT CustomerID, Amount, SYSDATETIME(), 'Pending'
FROM OPENJSON(@json) WITH (CustomerID INT '$.CustomerID', Amount
DECIMAL(10,2) '$.Amount');
```

53. Design and query a filtered covering index for "pending orders in the last 7 days".

```
CREATE INDEX IX_Orders_RecentPending ON Orders (OrderDate) INCLUDE
(CustomerID, TotalAmount)
WHERE Status = 'Pending';
SELECT CustomerID, TotalAmount FROM Orders
WHERE Status = 'Pending' AND OrderDate >= DATEADD(DAY,-
7,SYSDATETIME());
```

54. Write a recursive CTE to generate a full calendar year of dates.

```
WITH Dates AS (
    SELECT CAST('2026-01-01' AS DATE) AS d
    UNION ALL SELECT DATEADD(DAY,1,d) FROM Dates WHERE d < '2026-12-
31'
)
SELECT d FROM Dates OPTION (MAXRECURSION 366);
```

55. Build an audit trigger capturing before/after values for salary changes.

```
CREATE TRIGGER trg_Employees_SalaryAudit ON Employees AFTER UPDATE AS
BEGIN
    INSERT INTO SalaryAudit (EmployeeID, OldSalary, NewSalary,
ChangedAt)
    SELECT i.EmployeeID, d.Salary, i.Salary, SYSDATETIME()
    FROM inserted i JOIN deleted d ON i.EmployeeID=d.EmployeeID WHERE
i.Salary<>d.Salary;
END;
```

56. Write a query to detect potential duplicate transactions (same account, amount, within 1 minute).

```
SELECT a.TransactionID, b.TransactionID AS PossibleDuplicateOf
FROM TransactionLog a JOIN TransactionLog b
  ON a.AccountID=b.AccountID AND a.Amount=b.Amount AND
a.TransactionID<b.TransactionID
  AND ABS(DATEDIFF(SECOND,a.CreatedAt,b.CreatedAt)) <= 60;
```

57. Implement row-level security restricting sales reps to their own region.

```
CREATE FUNCTION fn_RegionPredicate(@Region NVARCHAR(50)) RETURNS TABLE
WITH SCHEMABINDING AS
RETURN SELECT 1 AS Ok WHERE @Region =
CAST(SESSION_CONTEXT(N'UserRegion') AS NVARCHAR(50));
CREATE SECURITY POLICY RegionFilter ADD FILTER PREDICATE
fn_RegionPredicate(Region) ON Customers;
```

58. Write a query using `CROSS APPLY` and a table-valued function to get each customer's top 2 categories.

```
SELECT c.CustomerID, t.Category, t.Total FROM Customers c
CROSS APPLY (
    SELECT TOP 2 p.Category, SUM(oi.Quantity*oi.UnitPrice) AS Total
    FROM Orders o JOIN OrderItems oi ON o.OrderID=oi.OrderID JOIN
Products p ON oi.ProductID=p.ProductID
    WHERE o.CustomerID = c.CustomerID GROUP BY p.Category ORDER BY
Total DESC
) t;
```

59. Build a stored procedure implementing safe upsert logic (no `MERGE`).

```
CREATE PROCEDURE usp_UpsertCustomer @CustomerID INT, @Email
NVARCHAR(255) AS
BEGIN
    IF EXISTS (SELECT 1 FROM Customers WHERE CustomerID=@CustomerID)
        UPDATE Customers SET Email=@Email WHERE
CustomerID=@CustomerID;
    ELSE
        INSERT INTO Customers (CustomerID, Email) VALUES (@CustomerID,
@Email);
END;
```

60. Write a query identifying orphaned OrderItems (referencing a deleted order).

```
SELECT oi.* FROM OrderItems oi
WHERE NOT EXISTS (SELECT 1 FROM Orders o WHERE o.OrderID =
oi.OrderID);
```

61. Compute a cohort retention matrix (cohort month x months-since-signup).

```
WITH Cohorts AS (SELECT CustomerID, MIN(OrderDate) AS FirstOrder FROM
Orders GROUP BY CustomerID),
Activity AS (SELECT o.CustomerID, o.OrderDate, c.FirstOrder FROM
Orders o JOIN Cohorts c ON o.CustomerID=c.CustomerID)
SELECT DATEFROMPARTS(YEAR(FirstOrder),MONTH(FirstOrder),1) AS
CohortMonth,
    DATEDIFF(MONTH, FirstOrder, OrderDate) AS MonthsSince,
COUNT(DISTINCT CustomerID) AS ActiveCustomers
FROM Activity GROUP BY
```

```
DATEFROMPARTS(YEAR(FirstOrder),MONTH(FirstOrder),1), DATEDIFF(MONTH,
FirstOrder, OrderDate);
```

62. Write a temporal-table query returning an employee's salary as of a specific past date.

```
SELECT * FROM Employees FOR SYSTEM_TIME AS OF '2025-06-01' WHERE
EmployeeID = 42;
```

63. Build a query using `STRING_AGG` to produce a comma-separated product list per order.

```
SELECT o.OrderID, STRING_AGG(p.ProductName, ', ') AS Products
FROM Orders o JOIN OrderItems oi ON o.OrderID=oi.OrderID JOIN Products
p ON oi.ProductID=p.ProductID
GROUP BY o.OrderID;
```

64. Diagnose and rewrite a non-SARGable query filtering on `CAST(OrderDate AS DATE) = @Today`.

```
-- Non-SARGable: WHERE CAST(OrderDate AS DATE) = @Today
-- SARGable rewrite:
DECLARE @Today DATE = CAST(SYSDATETIME() AS DATE);
SELECT * FROM Orders WHERE OrderDate >= @Today AND OrderDate <
DATEADD(DAY,1,@Today);
```

65. Write a query computing each product's contribution to cumulative 80% of revenue (Pareto).

```
WITH ProdRev AS (
    SELECT p.ProductID, SUM(oi.Quantity*oi.UnitPrice) AS Rev
    FROM OrderItems oi JOIN Products p ON oi.ProductID=p.ProductID
GROUP BY p.ProductID
),
Ranked AS (
    SELECT *, SUM(Rev) OVER (ORDER BY Rev DESC ROWS UNBOUNDED
PRECEDING) * 100.0
              / SUM(Rev) OVER () AS CumulativePct
    FROM ProdRev
)
SELECT * FROM Ranked WHERE CumulativePct <= 80;
```

60.6 Real-World Business Problems (66-75)

66. A finance team needs month-end account reconciliation: flag accounts where the sum of transactions doesn't match the recorded balance change.

```
SELECT AccountID FROM TransactionLog
GROUP BY AccountID
HAVING SUM(CASE WHEN TransactionType='Deposit' THEN Amount ELSE -
Amount END)
```

```
    <> (SELECT Balance FROM Accounts a WHERE a.AccountID =
TransactionLog.AccountID) - 0;
```

67. Marketing needs a list of customers to target: high past spend, but no order in 60 days.

```
SELECT CustomerID, SUM(TotalAmount) AS PastSpend, MAX(OrderDate) AS
LastOrder
FROM Orders GROUP BY CustomerID
HAVING SUM(TotalAmount) > 1000 AND MAX(OrderDate) < DATEADD(DAY,-
60,SYSDATETIME());
```

68. Operations wants a nightly job flagging inventory items below reorder threshold.

```
CREATE PROCEDURE usp_CheckLowStock AS
BEGIN
    SELECT WarehouseID, ProductID, QuantityOnHand FROM Inventory i
    WHERE QuantityOnHand < (SELECT ReorderThreshold FROM Products p
WHERE p.ProductID = i.ProductID);
END;
```

69. Compliance requires proving what an employee's salary was on any given past date (audit request).

```
SELECT * FROM Employees FOR SYSTEM_TIME AS OF @RequestedDate WHERE
EmployeeID = @EmployeeID;
```

70. The BI team needs a Power BI-ready view combining sales, customer, and date dimensions.

```
CREATE VIEW Reporting.vw_SalesFact WITH SCHEMABINDING AS
SELECT f.OrderKey, dc.Region, dd.Year, dd.Quarter, f.Amount
FROM Warehouse.FactOrders f
JOIN Warehouse.DimCustomer dc ON f.CustomerKey=dc.CustomerKey
JOIN Warehouse.DimDate dd ON f.DateKey=dd.DateKey;
```

71. Security requires masking customer SSNs for the customer-support team, but not for finance.

```
ALTER TABLE Customers ALTER COLUMN SSN ADD MASKED WITH
(FUNCTION='partial(0,"XXX-XX-",4)');
GRANT UNMASK TO FinanceRole;
```

72. A fraud-review team needs transactions exceeding 3 standard deviations from a customer's typical spend.

```
WITH Stats AS (SELECT CustomerID, AVG(TotalAmount) AS Avg,
STDEV(TotalAmount) AS Std FROM Orders GROUP BY CustomerID)
SELECT o.* FROM Orders o JOIN Stats s ON o.CustomerID=s.CustomerID
WHERE ABS(o.TotalAmount - s.Avg) > 3*s.Std;
```

73. The data team must archive orders older than 3 years without downtime on the live table (partition switch).

```
ALTER TABLE OrdersPartitioned SWITCH PARTITION 1 TO OrdersArchive_Old;
```

74. Customer success wants each account's health score based on recency, frequency, and monetary value (RFM).

```
SELECT CustomerID,
    DATEDIFF(DAY, MAX(OrderDate), SYSDATETIME()) AS Recency,
    COUNT(*) AS Frequency,
    SUM(TotalAmount) AS Monetary
FROM Orders GROUP BY CustomerID;
```

75. Engineering needs a safe, parameterized dynamic report endpoint accepting arbitrary column sort order.

```
CREATE PROCEDURE usp_GetOrdersSorted @SortColumn NVARCHAR(50) AS
BEGIN
    DECLARE @AllowedColumns TABLE (ColName NVARCHAR(50));
    INSERT INTO @AllowedColumns VALUES ('OrderDate'),('TotalAmount'),
('Status');
    IF NOT EXISTS (SELECT 1 FROM @AllowedColumns WHERE ColName =
@SortColumn)
        THROW 50001, 'Invalid sort column', 1;
    DECLARE @sql NVARCHAR(MAX) = N'SELECT * FROM Orders ORDER BY ' +
QUOTENAME(@SortColumn);
    EXEC sp_executesql @sql;
END;
```

*(Note challenge 75: because a column **name** cannot itself be parameterized via `sp_executesql` the way a value can, it is validated against an explicit allow-list first, then safely bracketed with `QUOTENAME()` — the correct, safe pattern for dynamic sorting/column selection, as opposed to dynamic data values, which always go through parameters directly.)*

61. Common SQL Server Mistakes

#	Mistake	Explanation / Fix
1	**Incorrect joins**	Joining on the wrong column, or omitting a join condition entirely (accidental cross join) — always verify row counts against expectations after writing a new join (Section 11.2).
2	**Duplicate rows from joins**	A one-to-many or many-to-many relationship silently inflating aggregate results (Section 11.2) — verify with an independent `COUNT(DISTINCT ...)` check.
3	**NULL mistakes**	Assuming `= NULL` works (it doesn't — use `IS NULL`); forgetting `NULL` breaks simple equality/inequality comparisons in unexpected ways (Section 12.4).
4	**Incorrect GROUP BY**	Including a non-aggregated column not in the `GROUP BY` list conceptually, or grouping by too many/few columns, silently changing result granularity.
5	**Incorrect HAVING**	Using `HAVING` for a row-level filter that belongs in `WHERE` (evaluated too late, after unnecessary aggregation work) — `HAVING` is for filtering *aggregated* results only.
6	**NOT IN + NULL**	Returns zero rows unexpectedly if the subquery result contains any `NULL` (Section 12.4) — use `NOT EXISTS` instead.
7	**Implicit conversion**	Comparing mismatched types (`varchar` vs `nvarchar`, string vs numeric) silently defeats

#	Mistake	Explanation / Fix
		index seeks (Section 29.4).
8	**Non-SARGable queries**	Wrapping indexed columns in functions in `WHERE` clauses (Section 29.4, 30.1).
9	**Poor indexing**	Missing indexes on frequently-filtered/joined columns, verified via execution plans (Part 29) rather than guessed.
10	**Over-indexing**	Too many indexes on a heavily-written table, degrading `INSERT`/`UPDATE`/`DELETE` performance (Section 26.9).
11	**Cursor overuse**	Row-by-row procedural logic where a set-based statement would work — almost always slower (Section 30.4).
12	**Dynamic SQL injection**	String-concatenating untrusted input into dynamic SQL instead of using `sp_executesql` parameters (Section 36.2).
13	**Transaction mistakes**	Forgetting to check `@@TRANCOUNT` before `ROLLBACK`, or leaving a transaction open (forgotten `COMMIT`) holding locks indefinitely.
14	**Blocking**	Long-running transactions holding locks longer than necessary, blocking unrelated queries (Section 24.3) — keep transactions as short as possible.
15	**Deadlocks**	Two transactions acquiring the same resources in a different order — standardize resource-acquisition order across the codebase to reduce risk (Section 24.3).
16	**Poor data types**	Using `money`/`float` for

#	Mistake	Explanation / Fix
		financial values instead of `decimal`; oversized `nvarchar` columns with no realistic bound; `datetime` instead of `datetime2` in new development (Part 4).
17	**Missing constraints**	Relying on application code alone to enforce data integrity that a `CHECK`/`FOREIGN KEY` constraint should guarantee at the database level (Section 5.6).
18	**Bad normalization**	Either under-normalizing (redundant, inconsistent data) or over-normalizing an OLTP schema into excessive joins for simple queries (Part 34).
19	**MERGE misuse**	See Section 61.1 below — a dedicated, detailed treatment given the specific historical concerns.

61.1 MERGE — Historical Concerns and Safer Alternatives

```
-- MERGE - concise, but has had documented concurrency-related
correctness issues
-- under certain isolation levels and concurrent execution, plus
subtle behavior around
-- triggers and constraint enforcement that have caused real
production bugs historically
MERGE INTO Sales.Customers AS target
USING Staging.CustomerUpdates AS source ON target.CustomerID =
source.CustomerID
WHEN MATCHED THEN UPDATE SET target.Email = source.Email
WHEN NOT MATCHED THEN INSERT (CustomerID, Email) VALUES
(source.CustomerID, source.Email);

-- SAFER ALTERNATIVE - explicit, separate UPDATE and INSERT
statements, in one transaction
BEGIN TRANSACTION;
    UPDATE c SET c.Email = s.Email
    FROM Sales.Customers c JOIN Staging.CustomerUpdates s ON
c.CustomerID = s.CustomerID;

    INSERT INTO Sales.Customers (CustomerID, Email)
```

```
    SELECT s.CustomerID, s.Email FROM Staging.CustomerUpdates s
    WHERE NOT EXISTS (SELECT 1 FROM Sales.Customers c WHERE
c.CustomerID = s.CustomerID);
COMMIT TRANSACTION;
```

MERGE has had well-documented issues (particularly under Read Committed isolation without RCSI, and around trigger-firing behavior and unique-constraint violations under concurrent load) that have caused genuine, hard-to-reproduce production bugs — Microsoft's own documentation has historically carried explicit cautionary notes about MERGE's concurrency behavior. **Professional guidance:** for simple upsert logic, prefer explicit separate UPDATE/INSERT statements wrapped in a transaction (as shown), which is more verbose but has fully predictable, well-understood behavior; reserve MERGE for scenarios where its specific capabilities (matching on multiple conditions, WHEN NOT MATCHED BY SOURCE for deletions) are genuinely needed and the concurrency profile of the operation is well understood and low-risk (e.g. a single-threaded nightly batch job, not a highly concurrent OLTP path).

---

62. Professional SQL Server Best Practices

Category	Checklist
Naming conventions	Consistent, descriptive schema/table/column names; a documented convention for constraint names (PK_, FK_, CK_, IX_ prefixes as used throughout this handbook).
Data types	decimal for money, never float/money; datetime2 over datetime; nvarchar sized realistically, never blanket nvarchar(max) by default (Part 4).
Constraints	Every business rule expressible as a constraint enforced at the database level, not application code alone (Section 5.6).
Indexing	Deliberate clustered index choice; covering indexes for known hot query paths; regular review against actual query patterns, not guesswork (Part 26).
Query design	SARGable predicates; select only needed columns; set-based over cursors (Part 30).
Transactions	Kept as short as possible; explicit TRY/CATCH with proper @@TRANCOUNT checking (Parts 23, 25).
Error handling	THROW preferred over RAISERROR; meaningful custom error messages/numbers (Part 25).
Security	Least privilege; parameterized/dynamic SQL done safely; sensitive data

Category	Checklist
	masked/encrypted appropriately (Parts 35-36).
Backups	Recovery model matched to actual RPO/RTO requirements; backups tested via actual restores, not just assumed to work (Part 37).
Documentation	Schema and complex procedures documented; system-catalog-driven auditing queries maintained (Part 41).
Performance	Baseline metrics captured (`SET STATISTICS IO/TIME`); execution plans reviewed for genuinely slow queries before guessing at fixes (Parts 28-30).
Testing	Database changes tested in a non-production environment first, including restore drills; stored procedures covered by automated tests where practical (see the Testing and Debugging handbook for the application-layer testing discipline this pairs with).
Deployment	Schema changes version-controlled and deployed via repeatable scripts/migrations, not ad-hoc manual changes directly against production.
Monitoring	Blocking, deadlocks, tempdb contention, and long-running queries monitored proactively, not only investigated reactively after a user complaint.
Disaster recovery	HA and backup strategy both explicitly planned and periodically tested — never assumed to work untested (Part 45).

63. SQL Server Cheat Sheet

63.1 Database and Table Commands

```
CREATE DATABASE db; ALTER DATABASE db SET RECOVERY FULL; DROP DATABASE
db;
CREATE SCHEMA sch;
CREATE TABLE sch.t (id INT IDENTITY PRIMARY KEY, col NVARCHAR(50) NOT
NULL);
ALTER TABLE sch.t ADD col2 INT NULL;
TRUNCATE TABLE sch.t; DROP TABLE sch.t;
```

63.2 CRUD

```
INSERT INTO t (col) VALUES ('x');
SELECT * FROM t;
UPDATE t SET col = 'y' WHERE id = 1;
DELETE FROM t WHERE id = 1;
INSERT INTO t (col) OUTPUT inserted.id VALUES ('x');
```

63.3 Filtering, Joins, Aggregation

```
SELECT TOP 10 * FROM t WHERE col LIKE 'A%' ORDER BY id OFFSET 0 ROWS
FETCH NEXT 10 ROWS ONLY;
SELECT a.*, b.* FROM a JOIN b ON a.id = b.a_id;
SELECT col, COUNT(*) FROM t GROUP BY col HAVING COUNT(*) > 1;
```

63.4 Subqueries and CTEs

```
SELECT * FROM t WHERE EXISTS (SELECT 1 FROM other o WHERE o.t_id =
t.id);
WITH cte AS (SELECT * FROM t WHERE col > 0) SELECT * FROM cte;
```

63.5 Window Functions

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY grp ORDER BY val DESC) AS
rn,
    LAG(val) OVER (ORDER BY dt), SUM(val) OVER (ORDER BY dt ROWS
UNBOUNDED PRECEDING)
FROM t;
```

63.6 Temporary Objects

```
CREATE TABLE #tmp (id INT); DECLARE @tv TABLE (id INT);
```

63.7 Functions, Procedures, Views, Triggers

```
CREATE FUNCTION dbo.f(@x INT) RETURNS INT AS BEGIN RETURN @x*2 END;
CREATE PROCEDURE dbo.p @x INT AS BEGIN SELECT @x END;
CREATE VIEW dbo.v AS SELECT * FROM t;
CREATE TRIGGER trg ON t AFTER INSERT AS BEGIN /* ... */ END;
```

63.8 Transactions

```
BEGIN TRANSACTION; SAVE TRANSACTION sp1; COMMIT TRANSACTION; ROLLBACK
TRANSACTION;
```

63.9 Indexes

```
CREATE CLUSTERED INDEX ix ON t (col);
CREATE NONCLUSTERED INDEX ix2 ON t (col) INCLUDE (col2) WHERE col3 =
1;
CREATE CLUSTERED COLUMNSTORE INDEX ccx ON factTable;
```

63.10 JSON

```
SELECT JSON_VALUE(col,'$.a'), JSON_QUERY(col,'$.b') FROM t;
SELECT * FROM OPENJSON(@json) WITH (a INT '$.a');
SELECT * FROM t FOR JSON PATH;
```

63.11 Metadata

```
SELECT * FROM sys.tables; SELECT * FROM sys.columns WHERE object_id =
OBJECT_ID('t');
```

63.12 Backup / Restore

```
BACKUP DATABASE db TO DISK='d.bak'; BACKUP LOG db TO DISK='d.trn';
RESTORE DATABASE db FROM DISK='d.bak' WITH NORECOVERY;
RESTORE LOG db FROM DISK='d.trn' WITH RECOVERY, STOPAT='2026-01-
01T00:00:00';
```

63.13 Performance Analysis

```
SET STATISTICS IO ON; SET STATISTICS TIME ON;
SELECT * FROM sys.dm_db_missing_index_details;
```

63.14 Security

```
CREATE LOGIN l WITH PASSWORD='...'; CREATE USER u FOR LOGIN l;
GRANT SELECT ON t TO u; DENY DELETE ON t TO u; REVOKE INSERT ON t FROM
u;
```

63.15 Python Integration

```
import pyodbc, pandas as pd
from sqlalchemy import create_engine, text
engine = create_engine('mssql+pyodbc://user:pass@server/db?
driver=ODBC+Driver+18+for+SQL+Server')
df = pd.read_sql("SELECT * FROM t", engine)
df.to_sql('t2', engine, if_exists='replace', index=False)
```

63.16 Top 100 SQL Server Commands and Patterns Every Professional Should Know

SELECT · INSERT · UPDATE · DELETE · MERGE (cautiously) · TOP · OFFSET/FETCH · WHERE · ORDER BY · GROUP BY · HAVING · DISTINCT · JOIN (all 6 types) · UNION/UNION ALL · INTERSECT/EXCEPT · WITH (CTE) · Recursive CTE · OVER/PARTITION BY · ROW_NUMBER/RANK/DENSE_RANK · LAG/LEAD · CASE WHEN · ISNULL/COALESCE/NULLIF · CAST/CONVERT · TRY_CAST/TRY_CONVERT · CROSS APPLY/OUTER APPLY · PIVOT/UNPIVOT · IDENTITY · SEQUENCE/NEXT VALUE FOR · PRIMARY KEY/FOREIGN KEY/CHECK/UNIQUE/DEFAULT · CREATE VIEW (+ indexed views) · CREATE PROCEDURE/EXEC · CREATE FUNCTION (scalar/TVF) · CREATE TRIGGER · BEGIN TRANSACTION/COMMIT/ROLLBACK/SAVE TRANSACTION · SET TRANSACTION ISOLATION LEVEL · BEGIN TRY/BEGIN CATCH/THROW · CREATE INDEX (clustered/nonclustered/columnstore/filtered/covering) · SET STATISTICS IO/TIME ON · sp_executesql · QUOTENAME · STRING_AGG/STRING_SPLIT · CONCAT/CONCAT_WS · LEN/LEFT/RIGHT/SUBSTRING/CHARINDEX/PATINDEX/REPLACE · GETDATE/SYSDATETIME/DATEADD/DATEDIFF/DATEPART/EOMONTH · ISJSON/JSON_VALUE/JSON_QUERY/JSON_MODIFY/OPENJSON/FOR JSON · xml type methods/FOR XML · GRANT/DENY/REVOKE · Row-Level Security (CREATE SECURITY POLICY) · Dynamic Data Masking (MASKED WITH) · TDE/Always Encrypted concepts · BACKUP DATABASE/BACKUP LOG/RESTORE DATABASE/RESTORE LOG · sp_add_job/sp_add_jobstep/sp_add_schedule · BULK INSERT/OPENROWSET/bcp · sys.tables/sys.columns/sys.indexes/sys.objects/sys.sql_modules · System-

versioned temporal tables (`FOR SYSTEM_TIME`) · CDC (`sys.sp_cdc_enable_table`) · Change Tracking (`CHANGETABLE`) · `CREATE PARTITION FUNCTION/SCHEME/SWITCH` · Always On Availability Groups concepts · Replication (publisher/distributor/subscriber) concepts · Star schema (fact/dimension) design · SCD Type 2 pattern · `pyodbc`/SQLAlchemy/`pandas.read_sql`/`to_sql` · `#temp`/`##temp`/table variables · `TABLESAMPLE`/`ORDER BY NEWID()` · `PERCENTILE_CONT`/`PERCENT_RANK`/`NTILE` · `FIRST_VALUE`/`LAST_VALUE` · Index seek/scan/key lookup diagnosis · SARGability rewrites · Deadlock/blocking diagnosis · `tempdb` monitoring concepts.

Recommended Learning Roadmap

Given your existing PostgreSQL/MySQL foundation, this is the suggested order to work through SQL Server/T-SQL specifically:

Stage 1 — Orientation (Parts 1-4): Architecture, tools, database/schema management, data types. Spend the least time here on concepts you already know from PostgreSQL/MySQL; focus on the `[T-SQL]`/`[SQL Server Feature]`-labeled differences.

Stage 2 — Core T-SQL Fluency (Parts 5-13): Table design, CRUD, querying, functions, conditional logic, joins, subqueries, CTEs — this is where T-SQL's syntax differences from your existing knowledge concentrate most heavily (`TOP`, `OUTPUT`, `ISNULL`, date functions).

Stage 3 — SQL-Server-Distinctive Features (Parts 14-22): Window functions, temp objects, `APPLY`, `PIVOT`/`UNPIVOT`, set operations, views, stored procedures, UDFs, triggers — the features most likely to differ meaningfully in capability or syntax from PostgreSQL/MySQL equivalents.

Stage 4 — Transactions, Performance, and Indexing (Parts 23-30): ACID, isolation levels, error handling, indexing, execution plans, and optimization — the single highest-value stage for becoming genuinely production-capable, and worth revisiting repeatedly as real query problems arise.

Stage 5 — Specialized Data Handling (Parts 31-34): JSON, XML, data cleaning, normalization — practical, frequently-needed skills.

Stage 6 — Administration and Enterprise Operations (Parts 35-46): Security, dynamic SQL, backup/restore, Agent, import/export, tempdb, system catalog, temporal tables, CDC, partitioning, HA, replication — essential for DBA-track roles, valuable context even for developer-track roles.

Stage 7 — Analytics, Engineering, and Integration (Parts 47-54): Data analytics, data science, Python integration, data engineering, warehousing, Power BI, backend development, AI/ML — apply everything learned so far to your actual professional domain (Data Analyst, Data Scientist, Data Engineer, or Backend Developer).

Stage 8 — Cross-Database Mastery (Parts 55-56): Now that SQL Server is solid, consolidate the comparison with PostgreSQL/MySQL explicitly, cementing translation fluency across all three.

Stage 9 — Applied Practice (Parts 57-60): Build the 10 projects and capstone; work through the interview questions and coding challenges under realistic time pressure, simulating actual technical interviews.

Stage 10 — Professional Polish (Parts 61-63): Internalize the common-mistakes list and best-practices checklist as ongoing references, and keep the cheat sheet at hand for daily work.

Conclusion

SQL Server, at its foundation, solves the exact same relational problems PostgreSQL and MySQL solve — and the deep conceptual fluency you already bring from those systems transfers directly: relational modeling, joins, transactions, indexing theory, and SQL's declarative query logic are not being re-learned here, only re-expressed through T-SQL's particular syntax and SQL Server's particular engine behavior. What this handbook has tried to make explicit, at every turn, is precisely where that transfer holds cleanly (`CASE`, window functions, CTEs — nearly identical everywhere) and where it does not (`TOP` vs `LIMIT`, lock-based vs MVCC-based concurrency by default, `IDENTITY/SEQUENCE`, the complete absence of native JSON/regex types, and SQL Server's genuinely distinctive built-in capabilities like columnstore indexes, temporal tables, and Always On Availability Groups).

Professional SQL Server fluency is ultimately less about memorizing every function in Part 9 or every DMV in Part 41, and more about the same disciplined thinking that likely already serves you well in PostgreSQL and MySQL: understand what a query logically needs to compute (Part 7's logical processing order), verify how the engine actually executes it (Parts 28-29's execution-plan literacy), and choose the right tool for the right layer of the stack — SQL Server for set-based data work and transactional integrity, Python for statistical modeling, Power BI for the final presentation layer (Parts 47-54's integration guidance) — rather than forcing any one layer to do work another is better suited for.

Returning to the best-practices checklist in Part 62 and the cheat sheet in Part 63 as ongoing references, working consistently through the projects in Part 57 and the capstone in Part 58, and revisiting the PostgreSQL/MySQL comparison in Part 55 whenever a new T-SQL feature's "does this exist elsewhere, and how" question arises, is the most direct path from your existing multi-database foundation to genuine SQL Server and T-SQL professional mastery.

End of Document

Kajola Gbenga | Data Scientist & AI Engineer